# 1. Vad är en databas?

Vi människor har alltid behövt hålla reda på saker. Vilka varor finns i lager? Vem äger ett hus? Hur mycket pengar finns på ett konto? Vilka elever går i en viss klass?

Vi kan skriva ner informationen på papper, spara den i filer eller hålla den i minnet. Men när mängden information växer behöver vi kunna organisera den så att den blir enkel att hitta, använda och förändra.

Det är i grunden vad databaser handlar om.

## Grund

### Data, nycklar och värden

Vi kan börja med något mycket enkelt:

```
Förnamn: Benjamin
```

Här har vi en **nyckel** (`Förnamn`) och ett **värde** (`Benjamin`).

Nyckeln beskriver vad informationen handlar om och värdet innehåller själva uppgiften.

Om vi samlar flera sådana uppgifter kan vi beskriva exempelvis en kund i en bokhandel:

```
Kund:
    Förnamn: Benjamin
    Efternamn: Berglund
    Telefon: 0123–456789
    Adress:
        Gata: Hemmagatan 1
        Postnummer: 124 56
        Ort: Hemmavan
```

Nu har vi skapat en enkel datastruktur. Vi har också grupperat några av uppgifterna: gatan, postnumret och orten beskriver tillsammans en adress.

Samma information kan representeras på flera olika sätt.

I JavaScript kan vi exempelvis skriva:

```
const customer = {
    firstName: "Benjamin",
    lastName: "Berglund",
    phone: "0123-456789",
    address: {
        street: "Hemmagatan 1",
        postalCode: "124 56",
        city: "Hemmavan"
    }
};
```

Men vi skulle också kunna organisera informationen i en tabell.

## Tabeller

| Förnamn | Efternamn | Telefon | Ort |
|---|---|---|---|
| Benjamin | Berglund | 0123-456789 | Hemmavan |
| Olof | Olofsson | 0231-564789 | Storfors |
| Anaïs | Martin | +43 21-976 542 | Loire |

Här beskriver varje **rad** en kund, och varje **kolumn** en viss typ av uppgift om kunden.

Vi kan förstå tabellen genom tre enkla begrepp:

- **Tabell:** En samling uppgifter som hör ihop.
- **Rad:** En enskild post, exempelvis en kund.
- **Kolumn:** En viss egenskap hos posterna, exempelvis förnamn.

I en relationell databas bygger vi upp vår information med sådana tabeller.

## Vad gör detta till en databas?

En tabell i ett dokument eller kalkylblad är inte automatiskt en databas.

En **databas** är en organiserad samling data som kan lagras, hanteras och användas. För att arbeta med den använder vi vanligtvis ett **databashanteringssystem** (DBMS, Database Management System).

I relationella databaser använder vi frågespråket **SQL (Structured Query Language)** för att läsa, lägga till, förändra och ta bort information. SQL är utformat för att likna vanlig engelska, samtidigt som språket har en tydlig struktur som datorer kan tolka.

Exempel på databashanteringssystem som använder SQL är MySQL, PostgreSQL, SQLite och Microsoft SQL Server.

Databashanteraren hjälper oss bland annat att:

- Lagra och hämta information.
- Lägga till, ändra och ta bort uppgifter.
- Söka efter information som uppfyller vissa villkor.
- Kontrollera att informationen följer bestämda regler.
- Hantera att flera användare arbetar med samma information.

Skillnaden mellan databasen och databashanteraren kan jämföras med skillnaden mellan ett dokument och programmet som hanterar dokumentet.

## Att ställa frågor till informationen

Anta att bokhandeln har en databas med tusentals kunder.

Vi kanske vill veta vilka kunder som bor i Hemmavan, eller vilka som har ett efternamn som börjar med B.

Vi skulle kunna läsa igenom alla poster manuellt, men det vore knappast praktiskt.

I stället kan vi ställa en fråga till databasen.

Med SQL kan en sådan fråga se ut så här:

```
SELECT *
FROM customers
WHERE city = 'Hemmavan';
```

Vi behöver inte förstå syntaxen än. Det viktiga är att vi beskriver **vilken information vi vill ha**, snarare än hur databashanteraren ska leta efter den.

Detta är en grundläggande idé bakom SQL.

## Relationer mellan data

Låt oss anta att bokhandeln också behöver hålla reda på kundernas adresser.

Vi skulle kunna lagra gatuadress, postnummer och ort tillsammans med varje kund. Men vad händer när en kund vill ha en separat leveransadress? Eller när adressuppgifter behöver förändras?

Ett annat sätt är att lagra kunder och adresser i separata tabeller och beskriva hur de hör ihop.

### Kunder (customers)

| id | name |
|---|---|
| 1 | Benjamin |
| 2 | Olof |

### Adresser (addresses)

| id | customer_id | street | city |
|---|---|---|---|
| 1 | 1 | Hemmagatan 1 | Hemmavan |
| 2 | 1 | Storgatan 4 | Storfors |
| 3 | 2 | Parkvägen 2 | Storfors |

Kolumnen `customer_id` innehåller kundens id och visar vilken kund varje adress hör till.

Nu kan en kund ha flera adresser, utan att vi behöver skapa fler adresskolumner i kundtabellen.

Detta är en av de viktigaste principerna bakom relationella databaser: vi kan organisera uppgifter om olika saker var för sig, samtidigt som vi bevarar sambanden mellan dem.

Vi kommer att återkomma till hur relationer fungerar i senare kapitel.

## Avancerat

### En databas är en modell av verkligheten

När vi lagrar information beskriver vi inte hela verkligheten. Vi väljer ut de egenskaper och samband som är relevanta för vårt syfte.

En kund har betydligt fler egenskaper än namn och adress. Men om vi bygger ett kundregister kanske det är allt vi behöver veta.

När bokhandeln senare ska hantera beställningar, produkter och betalningar behöver vi andra uppgifter och andra samband.

Det innebär att en databas inte bara är ett sätt att lagra information. **Den är också en modell av den del av verkligheten vi vill beskriva.**

Modellen påverkar vad vi senare kan fråga efter, beräkna och dra slutsatser om.

## Är alla databaser relationella?

Nej.

Information kan organiseras på olika sätt, och olika databastyper är lämpliga för olika problem.

Exempelvis:

| Databastyp | Grundläggande idé |
|---|---|
| Relationell | Information organiseras i tabeller med definierade samband. |
| Dokumentorienterad | Information organiseras som dokument, exempelvis JSON. |
| Nyckel-värde | Information hämtas och lagras med hjälp av identifierande nycklar. |
| Grafdatabas | Information representeras som noder och relationer mellan dessa. |

Det finns många fler varianter och kombinationer.

Den här boken handlar framför allt om **relationella databaser och SQL**, men många av resonemangen kring hur vi representerar och organiserar information är relevanta även för andra databastyper.

## Data och datatyper

Hittills har vi beskrivit information med text och tabeller. Men alla värden är inte av samma slag.

Exempelvis:

```
Namn: Anna
Ålder: 27
Pris: 149.50
Registrerad: 2026-01-15
Aktiv: Ja
```

Vi vill kunna göra olika saker med dessa värden.

Ett namn kan vi jämföra eller söka efter. En ålder kan vi räkna med. Ett datum kan vi jämföra med andra datum.

Databaser använder därför **datatyper** för att beskriva vilken sorts värden som kan lagras i en kolumn.

Det hjälper databashanteraren att tolka och bearbeta informationen på ett förutsägbart sätt.

Vi kommer att gå närmare in på datatyper när vi börjar skapa egna tabeller.

### Databasens ansvar

Vi har redan sett att en databas kan lagra information och besvara frågor.

Men vad händer om två användare försöker förändra samma uppgift samtidigt? Eller om en ändring avbryts innan den är färdig?

En databashanterare behöver kunna hantera sådana situationer utan att informationen blir motsägelsefull eller går förlorad.

Det är bland annat därför vi använder databashanterare i stället för att bara skriva och läsa vanliga textfiler.

Senare i boken kommer vi att undersöka hur databaser upprätthåller dataintegritet, hanterar transaktioner och gör information tillgänglig för flera användare.

## Reflektion

1. Vad är egentligen skillnaden mellan att lagra information och att organisera information?
2. Hur påverkas en datamodell av vad vi vill använda informationen till?

3. Finns det situationer där en vanlig fil är en bättre lösning än en databas?

# 2. Att förstå relationella data

En bokhandel behöver veta vilka kunder den har, var de bor och vart deras beställningar ska levereras. Informationen beskriver olika saker, men sakerna hör ihop.

När vi organiserar data behöver vi därför bestämma vad som är en självständig företeelse, vad som är en egenskap och vad som är ett samband. Valen påverkar vilka frågor vi kan besvara och hur lätt informationen går att förändra.

## Grund

### Entiteter och attribut

En **entitet** är något vi vill hålla reda på, exempelvis en kund eller en adress. Ett **attribut** är en egenskap som vi vill registrera om entiteten.

För en kund kanske vi vill spara namn och e-postadress. För en adress behöver vi gata, postnummer och ort.

**customers**

| id | name | email |
|---|---|---|
| 1 | Benjamin | benjamin@example.com |
| 2 | Olof | olof@example.com |
| 3 | Anaïs | anais@example.com |

En **entitet** hör till modellen av verkligheten; en **tabell** är vårt sätt att representera den i databasen.

## Identitet – vilken kund menar vi?

Det kan finnas två kunder med samma namn. Ett namn räcker alltså inte alltid för att identifiera en kund.

Vi kan därför tilldela varje kund ett eget id. Kolumnen `id` kan fungera som tabellens **primärnyckel** (*primary key*). Primärnyckelns värde måste vara unikt för varje rad och får inte vara `NULL`.

När vi vill hitta eller referera till en viss rad använder vi i första hand primärnyckeln. Skriver vi `WHERE id = 2` menar vi just Olofs rad, och därmed den information vi har lagrat om honom.

Primärnyckeln kan även bestå av flera kolumner, men här använder vi ett enda id.

## Var ska adresserna finnas?

Vi skulle kunna lägga adresserna direkt i kundtabellen.

| id | name | street | city |
|---|---|---|---|
| 1 | Benjamin | Hemmagatan 1 | Hemmavan |
| 2 | Olof | Parkvägen 2 | Storfors |

Men om Benjamin vill registrera både en fakturaadress och en leveransadress? Vi skulle kunna lägga till fler kolumner, men vad händer när vi behöver en tredje adress?

Vi kan i stället dela upp informationen i två tabeller.

### customers

| id | name |
|---|---|
| 1 | Benjamin |
| 2 | Olof |
| 3 | Anaïs |

**addresses**

| id | customer_id | street | postal_code | city |
|---|---|---|---|---|
| 1 | 1 | Hemmagatan 1 | 124 56 | Hemmavan |
| 2 | 1 | Storgatan 4 | 154 32 | Storfors |
| 3 | 2 | Parkvägen 2 | 154 32 | Storfors |

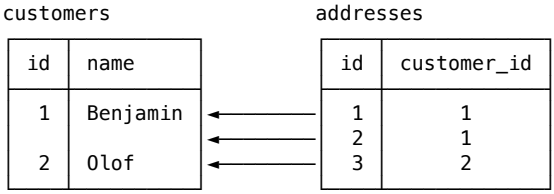
Kolumnen `customer_id` innehåller värdet från `customers.id`. Den kan definieras som en **främmande nyckel** (*foreign key*).

En sådan begränsning hjälper databasen att kontrollera att en adress inte hänvisar till en kund som saknas. Att bara lagra samma id-värde räcker inte för att databasen ska kontrollera sambandet.

## En-till-många (1:N)

I vår modell kan en kund ha flera adresser, men varje adresspost hör till en kund.

Det är en **en-till-många-relation** (1:N).




Notera att två adressposter i vårt exempel kan ha samma `customer_id`. Det är just så en kund kan ha flera adresser.

## En-till-en (1:1)

I en **en-till-en-relation** hör varje post ihop med högst en post i den andra tabellen, och tvärtom.

Bokhandeln kanske lagrar särskilda kundinställningar separat, med högst en inställningspost per kund.

Ofta skulle informationen kunna ligga i samma tabell. Uppdelningen kan ändå vara lämplig om uppgifterna har andra åtkomstbehov eller bara finns för vissa kunder.

## Många-till-många (M:N)

Anta att bokhandeln längre fram vill registrera produkter och deras författare.

En bok kan ha flera författare, och en författare kan ha skrivit flera böcker. Det är ett exempel på en **många-till-många-relation**.

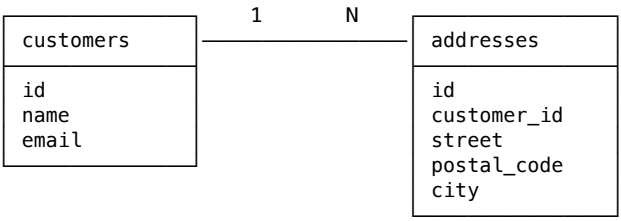
Den kan representeras med en **kopplingstabell**, exempelvis `product_authors`, som innehåller `product_id` och `author_id`.

Vi behöver inte skapa dessa tabeller än. Vi återkommer till dem när bokhandelns sortiment växer.

## Att rita en datamodell

Innan vi skapar tabeller kan vi rita upp vilka entiteter vi behöver och hur de hör ihop. Det kallas **ER-modellering** (*Entity–Relationship*).

Vår första modell kan beskrivas så här:




Papper och penna är ofta fullt tillräckligt. Det viktiga är att förstå modellen, inte att rita ett snyggt diagram.

## Avancerat

### Kardinalitet och valfrihet

När vi beskriver ett samband behöver vi ställa två frågor:

- **Hur många?** Kan en kund ha en eller flera adresser?
- **Är sambandet obligatoriskt?** Måste varje kund ha en registrerad adress?

En kund kan i vår modell ha noll, en eller flera adresser (0..N). Varje adresspost måste däremot höra till exakt en kund (1..1), om vi bestämmer att det är en regel i verksamheten.

Sådana regler är en del av datamodellen. De följer inte automatiskt av vad vi kallar tabellerna.

### När två kunder delar adress

Två kunder kan bo på samma gata och samma nummer. Betyder det att de ska dela *samma adresspost*?

Inte nödvändigtvis.

I vår första modell har varje adresspost en ägare genom `customer_id`. Två kunder på samma fysiska adress kan därför ha varsin adresspost med samma gata och postnummer.

Om vi senare vill hantera en gemensam adresspost som flera kunder refererar till behöver vi förändra modellen. Vi skulle exempelvis kunna införa kopplingstabellen `customer_addresses`.

Det är en bra illustration av varför **verksamhetens behov avgör hur informationen ska organiseras**.

## När ett samband får egna attribut

Anta att vi behöver markera vilken adress som används för leverans och vilken som används för fakturering.

Vi kan lägga till ett `address_type` i vår enkla adressmodell.

Men om flera kunder ska kunna dela samma adresspost blir adressens roll något som hör till *sambandet mellan kund och adress*, inte till själva gatan och postnumret. Då kan uppgiften ligga i kopplingstabellen.

Vi kommer tillbaka till detta när vi arbetar med datamodellering och normalisering.

## Från adressregister till bokhandel

Hittills har vi bara beskrivit kunder och adresser. Senare behöver vi även hålla reda på sortiment, produkter, författare, varukorgar och beställningar.

Det går att grovt skilja mellan:

- **Masterdata:** relativt varaktiga uppgifter om exempelvis kunder och produkter.
- **Referensdata:** kontrollerade värden, exempelvis produkttyper.
- **Transaktionsdata:** uppgifter om verksamhetens händelser, exempelvis ordrar och orderrader.

Det är användbara kategorier, men inte absoluta gränser. Författare är exempelvis oftast masterdata snarare än referensdata, eftersom de är självständiga företeelser med egna egenskaper.

Vi ska fortsätta bygga ut samma bokhandel allt eftersom vi behöver fler delar av SQL.

### Relation i två betydelser

I ett ER-diagram beskriver en *relation* ett **samband mellan entiteter**.

I den matematiska relationsmodellen betyder ordet något annat: en relation är en mängd tupler över definierade attribut. Den kan förenklat liknas vid en tabell, men SQL-tabeller följer inte alltid alla den matematiska modellens egenskaper.

Orden är besläktade, men betydelserna är inte identiska.

### Från modell till implementation

ER-diagrammet hjälper oss att förstå vad vi vill beskriva innan vi börjar skriva SQL.

Så småningom kommer vi att skapa tabeller, ange primärnycklar och främmande nycklar och använda `JOIN` för att hämta information som finns i flera tabeller.

## Reflektion

1. Är en adress en självständig entitet eller en egenskap hos en kund? Vad avgör det?
2. Varför bör vi identifiera en kund med ett id snarare än ett namn?
3. Hur behöver modellen förändras om flera kunder ska kunna använda samma adresspost?

# 3. Att komma igång med SQL

Vi har organiserat bokhandelns information i tabeller. Men en databas blir inte särskilt användbar om vi måste läsa igenom alla tabeller för att hitta något.

Vi behöver kunna ställa frågor: Vilka kunder har vi? Vad heter de? Vilken information har vi registrerat om dem?

SQL låter oss beskriva vilken information vi vill ha, utan att vi behöver tala om exakt hur databashanteraren ska leta fram den.

## Grund

### Ett språk för databaser

**SQL** betyder *Structured Query Language* och är ett språk för att arbeta med framför allt relationella databaser.

Vi använder SQL för att läsa information, men också för att lägga till, förändra och ta bort data samt skapa själva databasstrukturen.

Vi börjar med att läsa information.

En **Query** (*databasfråga*) är en instruktion som ställer en fråga till databasen, exempelvis genom `SELECT`. Vi använder **Queries** för att hämta och bearbeta information. När vi talar om en *fråga* i vanligt språk menar vi däremot det informationsbehov vi vill få svar på.

En Query är en sorts **SQL-sats** (*statement*). SQL-satser kan även förändra data eller databasens struktur, exempelvis med `INSERT` eller `CREATE TABLE`.

## Vår första Query

Vi har sedan tidigare en tabell med bokhandelns kunder:

### customers

| id | name | email |
|---|---|---|
| 1 | Benjamin | benjamin@example.com |
| 2 | Olof | olof@example.com |
| 3 | Anaïs | anais@example.com |

Om vi vill hämta kundernas namn skriver vi:

```
SELECT name
FROM customers;
```

Resultatet blir:

| name |
|---|
| Benjamin |
| Olof |
| Anaïs |

Vi kan läsa denna Query nästan som en vanlig mening:

- `SELECT name` – välj kolumnen `name`.
- `FROM customers` – från tabellen `customers`.

Semikolonet (`;`) markerar slutet på SQL-satsen. Det är vanligt att avsluta varje SQL-sats med semikolon, och nödvändigt när vi vill skilja flera satser åt i samma skript.

## Flera kolumner

Vill vi se både namn och e-postadress väljer vi båda kolumnerna:

```
SELECT name, email
FROM customers;
```

| name | email |
| --- | --- |
| Benjamin | benjamin@example.com |
| Olof | olof@example.com |
| Anaïs | anais@example.com |

Kolumnerna skiljs åt med kommatecken.

Vill vi hämta **alla kolumner** kan vi använda en asterisk (`*`):

```
SELECT *
FROM customers;
```

Det är praktiskt när vi utforskar en tabell. När vi vet vilka uppgifter vi behöver är det ofta bättre att ange just de kolumnerna.

## Att välja vissa rader

Ofta vill vi inte se alla kunder, utan bara de som uppfyller ett visst villkor.

```
SELECT name, email
FROM customers
WHERE id = 2;
```

| name | email |
| --- | --- |
| Olof | olof@example.com |

`WHERE` anger ett villkor som avgör vilka rader som ska ingå i resultatet.

Här letar vi efter kunden vars `id` är `2`. Som vi såg i förra kapitlet använder vi primärnyckeln för att identifiera en bestämd post.

Vi kan också jämföra text:

```
SELECT *
FROM customers
WHERE name = 'Olof';
```

Textvärden skrivs inom enkla citattecken. Tal skrivs normalt utan citattecken.

I nästa kapitel tittar vi närmare på villkor och hur vi kombinerar dem.

## SQL består av satser

En instruktion som databashanteraren kan utföra kallas en **SQL-sats** (*statement*).

`SELECT` är en sådan sats. Några andra vanliga satser är:

| Sats | Användning |
|---|---|
| `SELECT` | Läsa data |
| `INSERT` | Lägga till data |
| `UPDATE` | Ändra data |
| `DELETE` | Ta bort data |
| `CREATE TABLE` | Skapa en tabell |

De fyra första används ofta för att beskriva **CRUD**: *Create, Read, Update, Delete*.

Det är fyra grundläggande handlingar vi kan utföra på information: skapa, läsa, förändra och radera.

Vi ska arbeta mer med `INSERT`, `UPDATE` och `DELETE` längre fram. Tills vidare räcker det att känna igen deras syften.

## Läsbar SQL

SQL behöver inte skrivas med stora bokstäver:

```
select name from customers;
```

Men vi kommer att skriva SQL:s nyckelord med stora bokstäver och dela upp längre frågor på flera rader:

```
SELECT name, email
FROM customers
WHERE id = 2;
```

Det gör frågorna lättare att läsa, särskilt när de blir mer komplicerade.

Vi kan också lägga till kommentarer:

```
-- Hämta kunden med id 2
SELECT name, email
FROM customers;
```

En kommentar som börjar med `--` ignoreras av databashanteraren. [1]

Notera att kommentaren inte påverkar frågan: exemplet ovan hämtar fortfarande **alla** kunder. Kommentarer är beskrivningar, inte instruktioner.

## Avancerat

### Vi beskriver resultatet, inte arbetsgången

SQL är i huvudsak ett **deklarativt språk**.

Det betyder att vi talar om *vad* vi vill få fram, medan databashanteraren bestämmer *hur* informationen ska hittas.

När vi skriver:

```
SELECT name
FROM customers
WHERE id = 2;
```

beskriver vi vårt önskade resultat. Vi säger inte att databashanteraren ska läsa rad ett, sedan rad två och därefter sluta.

Databashanteraren kan välja mellan olika sätt att hitta informationen, exempelvis med hjälp av ett index. Vi ska undersöka detta senare i boken.

### En Query följer en struktur

En SQL-Query kan innehålla flera delar. En enkel mall är:

```
SELECT column_name
FROM table_name
WHERE condition;
```

Delarna har olika roller:

- `SELECT` beskriver vilka kolumner eller uttryck som ska ingå.
- `FROM` anger varifrån informationen hämtas.
- `WHERE` väljer vilka rader som ska tas med.

Det är viktigt att skilja mellan **ordningen vi skriver delarna** och **hur frågan logiskt behandlas**.

Vi skriver `SELECT` först, men för att kunna välja kolumner från rätt rader behöver databassystemet också ta hänsyn till `FROM` och `WHERE`. I en förenklad logisk beskrivning bestäms först datakällan, sedan filtreras raderna och därefter formas resultatets kolumner.

Den skillnaden blir användbar när vi längre fram arbetar med gruppering, beräkningar och flera tabeller.

## Namn och alias

Vi kan ge en kolumn ett annat namn i resultatet med `AS`:

```
SELECT name AS customer_name
FROM customers;
```

| customer_name |
|---|
| Benjamin |
| Olof |
| Anaïs |

Detta förändrar inte namnet på kolumnen i databasen. Det påverkar bara hur resultatet presenteras.

Vi kan även ge tabeller kortare namn, så kallade **alias**:

```
SELECT c.name, c.email
FROM customers AS c;
```

Prefixet `c.` talar om att kolumnerna kommer från tabellen som vi kallat `c`. Detta blir särskilt användbart när vi hämtar information från flera tabeller:

```
SELECT c.name, c.email, a.street, a.city
FROM customers AS c, addresses AS a
WHERE a.customer_id = c.id;
```

Här kopplar vi samman kunderna med deras adresser genom att jämföra `addresses.customer_id` med `customers.id`. Vi går igenom hur sådana kopplingar fungerar längre fram.

## SQL är ett språk med olika dialekter

SQL är standardiserat, men databassystem som MySQL, PostgreSQL, SQLite och SQL Server har också egna tillägg och skillnader.

Grundläggande satser som våra `SELECT`-exempel fungerar i regel på samma sätt. Andra konstruktioner kan ha olika syntax.

Vi kommer att använda standardnära SQL i boken och kommentera viktiga skillnader i fotnoter. Mer omfattande skillnader samlar vi i separata referenshäften.

## En Query förändrar inte data

Vår `SELECT`-fråga läser data utan att ändra den.

Andra satser, exempelvis `UPDATE` och `DELETE`, kan däremot förändra eller ta bort information permanent om ändringarna sparas.

Det är en viktig skillnad mellan att **fråga efter information** och att **förändra den**.

När vi kommer till dessa operationer ska vi därför också prata om villkor, dataintegritet och hur vi undviker oavsiktliga ändringar.

## Reflektion

1. Varför kan det vara värdefullt att beskriva *vilket resultat* vi vill ha, i stället för att bestämma exakt *hur* databasen ska hitta det?
2. Vad är skillnaden mellan att läsa data och att förändra data?
3. När kan `SELECT *` vara praktiskt, och varför kan det ibland vara bättre att välja kolumnerna uttryckligen?

# 4. Datatyper och värden

Ett värde betyder inte särskilt mycket utan ett sammanhang.

Talet `123` kan vara ett antal, ett pris, ett postnummer eller en identifierare. Samma värde kan representera helt olika saker beroende på vad vi har bestämt att det ska betyda.

En **datatyp** beskriver vilka slags värden vi kan hantera och vilka operationer som är möjliga med dem. Men det är sammanhanget som talar om vad värdet *betyder*.

Vissa datatyper är enkla, exempelvis heltal och text. Andra representerar mer sammansatt information, exempelvis datum, geografiska positioner eller JSON-strukturer. Och när vi bygger en datamodell kan vi själva beskriva mer komplexa företeelser, som en kund eller en produkt.

Att välja datatyper handlar därför inte bara om lagring. Det handlar också om **hur vi beskriver verkligheten**.

## Grund

### Samma tecken, olika betydelse

Titta på dessa värden:

```
123
'123'
```

Det första kan vi behandla som ett tal och det andra som text. De ser nästan likadana ut, men vi vill kunna göra olika saker med dem.

Med talet `123` kan vi räkna:

```
SELECT 123 + 10;
```

Det ger `133`.

Men om `'123'` är början på ett telefonnummer eller ett artikelnummer kanske det inte är meningsfullt att addera 10.

En datatyp hjälper oss att skilja mellan olika slags värden. **Vilken datatyp vi väljer ska bero på hur informationen ska användas, inte bara hur den ser ut.**

## Några vanliga datatyper

SQL har många datatyper. Vi börjar med några av de viktigaste.

| Datatyp | Används för | Exempel |
|---|---|---|
| `INTEGER` | Heltal | `12` |
| `DECIMAL` | Exakta decimaltal | `199.90` |
| `VARCHAR` | Text av varierande längd | `'Vägen hem'` |
| `CHAR` | Text med bestämd längd | `'SE'` |
| `BOOLEAN` | Sant eller falskt | `TRUE` |
| `DATE` | Datum | `'2001-05-12'` |

Vi kommer att arbeta med fler datatyper efter hand, men dessa räcker långt.

Datatypernas exakta namn och implementationer varierar mellan olika databassystem. Det gäller särskilt `BOOLEAN`.[2]

## Heltal och decimaltal

Antalet böcker i lager är ett heltal:

```
stock: 12
```

Ett pris kan däremot innehålla decimaler:

```
price: 199.90
```

Därför behöver vi skilja mellan heltal, exakta decimaltal och approximativa tal.

Vanliga typer är:

- `INTEGER` för heltal.
- `DECIMAL(10, 2)` för exakta decimaltal med upp till tio siffror, varav två efter decimaltecknet.
- `FLOAT` och `DOUBLE` för approximativa flyttal.

Vi brukar föredra `DECIMAL` framför flyttal när vi lagrar pengar eftersom vissa decimaltal inte kan representeras exakt som binära flyttal.

## Text är inte bara bokstäver

Vi lagrar namn och boktitlar som text:

```
title: Vägen hem
```

Men även uppgifter som bara består av siffror kan vara text.

Ett postnummer, exempelvis `01234`, är ingen storhet vi ska räkna med. Nollan i början är en del av beteckningen och får inte försvinna.

Detsamma gäller ofta telefonnummer, ISBN och andra identifierare.

För text använder vi bland annat `VARCHAR` och `CHAR`. Om vi exempelvis behöver lagra en titel med högst `200` tecken kan vi ange `VARCHAR(200)`.

Vi återkommer till hur detta används när vi börjar skapa tabeller.

## Datum och booleska värden

En kunds födelsedatum är inte bara en godtycklig textsträng. Vi vill kunna jämföra datum och beräkna tidsavstånd.

Därför använder vi en datumtyp, exempelvis `DATE`.

```
birthday: 1982-09-03
```

På samma sätt kan ett påstående vara sant eller falskt:

```
active: TRUE
```

Datatypen `BOOLEAN` uttrycker just detta.

Vi behöver inte gå in på datumformat, tidszoner eller datumberäkningar ännu. Det gör vi i kapitlet om datum och tid.

## NULL – när värdet saknas

I en databastabell kanske vi ännu inte känner till kundens födelsedatum.

SQL använder `NULL` för att ange att ett värde saknas.

Det är viktigt att skilja på dessa exempel:

```
age: 0
name: ''
birthday: NULL
```

`0` är ett tal. `''` är en tom textsträng. `NULL` betyder att det inte finns något känt värde i kolumnen.

Detta får betydelse när vi söker och räknar med data. Vi kommer tillbaka till `NULL` i kommande kapitel.

## Från värden till attribut

I bokhandeln vill vi beskriva en produkt med namn, pris och antal i lager:

```
Produkt:
    Titel: Vägen hem
    Pris: 200.00
    Lagersaldo: 12
```

Vi har då valt vilken information som ska höra till en produkt.

Det kan representeras som en tabell:

**products**

| id | title | price | stock |
|---|---|---|---|
| 1 | Vägen hem | 200.00 | 12 |
| 2 | Den blå staden | 150.00 | 0 |
| 3 | Vinterljus | 249.00 | 4 |

Varje kolumn behöver en datatyp:

| Attribut | Datatyp | Vad värdet betyder |
|---|---|---|
| `id` | `INTEGER` | Produktens identifierare |
| `title` | `VARCHAR(200)` | Produktens titel |
| `price` | `DECIMAL(10, 2)` | Produktens pris |
| `stock` | `INTEGER` | Antalet exemplar i lager |

Här har vi inte bara valt datatyper. Vi har också skapat en **modell av vad en produkt är** i just vår bokhandel.

## En giltig datatyp räcker inte

Datatypen `INTEGER` kan representera negativa tal. Men innebär det att ett lagersaldo på `−12` är rimligt?

Inte om vi bestämt att lagersaldot beskriver antalet exemplar som faktiskt finns tillgängliga.

Vi kan därför skilja mellan tre nivåer:

1. **Datatyp:** `INTEGER` – vi tillåter heltal.
2. **Betydelse:** `stock` – värdet beskriver antal exemplar.
3. **Regel:** Lagersaldot får inte vara negativt.

Vi skulle senare kunna uttrycka regeln i databasen med en begränsning, exempelvis `CHECK (stock >= 0)`.

Det visar en viktig skillnad: **Ett värde kan vara tekniskt giltigt för sin datatyp, men ändå vara felaktigt i verksamhetens sammanhang.**

## Avancerat

### Enkla och sammansatta värden

Ett heltal eller en textsträng är exempel på värden som vi vanligtvis behandlar som en enda enhet. De brukar kallas **skalära värden**.

Andra datatyper rymmer en tydligare intern struktur.

Exempelvis kan ett JSON-värde innehålla flera namngivna uppgifter:

```
{
  "street": "Hemmagatan 1",
  "postalCode": "124 56",
  "city": "Hemmavan"
}
```

En geografisk punkt kan beskrivas med koordinater. Ett datum har också en meningsfull intern uppbyggnad, även om `DATE` i SQL normalt används som ett skalärt värde.

Det är alltså inte alltid en skarp gräns mellan *enkel* och *komplex* information. Vi behöver skilja mellan hur människor uppfattar värdet, hur det representeras och hur databassystemet behandlar typen.

JSON, geometriska typer och andra utökningar varierar mellan SQL-dialekter. Vi kommer att undersöka dem mer ingående längre fram.

## Har vi skapat datatypen Produkt?

När vi bestämmer att en produkt har `title`, `price` och `stock` har vi i en begreppslig mening definierat en egen **sammansatt typ: Produkt**.

Vi har bestämt vilka egenskaper den har och vilka värden som kan vara rimliga.

Det betyder däremot inte att `CREATE TABLE products` automatiskt skapar en ny användardefinierad SQL-datatyp med namnet `Produkt`.

I SQL skiljer vi på datatyper, tabeller och de regler som hör till tabellernas kolumner. Vissa databassystem har dessutom särskilt stöd för egna sammansatta datatyper.[3]

Den viktiga poängen är att datamodellering också handlar om att definiera **vilka slags saker** vårt system ska kunna beskriva.

## Precision och ungefärliga värden

Ett pris som `199.90` bör kunna representeras på ett förutsägbart sätt.

Med `DECIMAL(10, 2)` beskriver vi ett exakt decimaltal med en bestämd precision och skala. Med `FLOAT` eller `DOUBLE` arbetar vi däremot med approximativa värden, vilket ofta är praktiskt för mätningar och vetenskapliga beräkningar.

Inget av alternativen är automatiskt bäst. Vi väljer efter vad värdet betyder och vad vi behöver kunna göra med det.

## När databasen omvandlar värden

Vi kan ibland behöva omvandla information från en datatyp till en annan.

```
SELECT CAST('123' AS INTEGER);
```

Här ber vi databasen att tolka texten `'123'` som ett heltal.

En del SQL-system gör även automatiska typomvandlingar. Det är bekvämt, men kan ibland dölja fel eller ge oväntade resultat.

Därför är det bra att förstå vilken datatyp ett värde har, även när databashanteraren verkar kunna gissa rätt.

## När samma värde får olika betydelse

Tänk dig att vi har dessa två attribut:

```
product_id: 123
stock: 123
```

Båda kan lagras som heltal. Men de beskriver olika saker.

Det ena är en **identifierare**, det andra en **kvantitet**.

Det är meningsfullt att beräkna `stock + 10`, men knappast att beräkna `product_id + 10` för att beskriva något om produkten.

Den skillnaden syns inte alltid i själva SQL-datatypen. Den finns i vår modell och i hur vi använder informationen.

## När regler blir en del av modellen

En produkt kanske måste ha en titel, ett pris som inte är negativt och ett lagersaldo som inte understiger noll.

Vi kan uttrycka flera sådana krav med datatyper och begränsningar:

```
CREATE TABLE products (
    id INTEGER PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    stock INTEGER NOT NULL,
    CHECK (price >= 0),
    CHECK (stock >= 0)
);
```

Vi behöver inte behärska `CREATE TABLE` ännu. Exemplet visar bara hur våra beslut om informationens betydelse senare kan bli **regler som databasen själv kontrollerar**.

Det betyder inte att alla verksamhetsregler hör hemma i databasen. Men regler som avgör vilka datatillstånd vi accepterar är ofta naturliga att uttrycka nära informationen.

Vi återkommer till detta när vi skapar tabeller och när vi diskuterar skillnaden mellan informationshantering och processer.

## Reflektion

1. Varför kan ett postnummer behöva lagras som text, trots att det bara innehåller siffror?
2. Är `-12` ett felaktigt heltal eller ett felaktigt lagersaldo? Vad är skillnaden?
3. Vad har vi egentligen bestämt när vi säger att vår datamodell innehåller en *Produkt* med titel, pris och lagersaldo?

# 5. Att hämta och filtrera data

Att ha mycket information är inte samma sak som att hitta det man behöver.

En bokhandel kanske har tusentals kunder, men just nu behöver vi bara veta vilka som bor i en viss stad. Eller vilka kunder som har ett namn som börjar på B.

Vi behöver kunna avgränsa informationen. Med SQL beskriver vi vilka uppgifter som är relevanta och låter databashanteraren hämta dem.

## Grund

### Att välja information med SELECT

Vi fortsätter med bokhandelns kunder. Vi har nu registrerat ytterligare några och lagt till deras födelsedatum i kolumnen `birthday`:

**customers**

| id | name | email | birthday |
|---|---|---|---|
| 1 | Benjamin | benjamin@example.com | 1968-04-12 |
| 2 | Olof | olof@example.com | 1982-09-03 |
| 3 | Anaïs | anais@example.com | 1995-06-21 |
| 4 | Beatrice | beatrice@example.com | 2000-02-15 |
| 5 | Anna | anna@example.com | 1975-11-08 |

Vi har redan sett hur vi kan hämta vissa kolumner:

```
SELECT name, email
FROM customers;
```

För att välja *vilka rader* som ska komma med använder vi `WHERE`.

## Villkor med WHERE

```
SELECT *
FROM customers
WHERE id = 2;
```

Resultat:

| id | name | email | birthday |
|---|---|---|---|
| 2 | Olof | olof@example.com | 1982-09-03 |

`WHERE` följs av ett **villkor**. En rad kommer med i resultatet när villkoret är uppfyllt.

Vi kan jämföra både tal och text:

```
SELECT name
FROM customers
WHERE name = 'Anna';
```

## Jämförelseoperatorer

De vanligaste operatorerna är:

| Operator | Betydelse |
|---|---|
| = | Lika med |
| <> | Inte lika med |
| > | Större än |
| < | Mindre än |
| >= | Större än eller lika med |
| <= | Mindre än eller lika med |

Exempelvis:

```
SELECT name, birthday
FROM customers
WHERE birthday > '1970-01-09';
```

ger oss kunderna som är födda efter den 9 januari 1970.

När vi jämför textvärden behöver vi vanligtvis skriva värdet inom enkla citattecken, som i `name = 'Anna'`.

Vi använder `<>` för ”inte lika med”. I flera databassystem fungerar även `!=`, men `<>` är den standardiserade formen.

## Kombinera villkor med AND och OR

Vi kan ställa flera krav samtidigt.

```
SELECT name, birthday
FROM customers
WHERE birthday >= '1980-01-01'
  AND birthday < '2000-01-01';
```

Här måste **båda** villkoren vara uppfyllda. Vi får Olof och Anaïs, som är födda under perioden 1980–1999.

Med `OR` räcker det att **minst ett** villkor är uppfyllt:

```
SELECT *
FROM customers
WHERE name = 'Anna' OR name = 'Olof';
```

Vi kan också använda `NOT` för att förneka ett villkor:

```
SELECT *
FROM customers
WHERE NOT name = 'Olof';
```

## Söka inom ett intervall

När vi vill hitta värden mellan två gränser kan vi använda `BETWEEN`:

```
SELECT name, birthday
FROM customers
WHERE birthday BETWEEN '1980-01-01' AND '1999-12-31';
```

Det motsvarar:

```
SELECT name, birthday
FROM customers
```

```
WHERE birthday >= '1980-01-01'
  AND birthday <= '1999-12-31';
```

**Båda gränserna ingår** i `BETWEEN`.

## Söka bland flera värden

Om vi vill hitta kunder med några särskilda id-värden kan vi använda `IN`:

```
SELECT *
FROM customers
WHERE id IN (1, 3, 5);
```

Det är ofta enklare att läsa än en lång serie villkor med `OR`.

## Söka efter delar av text

Vi kanske bara vet att kundens namn börjar med bokstaven B.

Då använder vi `LIKE`:

```
SELECT *
FROM customers
WHERE name LIKE 'B%';
```

Mönstret `'B%'` betyder att namnet börjar med B och kan följas av vilka tecken som helst.

| Mönster | Betydelse |
|---|---|
| `'B%'` | Börjar med B |
| `'%a'` | Slutar med a |
| `'%ann%'` | Innehåller ann |
| `'_n%'` | Har n som andra tecken |

`%` betyder noll eller fler tecken, medan `_` betyder exakt ett tecken.

Om stora och små bokstäver räknas som likvärdiga vid sökning beror bland annat på databassystemets inställningar för textjämförelser.[4]

## Sortera resultat med ORDER BY

SQL garanterar inte någon särskild radordning om vi inte uttryckligen begär den.

Vi kan sortera på namn:

```
SELECT *
FROM customers
ORDER BY name;
```

För fallande ordning använder vi `DESC`:

```
SELECT *
FROM customers
ORDER BY name DESC;
```

`ASC` betyder stigande ordning och är normalt standardvalet.

Vi kan också sortera på flera kolumner:

```
SELECT *
FROM customers
ORDER BY name, id;
```

Först sorterar vi på `name`. Om flera kunder har samma namn används `id` för att avgöra ordningen mellan dem.

## Begränsa antalet rader

Ibland vill vi bara få ett fåtal rader tillbaka.

I MySQL kan vi exempelvis skriva:

```
SELECT *
FROM customers
ORDER BY id
LIMIT 3;
```

Det ger de tre kunderna med lägst id.

Vi kan också hoppa över de första två och sedan hämta tre:

```
SELECT *
FROM customers
ORDER BY id
LIMIT 3 OFFSET 2;
```

I MySQL kan vi också skriva samma sak med två tal efter `LIMIT`:

```
SELECT *
FROM customers
ORDER BY id
LIMIT 2, 3;
```

Det första talet anger **hur många rader vi hoppar över** och det andra anger **hur många rader vi hämtar**. Alltså är `LIMIT 2, 3` likvärdigt med `LIMIT 3 OFFSET 2`.

Var uppmärksam på att talens ordning är omvänd mellan de två skrivsätten. Kommasyntaxen stöds även av SQLite, men inte av PostgreSQL. I den här boken föredrar vi normalt `LIMIT ... OFFSET ...` eftersom det tydligare visar vad talen betyder.

`LIMIT` används i flera populära SQL-dialekter, men är inte den standardiserade syntaxen.[5]

Att sortera innan vi begränsar resultatet är viktigt. Annars vet vi inte *vilka* tre rader vi får.

## Kombinera allt

Nu kan vi formulera mer specifika frågor:

```
SELECT name, email
FROM customers
WHERE name LIKE 'B%'
ORDER BY name
LIMIT 3;
```

Vi hämtar namn och e-postadress för de kunder vars namn börjar på B, sorterar dem alfabetiskt och tar högst tre rader.

Notera ordningen:

```
SELECT ...
FROM ...
WHERE ...
ORDER BY ...
LIMIT ...;
```

Vi kommer att bygga vidare på den strukturen genom hela boken.

## Avancerat

### NULL – när ett värde saknas

Anta att vi har adresser där vissa uppgifter ännu inte är registrerade.

**addresses**

| id | customer_id | street | postal_code | city |
|---|---|---|---|---|
| 1 | 1 | Hemmagatan 1 | 124 56 | Hemmavan |
| 2 | 1 | Storgatan 4 | 154 32 | Storfors |
| 3 | 2 | Parkvägen 2 | 154 32 | Storfors |
| 4 | 3 | NULL | NULL | Malmö |

`NULL` betyder att det saknas ett känt värde. Det är inte samma sak som noll eller en tom textsträng.

Vi kan inte söka efter `NULL` med vanlig likhet:

```
-- Detta fungerar inte som avsett
SELECT *
FROM addresses
WHERE street = NULL;
```

Vi använder i stället `IS NULL`:

```
SELECT *
FROM addresses
WHERE street IS NULL;
```

Och för att hitta de adresser där gatan är registrerad:

```
SELECT *
FROM addresses
WHERE street IS NOT NULL;
```

Varför? I SQL kan ett jämförelseuttryck vara **sant**, **falskt** eller **okänt**. En jämförelse med `NULL` ger normalt det tredje alternativet.

`WHERE` väljer endast rader där villkoret är sant.

## Operatorprioritet och parenteser

När vi kombinerar flera villkor kan ordningen påverka resultatet.

```
SELECT *
FROM customers
WHERE name = 'Anna'
   OR name = 'Olof' AND id > 2;
```

`AND` har högre prioritet än `OR`. Frågan betyder därför:

```
SELECT *
FROM customers
WHERE name = 'Anna'
   OR (name = 'Olof' AND id > 2);
```

Vill vi något annat behöver vi sätta parenteserna själva:

```
SELECT *
FROM customers
WHERE (name = 'Anna' OR name = 'Olof')
  AND id > 2;
```

Parenteser är ofta värdefulla även när de inte är tekniskt nödvändiga. De gör avsikten tydligare.

## DISTINCT – bara unika resultat

Vi kanske vill veta i vilka städer våra kunder har registrerade adresser.

```
SELECT city
FROM addresses;
```

Då kan samma stad förekomma flera gånger.

Om vi bara vill ha varje stad en gång:

```
SELECT DISTINCT city
FROM addresses;
```

`DISTINCT` tar bort upprepade resultatrader. Om vi väljer flera kolumner är det **kombinationen av kolumnvärdena** som avgör om två rader betraktas som dubbletter.

## Rader är inte automatiskt en ordnad lista

En tabell kan se ut som en lista, men SQL:s resultat har ingen garanterad ordning utan `ORDER BY`.

Det är särskilt viktigt när vi använder `LIMIT` och `OFFSET`.

Anta att vi bygger en sida som visar tio kunder åt gången. Vi kan hämta de första tio, sedan hoppa över tio och hämta nästa tio.

Men om nya kunder läggs till mellan frågorna kan en sådan sidindelning ge överlappningar eller missa poster. För stabilare sidindelning behöver vi bestämma en entydig sortering och ibland använda andra metoder än `OFFSET`.

Det tar vi upp mer utförligt när vi arbetar med prestanda och applikationer.

## Hur bör vi jämföra text?

Vi har skrivit frågor som:

```
SELECT *
FROM customers
WHERE name = 'Anna';
```

Men ska `Anna` och `anna` räknas som samma namn? Hur ska `Å`, `Ä` och `Ö` sorteras?

Det beror på språkregler och vilka inställningar som används för att jämföra text, ofta kallade **kollationer** (*collations*).

En och samma SQL-Query kan därför bete sig något olika i olika databasmiljöer.

Det är värt att komma ihåg: ett villkor som ser enkelt ut kan bygga på ganska många antaganden om informationens betydelse.

### Att filtrera data är också att välja perspektiv

När vi skriver en fråga bestämmer vi vilka uppgifter som ska ingå i resultatet.

En fråga som hämtar kunder med registrerade gatuadresser ger en annan bild än en fråga som också inkluderar kunder vars adresser är ofullständiga.

Båda kan vara korrekta. Men de besvarar inte riktigt samma fråga.

Det blir särskilt viktigt när vi använder SQL för rapportering och analys. En förutsättning för att kunna lita på resultatet är att vi förstår vilka uppgifter som valts bort.

## Reflektion

1. Vad är skillnaden mellan att information saknas och att information har värdet noll eller en tom sträng?
2. Varför behöver vi uttryckligen ange en sorteringsordning när vi vill hämta de första tio posterna?
3. Hur kan två SQL-Queries ge olika, men ändå korrekta, bilder av samma datamängd?

# 6. Beräkningar, funktioner och uttryck

Informationen i en databas behöver inte alltid presenteras exakt som den är lagrad.

En bokhandel lagrar exempelvis priset på en bok, men kanske vill visa priset inklusive en viss avgift eller beräkna ett rabatterat pris. Ett förnamn och ett efternamn kan behöva visas tillsammans. Vi kan också vilja beskriva ett lagersaldo som ”I lager” eller ”Slut i lager”.

Med SQL kan vi inte bara hämta information. Vi kan också **beräkna och presentera nya värden utifrån den information som redan finns**.

## Grund

### Uttryck i SELECT

Vi har hittills valt kolumner med `SELECT`:

```
SELECT name, email
FROM customers;
```

Men `SELECT` kan också innehålla **uttryck**. Ett uttryck är något som databasen kan beräkna till ett värde.

Vi kan börja helt utan en tabell:

```
SELECT 10 + 5;
```

Resultatet blir `15`.

Vi kan använda vanliga räkneoperatorer:

| Operator | Betydelse |
|---|---|
| `+` | Addition |
| `-` | Subtraktion |
| `*` | Multiplikation |
| `/` | Division |

Precis som i vanlig matematik kan vi använda parenteser för att styra beräkningen:

```
SELECT (100 + 50) * 2;
```

## Beräkningar med kolumner

Nu utökar vi bokhandelns databas med en tabell för produkter.

### products

| id | title | price | stock |
|---|---|---|---|
| 1 | Vägen hem | 200.00 | 12 |
| 2 | Den blå staden | 150.00 | 0 |
| 3 | Vinterljus | 249.00 | 4 |
| 4 | Små historier | 120.00 | 25 |

Här lagrar vi pris och lagersaldo för varje produkt.

Vi kan beräkna ett pris med 10 procent rabatt utan att ändra det lagrade priset:

```
SELECT title, price * 0.90
FROM products;
```

Beräkningen görs för varje rad i resultatet.

## Namnge beräknade kolumner

Resultatet blir tydligare om vi ger beräkningen ett namn:

```
SELECT
    title,
    price,
    price * 0.90 AS discounted_price
FROM products;
```

| title | price | discounted_price |
|---|---|---|
| Vägen hem | 200.00 | 180.00 |
| Den blå staden | 150.00 | 135.00 |
| Vinterljus | 249.00 | 224.10 |
| Små historier | 120.00 | 108.00 |

`AS` ger uttrycket ett **alias**, precis som vi tidigare kunde ge en kolumn ett annat namn.

Det är fortfarande samma tabell och samma lagrade värden. Vi har bara skapat en ny kolumn i vårt resultat.

## Funktioner

En **funktion** tar emot ett eller flera värden och ger oss ett resultat.

Vi kan exempelvis avrunda ett tal med `ROUND()`:

```
SELECT ROUND(12.345, 2);
```

Resultatet är `12.35`.

Vi kan också använda funktioner tillsammans med våra kolumner:

```
SELECT
    title,
    ROUND(price * 0.90, 2) AS discounted_price
FROM products;
```

En funktion kan alltså användas som en del av ett större uttryck.

## Textfunktioner

SQL har även funktioner för att arbeta med text.

`CONCAT()` sätter ihop flera textvärden:

```
SELECT CONCAT('Hej', ' ', 'Anna');
```

Resultatet blir `Hej Anna`.

Vi kan exempelvis skapa en kort produktbeskrivning:

```
SELECT CONCAT('Bok: ', title) AS description
FROM products;
```

Andra användbara textfunktioner är:

| Funktion | Användning |
|---|---|
| `CHAR_LENGTH(text)` | Antal tecken i en text |
| `UPPER(text)` | Gör text till versaler |
| `LOWER(text)` | Gör text till gemener |
| `SUBSTRING(text FROM start FOR antal)` | Hämtar en del av texten |
| `REPLACE(text, gammalt, nytt)` | Ersätter text |

Exempel:

```
SELECT UPPER('bokhandel');
```

ger `BOKHANDEL`.

Och:

```
SELECT REPLACE('Min bok', 'Min', 'Din');
```

ger `Din bok`.

Vissa funktioner har olika namn eller syntax i olika SQL-dialekter.[6]

## CASE – olika resultat beroende på villkor

Ibland vill vi att ett uttryck ska ge olika resultat beroende på ett villkor.

Vi kan exempelvis visa en text som beskriver om en produkt finns i lager:

```
SELECT
    title,
    stock,
    CASE
        WHEN stock > 0 THEN 'I lager'
        ELSE 'Slut i lager'
    END AS stock_status
FROM products;
```

| title | stock | stock_status |
|---|---|---|
| Vägen hem | 12 | I lager |
| Den blå staden | 0 | Slut i lager |
| Vinterljus | 4 | I lager |
| Små historier | 25 | I lager |

`CASE` är ett **uttryck** som ger ett värde. Vi använder `WHEN` för att ange ett villkor, `THEN` för resultatet och `ELSE` för alternativet om inget villkor passar.

Vi kan också ha flera `WHEN`:

```
SELECT
    title,
    CASE
        WHEN stock = 0 THEN 'Slut'
        WHEN stock < 5 THEN 'Få kvar'
        ELSE 'I lager'
    END AS stock_status
FROM products;
```

Databasen väljer resultatet för det **första** villkor som är uppfyllt.

Det här är SQL:s sätt att skapa villkorade värden utan att behöva ändra informationen i tabellen.

## Avancerat

### NULL i beräkningar

Vad händer om vi inte känner till priset på en produkt?

Om `price` är `NULL` blir resultatet av exempelvis `price * 0.90` också `NULL`.

Det är rimligt: vi kan inte beräkna ett rabatterat pris om vi inte vet utgångspriset.

SQL har funktionen `COALESCE()`, som väljer det första argumentet som inte är `NULL`:

```
SELECT COALESCE(NULL, 100);
```

ger `100`.

Vi skulle kunna skriva:

```
SELECT
    title,
    COALESCE(price, 0) AS display_price
FROM products;
```

Men detta kräver eftertanke. Ett saknat pris betyder inte nödvändigtvis att produkten är gratis.

`COALESCE` kan alltså vara praktiskt för att hantera saknade värden, men vi behöver själva avgöra vad ett ersättningsvärde betyder.

## Datatyper påverkar beräkningen

Värdet `'100'` är text, medan `100` är ett tal.

Vissa databassystem försöker automatiskt omvandla värden mellan olika datatyper. Det kan fungera, men också ge oväntade resultat.

När vi behöver göra en uttrycklig omvandling kan vi använda `CAST`:

```
SELECT CAST('100' AS INTEGER);
```

Detta uttrycker tydligt att textvärdet ska tolkas som ett heltal.[7]

## Beräknade värden eller lagrade värden?

Vi kan tänka oss att lagra både `price` och `discounted_price` i produkttabellen.

Men om rabatten alltid är 10 procent och priset senare ändras, behöver vi komma ihåg att även uppdatera rabattpriset.

Ett alternativ är att bara lagra grundpriset och beräkna rabattpriset när vi behöver det.

Det finns samtidigt situationer där ett beräknat värde bör sparas. En faktiskt genomförd beställning behöver exempelvis kunna bevara vilket pris kunden betalade, även om produktens aktuella pris senare förändras.

Skillnaden handlar om **vad som är en aktuell beräkning och vad som är ett historiskt faktum**.

Vi återkommer till detta när bokhandeln får ordrar och orderrader.

## Logik i frågor är inte alltid affärslogik

Med `CASE` kan vi exempelvis avgöra om ett lagersaldo ska visas som ”Få kvar”.

Det är en enkel regel för hur vi presenterar information.

Men ska databasen också bestämma när varor ska beställas, vilken leverantör vi ska använda och vem som ska godkänna inköpet?

Det är en annan typ av problem. En beräkning eller klassificering av data behöver inte betyda att hela verksamhetsprocessen bör byggas in i databasen.

Vi kommer tillbaka till gränsen mellan informationshantering och processer i bokens avslutande kapitel.

### Funktioner på många rader

Funktionerna vi använt hittills arbetar med värden i enskilda uttryck eller rader.

Men vi kanske också vill veta det sammanlagda lagervärdet för alla produkter eller det genomsnittliga produktpriset.

Då behöver vi kunna göra beräkningar **över flera rader samtidigt**.

Det är vad aggregeringsfunktioner som `SUM`, `COUNT` och `AVG` används till. Dem behandlar vi längre fram i boken.

## Reflektion

1. När är det bättre att beräkna ett värde i en fråga än att lagra det i databasen?
2. Varför kan det vara problematiskt att ersätta ett okänt värde med noll?
3. Vad är skillnaden mellan att visa att en produkt är ”Slut i lager” och att bestämma hur den ska beställas hem?

# 7. Datum och tid

En bokhandel behöver kunna beskriva inte bara *vad* som finns i databasen, utan också *när* något hände eller ska hända.

Vi vet exempelvis när en kund är född. Senare kanske vi vill veta när kunden registrerade sig, när en order lades och hur länge en reservation gäller.

Men tid är inte alltid ett enda slags värde. Ett födelsedatum är något annat än en exakt tidpunkt. En leveranstid kan vara en varaktighet, och en kampanj kan gälla under en viss period.

Att arbeta med datum och tid handlar därför först om att förstå **vilken sorts tidsinformation vi försöker beskriva**.

## Grund

### Datum – vilken dag?

I kapitel 4 introducerade vi datatypen `DATE`. Den beskriver ett datum, utan något klockslag.

Vi har redan kundernas födelsedatum i tabellen `customers`:

| id | name | birthday |
|---|---|---|
| 1 | Benjamin | 1968-04-12 |
| 2 | Olof | 1982-09-03 |
| 3 | Anaïs | 1995-06-21 |
| 4 | Beatrice | 2000-02-15 |
| 5 | Anna | 1975-11-08 |

Vi kan hämta alla födelsedatum:

```
SELECT name, birthday
FROM customers;
```

Vi kan också jämföra dem:

```
SELECT name, birthday
FROM customers
WHERE birthday > '1980-01-01';
```

Det ger de kunder som är födda efter den 1 januari 1980.

Vi skriver datum i formen `ÅÅÅÅ-MM-DD`, exempelvis `'1982-09-03'`. I våra SQL-exempel använder vi ett citerat datumvärde som databashanteraren kan tolka i jämförelsen med en `DATE`-kolumn.

## Att sortera datum

Datum kan jämföras och sorteras precis som andra värden:

```
SELECT name, birthday
FROM customers
ORDER BY birthday;
```

Nu kommer den äldsta kunden först.

Vill vi i stället ha den yngsta först använder vi `DESC`:

```
SELECT name, birthday
FROM customers
ORDER BY birthday DESC;
```

## Datumintervall

Vi kan hämta de kunder som är födda under 1990-talet:

```
SELECT name, birthday
FROM customers
WHERE birthday BETWEEN '1990-01-01' AND '1999-12-31';
```

`BETWEEN` inkluderar båda gränserna.

Samma fråga kan uttryckas så här:

```
SELECT name, birthday
FROM customers
WHERE birthday >= '1990-01-01'
  AND birthday < '2000-01-01';
```

Den senare formen är särskilt användbar när vi arbetar med datum *och klockslag*, eftersom vi inte behöver försöka beskriva dagens allra sista möjliga tidsvärde.

## Dagens datum

Ibland vill vi jämföra med dagens datum i stället för ett datum vi skriver själva.

```
SELECT CURRENT_DATE;
```

`CURRENT_DATE` ger dagens datum enligt databassystemets aktuella tidsinställningar.

Vi skulle exempelvis kunna leta efter födelsedatum som ligger i framtiden:

```
SELECT name, birthday
FROM customers
WHERE birthday > CURRENT_DATE;
```

En sådan fråga kan också hjälpa oss att upptäcka orimliga uppgifter.

## Datum och klockslag

Ett datum berättar vilken dag något hände, men inte vid vilken tid.

När vi senare börjar registrera beställningar behöver vi kanske spara både datum och klockslag:

```
2026-10-11 14:35:00
```

SQL har bland annat typen `TIMESTAMP` för datum och tid tillsammans. Flera databassystem har även en typ som heter `DATETIME`.[8]

Bokhandeln kan exempelvis komplettera kunduppgifterna med ett nytt attribut `registered_at`, som anger när kunden registrerades.

Vi skapar inte kolumnen i SQL ännu, men den kan senare användas så här:

```
SELECT name, registered_at
FROM customers
ORDER BY registered_at DESC;
```

Det skulle visa de senast registrerade kunderna först.

### Aktuell tidpunkt

`CURRENT_TIMESTAMP` ger ett aktuellt datum- och tidsvärde:

```
SELECT CURRENT_TIMESTAMP;
```

Det är användbart om vi exempelvis vill registrera när något inträffade.

En del databassystem har dessutom funktionen `NOW()` med liknande användning.[9]

### Tidpunkter och varaktighet

Anta att en bokhandel reserverar en vara i 30 minuter.

Då behöver vi skilja mellan två saker:

- **Tidpunkt:** Reservationen skapades klockan 14:35.
- **Varaktighet:** Reservationen gäller i 30 minuter.

Om vi även registrerar när reservationen upphör får vi en ny tidpunkt:

```
reserved_at: 2026-10-11 14:35:00
expires_at:  2026-10-11 15:05:00
```

Det är ofta lättare att fråga om en reservation har gått ut när vi har en tydlig sluttid:

```
SELECT *
FROM reservations
WHERE expires_at < CURRENT_TIMESTAMP;
```

Här använder vi en framtida tabell, `reservations`, bara för att visa idén. Den ingår ännu inte i vår bokhandelsdatabas.

## Avancerat

### En dag är inte alltid ett klockslag

Ett födelsedatum hör normalt inte hemma i ett fält som också ska beskriva ett exakt klockslag. Vi är intresserade av vilken dag personen föddes, inte av när ett databasprogram råkade registrera uppgiften.

På samma sätt kan en kampanj ha ett startdatum och ett slutdatum utan att vi ännu behöver ange några klockslag.

Valet mellan `DATE` och ett datum- och tidsvärde handlar därför om **vilken precision informationen faktiskt behöver**.

Att lagra mer precision än vi har nytta av gör inte informationen bättre.

### Halvöppna intervall

När vi frågar efter händelser under en månad kan detta vara frestande:

```
SELECT *
FROM orders
WHERE ordered_at BETWEEN '2026-10-01' AND '2026-10-31';
```

Men om `ordered_at` innehåller klockslag riskerar den övre gränsen att utesluta beställningar senare under den 31 oktober. Ett datum utan klockslag kan i en sådan jämförelse tolkas som midnatt vid dagens början.

Ett säkrare uttryck är:

```
SELECT *
FROM orders
WHERE ordered_at >= '2026-10-01'
  AND ordered_at < '2026-11-01';
```

Det kallas ett **halvöppet intervall**: den nedre gränsen ingår, den övre gör det inte.

Vi kan använda det oavsett om databasen lagrar sekunder eller ännu högre precision.

Tabellen `orders` introduceras senare i boken. Här visar vi bara hur tidsvillkoret är uppbyggt.

## Att beräkna tidsskillnader

Vi kan behöva veta hur många dagar som gått mellan två datum.

Det låter enkelt, men exakt syntax varierar mellan SQL-dialekter. MySQL har exempelvis `DATEDIFF()`:

```
SELECT DATEDIFF('2026-10-11', '2026-10-01');
```

Resultatet är `10` dagar.[10]

Det finns också en skillnad mellan att beräkna **antal kalenderdagar** och **förfluten tid i timmar eller sekunder**. Skillnaden blir viktig vid exempelvis sommartidsomställningar.

## Ålder är inte bara två årtal

Det kan vara frestande att beräkna en kunds ålder så här:

```
innevarande år − födelseår
```

Men då får vi fel svar om kunden inte har fyllt år ännu.

För att beräkna ålder korrekt måste vi även jämföra månad och dag. Vi behöver dessutom vara tydliga med vid vilket datum åldern ska gälla.

Detta visar varför tidsberäkningar kräver mer eftertanke än vanliga numeriska beräkningar.

## Tidszoner

Om bokhandeln bara finns på en ort kanske det verkar självklart vad klockan är.

Men om en kund lägger en beställning från en annan tidszon uppstår en fråga: vilken tid ska vi lagra och visa?

En **tidpunkt** är densamma oavsett var i världen vi befinner oss. Men den lokala klocktiden som beskriver tidpunkten är olika på olika platser.

Därför behöver vi skilja mellan:

- **En absolut tidpunkt**, som när en order faktiskt registrerades.
- **Ett lokalt datum eller klockslag**, som när en butik öppnar.
- **En tidszon**, som avgör hur ett lokalt klockslag förhåller sig till en absolut tidpunkt.

För händelser som ska kunna jämföras globalt är det ofta praktiskt att lagra eller normalisera tidpunkter till UTC och omvandla dem till lokal tid vid presentation.

Men det betyder inte att alla datum och tider bör behandlas som UTC. En butiks öppettid klockan 09:00 beskriver exempelvis en lokal regel, inte en och samma globala tidpunkt.

## TIMESTAMP betyder inte alltid samma sak

Det är lätt att tro att en datatyp som heter `TIMESTAMP` fungerar likadant överallt. Så är det inte.

I MySQL sker exempelvis tidszonsomvandling för `TIMESTAMP` utifrån sessionens tidszon, medan `DATETIME` beter sig annorlunda.

PostgreSQL skiljer mellan `TIMESTAMP WITHOUT TIME ZONE` och `TIMESTAMP WITH TIME ZONE`.

SQLite har inte en separat, fast datum- och tidslagringstyp på samma sätt, utan kan representera tidsvärden i exempelvis text eller numeriska format.

Det här är en anledning till att tidszonshantering ofta behöver beskrivas i en dialektspecifik referens.

### När hände det, och när fick vi veta det?

Föreställ dig att en kund flyttade den 1 september, men att bokhandeln inte fick veta det förrän den 10 september.

Det ger oss två olika tidpunkter:

- När förändringen **gällde i verkligheten**.
- När förändringen **registrerades i vårt system**.

De är inte alltid samma sak.

I enkla system räcker det ofta att bara lagra den aktuella adressen. Men om vi behöver återskapa historiska förhållanden, exempelvis vilken leveransadress som gällde vid en viss beställning, behöver vi tänka mer noggrant kring historik.

Vi kommer att återvända till detta när vi behandlar transaktionsdata och datakvalitet.

## Reflektion

1. Varför är ett födelsedatum något annat än en exakt tidpunkt, även om båda handlar om tid?
2. När är det bättre att använda ett halvöppet intervall än `BETWEEN`?

3. Vad är skillnaden mellan när något hände i verkligheten och när det registrerades i databasen?

# 8. Aggregering och gruppering

En bokhandel kan veta priset och lagersaldot för varje produkt. Men ibland är det inte den enskilda produkten vi vill undersöka.

Vi kanske vill veta hur många produkter vi säljer, vad de kostar i genomsnitt eller hur många exemplar vi har i lager totalt.

Då behöver vi gå från **enskilda värden till sammanfattningar**.

En sammanfattning kan göra stora mängder information begripliga. Men den innebär också att vi väljer bort detaljer. Ett genomsnittligt pris berättar något om sortimentet – men inte vilka böcker som är dyra och vilka som är billiga.

## Grund

### Flera rader blir ett resultat

Vi fortsätter med bokhandelns produkter. Vi lägger till en produkt i vår tabell:

**products**

| id | title | price | stock |
|---|---|---|---|
| 1 | Vägen hem | 200.00 | 12 |
| 2 | Den blå staden | 150.00 | 0 |
| 3 | Vinterljus | 249.00 | 4 |
| 4 | Små historier | 120.00 | 25 |
| 5 | Mellan raderna | 180.00 | 9 |

Om vi vill veta hur många produkter som finns i tabellen kan vi räkna raderna:

```
SELECT COUNT(*)
FROM products;
```

Resultat:

| COUNT(*) |
|---|
| 5 |

`COUNT()` är en **aggregeringsfunktion**. Den använder värden från flera rader för att skapa ett sammanfattat resultat.

## Vanliga aggregeringsfunktioner

| Funktion | Vad den beräknar |
|---|---|
| `COUNT()` | Antal |
| `SUM()` | Summa |
| `AVG()` | Medelvärde |
| `MIN()` | Minsta värde |
| `MAX()` | Största värde |

Vi kan beräkna det totala antalet exemplar i lager:

```
SELECT SUM(stock) AS total_stock
FROM products;
```

| total_stock |
|---|
| 50 |

Vi kan också beräkna sortimentets genomsnittliga pris:

```
SELECT AVG(price) AS average_price
FROM products;
```

| average_price |
|---|
| 179.80 |

Observera att vi nu får **ett värde** som sammanfattar flera produkter.

## Flera sammanfattningar samtidigt

Vi kan göra flera beräkningar i samma fråga:

```
SELECT
    COUNT(*) AS product_count,
    SUM(stock) AS total_stock,
    MIN(price) AS lowest_price,
    MAX(price) AS highest_price
FROM products;
```

| product_count | total_stock | lowest_price | highest_price |
|---|---|---|---|
| 5 | 50 | 120.00 | 249.00 |

Här beskriver resultatet hela produkttabellen, inte en enskild produkt.

## Vilka rader ska räknas?

Vi kan använda `WHERE` för att välja vilka rader som ska ingå **innan** vi gör beräkningen.

```
SELECT COUNT(*) AS available_products
FROM products
WHERE stock > 0;
```

| available_products |
|---|
| 4 |

Vi räknar nu antalet olika produkter som faktiskt finns i lager.

Men om vi i stället frågar hur många **exemplar** som finns tillgängliga behöver vi `SUM(stock)`. Skillnaden mellan antal produkter och antal exemplar är en skillnad i *vad vi mäter*, inte bara i SQL-syntax.

## Gruppindelning med GROUP BY

Tänk dig att vi vill veta hur produkterna fördelar sig mellan olika kategorier.

Vi utökar därför tabellen med `product_type_id`, som pekar på en produkttyp.

**products**

| id | title | price | stock | product_type_id |
|---|---|---|---|---|
| 1 | Vägen hem | 200.00 | 12 | 1 |
| 2 | Den blå staden | 150.00 | 0 | 1 |
| 3 | Vinterljus | 249.00 | 4 | 1 |
| 4 | Små historier | 120.00 | 25 | 2 |
| 5 | Mellan raderna | 180.00 | 9 | 2 |

Produkttyperna finns i en separat tabell, `product_types`: `1` betyder *Roman* och `2` betyder *Novellsamling*. Vi kommer senare att lära oss hämta typnamnet från den tabellen. Än så länge räcker identifierarna.

Om vi vill räkna antalet produkter per typ:

```
SELECT
    product_type_id,
    COUNT(*) AS product_count
FROM products
GROUP BY product_type_id;
```

| product_type_id | product_count |
|---|---|
| 1 | 3 |
| 2 | 2 |

`GROUP BY` delar in raderna i grupper utifrån ett eller flera värden. Därefter gör aggregeringsfunktionen en beräkning för varje grupp.

## Flera beräkningar per grupp

Vi kan också få både antalet produkter och det sammanlagda lagersaldot per typ:

```
SELECT
    product_type_id,
    COUNT(*) AS product_count,
```

```
    SUM(stock) AS total_stock
FROM products
GROUP BY product_type_id
ORDER BY product_type_id;
```

| product_type_id | product_count | total_stock |
|---|---|---|
| 1 | 3 | 16 |
| 2 | 2 | 34 |

Resultatet innehåller en rad per grupp.

## Filtrera grupper med HAVING

Anta att vi bara är intresserade av produkttyper som innehåller fler än två olika produkter.

Vi kan inte använda `WHERE COUNT(*) > 2` för detta. `WHERE` filtrerar enskilda rader innan grupperingen är gjord.

I stället använder vi `HAVING`:

```
SELECT
    product_type_id,
    COUNT(*) AS product_count
FROM products
GROUP BY product_type_id
HAVING COUNT(*) > 2;
```

| product_type_id | product_count |
|---|---|
| 1 | 3 |

`HAVING` filtrerar **grupper efter aggregeringen**.

En sammanfattning av hur delarna samverkar:

```
SELECT product_type_id, COUNT(*) AS product_count
FROM products
WHERE price >= 150
GROUP BY product_type_id
HAVING COUNT(*) >= 2
ORDER BY product_type_id;
```

Här väljer vi först produkter som kostar minst 150, grupperar dem efter typ och behåller sedan bara grupper med minst två produkter.

## Avancerat

### Vad räknar COUNT egentligen?

Det finns en viktig skillnad mellan tre uttryck:

```
COUNT(*)
COUNT(price)
COUNT(DISTINCT price)
```

- `COUNT(*)` räknar rader.
- `COUNT(price)` räknar rader där `price` inte är `NULL`.
- `COUNT(DISTINCT price)` räknar antalet olika kända prisvärden.

Om varje produkt har ett pris är de två första lika stora. Men om vissa priser saknas blir resultatet annorlunda.

Att fråga ”hur många?” är alltså inte tillräckligt. Vi måste också bestämma **vad det är vi räknar**.

### NULL påverkar sammanfattningen

`SUM()` och `AVG()` ignorerar normalt `NULL`-värden.

Om vi har tre priser, `100`, `200` och `NULL`, blir medelvärdet `150`, inte `100`.

Det beror på att medelvärdet beräknas över de två kända priserna. Vi kan inte förutsätta att ett saknat pris är noll.

När vi tolkar ett medelvärde behöver vi därför ibland också fråga hur många värden som faktiskt ingick:

```
SELECT
    COUNT(*) AS products,
    COUNT(price) AS known_prices,
    AVG(price) AS average_price
FROM products;
```

## Att mäta lagervärde

Vi har hittills räknat antal produkter, antal exemplar och genomsnittligt pris.

Men vad är det totala lagervärdet, beräknat utifrån våra aktuella försäljningspriser?

För varje produkt behöver vi multiplicera priset med antalet exemplar. Därefter summerar vi:

```
SELECT SUM(price * stock) AS inventory_value
FROM products;
```

| inventory_value |
|---|
| 8016.00 |

Detta är ett **beräknat värde utifrån aktuella priser och lagersaldon**, inte nödvändigtvis varornas bokförda lagervärde eller inköpsvärde.

Vi kan också beräkna lagervärdet per produkttyp:

```
SELECT
    product_type_id,
    SUM(price * stock) AS inventory_value
FROM products
GROUP BY product_type_id
ORDER BY product_type_id;
```

Detta visar hur aggregering och uttryck kan kombineras.

## Varför får vi inte välja vilken kolumn som helst?

Betrakta denna fråga:

```
SELECT product_type_id, title, COUNT(*)
FROM products
GROUP BY product_type_id;
```

Vi vill få en rad per produkttyp, men vad ska då `title` innehålla? Varje typ kan innehålla flera titlar.

Frågan är därför inte väldefinierad som den står. Normalt behöver de kolumner och uttryck vi väljer antingen ingå i grupperingen, vara aggregerade eller vara funktionellt beroende av grupperingskolumnerna.[11]

Det här är inte bara en teknisk regel. Det handlar om att vi måste bestämma **vad en rad i vårt resultat betyder**.

## Ett genomsnitt kan dölja skillnader

Två produkttyper kan ha samma genomsnittliga pris trots att den ena innehåller nästan enbart jämnt prissatta böcker och den andra både mycket billiga och mycket dyra.

Ett medelvärde är därför inte en fullständig beskrivning av en fördelning.

Vi kan ibland behöva komplettera `AVG()` med `MIN()`, `MAX()` och `COUNT()` – eller gå tillbaka och studera de enskilda värdena.

När vi senare arbetar med fönsterfunktioner kommer vi dessutom att kunna visa sammanfattningar utan att förlora varje enskild rad i resultatet.

## Från gruppering till en ny modell

Vår ursprungliga produkttabell beskriver **produkter**.

Men frågan:

```
SELECT
    product_type_id,
```

```
    COUNT(*) AS product_count,
    SUM(stock) AS total_stock
FROM products
GROUP BY product_type_id;
```

beskriver i stället **produkttypernas sammanlagda sortiment och lager**.

Vi har skapat en ny representation av redan lagrade data, med en annan betydelse för varje rad.

Den nya representationen behöver inte lagras som en egen tabell. Längre fram kommer vi att se hur en sådan fråga kan bli en **View** – en återanvändbar, logisk modell av samma information.

## Reflektion

1. Vad är skillnaden mellan att räkna antalet produkter och att summera antalet exemplar i lager?
2. När riskerar ett genomsnitt att dölja viktig information?
3. När vi grupperar produkter efter typ, vad är det egentligen som varje rad i resultatet representerar?

# 9. Att kombinera flera tabeller

En kund har ett namn. En adress har en gata och en stad. En produkt har ett pris och tillhör en produkttyp.

Vi har valt att lagra dessa uppgifter i olika tabeller eftersom de beskriver olika företeelser. Men när vi vill använda informationen behöver vi ofta se dem tillsammans.

En fråga som ”Vilka kunder har adresser i Storfors?” handlar inte enbart om kunder eller adresser. Den handlar om **sambandet mellan dem**.

Med SQL kan vi kombinera information från flera tabeller utan att först behöva lägga allting i samma tabell.

## Grund

### Två tabeller som hör ihop

Vi börjar med bokhandelns kunder och adresser.

**customers**

| id | name |
|---|---|
| 1 | Benjamin |
| 2 | Olof |
| 3 | Anaïs |
| 4 | Beatrice |
| 5 | Anna |

**addresses**

| id | customer_id | street | city |
|---|---|---|---|
| 1 | 1 | Hemmagatan 1 | Hemmavan |
| 2 | 1 | Storgatan 4 | Storfors |
| 3 | 2 | Parkvägen 2 | Storfors |
| 4 | 3 | NULL | Malmö |

Kolumnen `customer_id` i `addresses` talar om vilken kund adressen tillhör.

När `customer_id` är 2 syftar den på kunden med `id` lika med 2, alltså Olof.

Vi behöver inte upprepa kundens namn på varje adressrad. I stället använder vi relationen mellan tabellerna.

## INNER JOIN – kombinera matchande rader

Vi vill se kundernas namn tillsammans med deras adresser:

```
SELECT
    c.name,
    a.street,
    a.city
FROM customers AS c
INNER JOIN addresses AS a
    ON a.customer_id = c.id;
```

Resultat:

| name | street | city |
|---|---|---|
| Benjamin | Hemmagatan 1 | Hemmavan |
| Benjamin | Storgatan 4 | Storfors |
| Olof | Parkvägen 2 | Storfors |
| Anaïs | NULL | Malmö |

`INNER JOIN` kombinerar rader där villkoret efter `ON` är uppfyllt.

I vårt exempel är villkoret att adressens `customer_id` ska motsvara kundens `id`.

Det är viktigt att lägga märke till att Benjamin förekommer två gånger. Det beror inte på att vi har två Benjamin i kundtabellen, utan på att han har två registrerade adresser.

## Varför använder vi alias?

Vi har tidigare sett hur vi ger tabeller kortare namn:

```
FROM customers AS c
INNER JOIN addresses AS a
```

När vi skriver `c.name` menar vi kolumnen `name` i tabellen som vi kallar `c`. `a.city` pekar på kolumnen `city` i `a`.

Det är särskilt användbart när två tabeller har kolumner med samma namn, till exempel `id`.

Utan prefixet skulle `SELECT id` vara tvetydigt i en fråga där båda tabellerna har en kolumn som heter `id`.

## JOIN och WHERE har olika uppgifter

Nu vill vi veta vilka kunder som har en adress i Storfors.

```
SELECT
    c.name,
    a.street,
    a.city
FROM customers AS c
INNER JOIN addresses AS a
    ON a.customer_id = c.id
WHERE a.city = 'Storfors';
```

| name | street | city |
|---|---|---|
| Benjamin | Storgatan 4 | Storfors |
| Olof | Parkvägen 2 | Storfors |

`ON` beskriver **hur tabellerna hör ihop**.

`WHERE` beskriver **vilka av de kombinerade raderna vi vill ha**.

Det är en användbar skillnad att hålla fast vid även när frågorna blir mer komplicerade.

## LEFT JOIN – behåll även rader utan matchning

En `INNER JOIN` visar bara kombinationer som har en matchning i båda tabellerna.

Men vad händer om vi vill se **alla kunder**, även dem som ännu inte har någon registrerad adress?

```
SELECT
    c.name,
    a.city
FROM customers AS c
LEFT JOIN addresses AS a
    ON a.customer_id = c.id
ORDER BY c.id, a.id;
```

| name | city |
|---|---|
| Benjamin | Hemmavan |
| Benjamin | Storfors |
| Olof | Storfors |
| Anaïs | Malmö |
| Beatrice | NULL |
| Anna | NULL |

`LEFT JOIN` behåller alla rader från den vänstra tabellen, `customers`. Där en matchande adress saknas får adresskolumnerna värdet `NULL` i resultatet.

Notera skillnaden mot Anaïs, vars adress finns men saknar gatuadress. `NULL` betyder alltså inte automatiskt att hela den relaterade raden saknas.

## Från id till begripligt namn

I kapitel 8 lade vi till `product_type_id` på produkterna.

### product_types

| id | name |
|---|---|
| 1 | Roman |
| 2 | Novellsamling |

Nu kan vi visa produkternas titlar och deras produkttyp:

```
SELECT
    p.title,
    t.name AS product_type
FROM products AS p
INNER JOIN product_types AS t
    ON p.product_type_id = t.id
ORDER BY p.id;
```

| title | product_type |
|---|---|
| Vägen hem | Roman |
| Den blå staden | Roman |
| Vinterljus | Roman |
| Små historier | Novellsamling |
| Mellan raderna | Novellsamling |

Vi lagrar produkttypens namn på ett ställe, men kan visa det tillsammans med varje produkt när vi behöver det.

## JOIN tillsammans med gruppering

Vi kan också kombinera relationer och aggregering.

```
SELECT
    t.name AS product_type,
    COUNT(*) AS product_count
FROM products AS p
INNER JOIN product_types AS t
    ON p.product_type_id = t.id
```

```
GROUP BY t.id, t.name
ORDER BY t.id;
```

| product_type | product_count |
|---|---|
| Roman | 3 |
| Novellsamling | 2 |

Det här liknar vår fråga från förra kapitlet, men resultatet har nu ett begripligt namn på varje grupp.

## Avancerat

### Vad händer om vi glömmer kopplingsvillkoret?

En äldre syntax för att kombinera tabeller är:

```
SELECT c.name, a.city
FROM customers AS c, addresses AS a
WHERE a.customer_id = c.id;
```

Den ger samma matchande kombinationer som vårt första `INNER JOIN`-exempel.

Men när tabellerna räknas upp med kommatecken och vi glömmer villkoret kombineras **varje kund med varje adress**.

Med fem kunder och fyra adresser ger det 20 kombinationer.

Det kallas en **kartesisk produkt** och kan också skrivas uttryckligen med `CROSS JOIN`:

```
SELECT c.name, a.city
FROM customers AS c
CROSS JOIN addresses AS a;
```

En kartesisk produkt är ibland avsiktlig, men ofta ett tecken på att vi har glömt beskriva hur informationen hör ihop.

Därför använder vi i regel tydlig `JOIN ... ON`-syntax när vi kombinerar relaterade tabeller.

## Villkor vid LEFT JOIN

Vid `INNER JOIN` kan villkor ibland flyttas mellan `ON` och `WHERE` utan att resultatet ändras.

Vid `LEFT JOIN` behöver vi vara mer försiktiga.

```
SELECT c.name, a.city
FROM customers AS c
LEFT JOIN addresses AS a
    ON a.customer_id = c.id
WHERE a.city = 'Storfors';
```

Här kommer kunder utan matchande adresser inte med, eftersom deras `a.city` blir `NULL` och inte uppfyller `WHERE`-villkoret.

Jämför med:

```
SELECT c.name, a.city
FROM customers AS c
LEFT JOIN addresses AS a
    ON a.customer_id = c.id
   AND a.city = 'Storfors';
```

Här behåller vi fortfarande **alla kunder** från vänstersidan, men kopplar bara på adresser i Storfors.

Det är två olika informationsbehov, och båda är möjliga att uttrycka korrekt.

## Fler än två tabeller

När vi börjar registrera beställningar kan vi ha en tabell `orders` som refererar till `customers` och en tabell `order_items` som refererar till `orders` och `products`.

Då kan en fråga koppla samman flera relationer:

```
SELECT
    o.id AS order_id,
    c.name AS customer_name,
    p.title,
    oi.quantity
FROM orders AS o
INNER JOIN customers AS c
    ON o.customer_id = c.id
INNER JOIN order_items AS oi
    ON oi.order_id = o.id
INNER JOIN products AS p
    ON oi.product_id = p.id;
```

Vi har ännu inte infört `orders` och `order_items` i vår praktiska modell. Exemplet visar vart relationerna kan leda.

Varje rad i resultatet beskriver här **en orderrad**, inte en hel order. En order med flera produkter kommer därför att förekomma på flera rader.

Det är avgörande när vi senare räknar och summerar.

## Varför en JOIN kan förändra antalet rader

En vanlig missuppfattning är att en `JOIN` bara lägger till kolumner.

Men när en kund har flera adresser ger kopplingen flera resultatrader för samma kund.

Om vi sedan skriver `COUNT(*)` räknar vi de kombinerade raderna, inte nödvändigtvis antalet unika kunder.

Det är ofta lämpligt att fråga: **Vad representerar en rad i det här resultatet?**

I våra exempel har den ibland representerat en kund, ibland en kundadress och ibland en orderrad.

## Olika slags JOIN

Utöver `INNER JOIN` och `LEFT JOIN` finns bland annat:

| JOIN | Princip |
|---|---|
| `INNER JOIN` | Bara matchande kombinationer |
| `LEFT JOIN` | Alla rader till vänster, plus matchningar |
| `RIGHT JOIN` | Alla rader till höger, plus matchningar |
| `FULL OUTER JOIN` | Alla rader från båda sidor, med matchningar där sådana finns |
| `CROSS JOIN` | Alla möjliga kombinationer |

Stödet varierar mellan databassystem. Till exempel har MySQL inget inbyggt `FULL OUTER JOIN`, och SQLite saknar `RIGHT JOIN` och `FULL OUTER JOIN` i äldre versioner.[1][2]

## Från relationer till en ny modell av informationen

När vi skriver en Query som:

```
SELECT
    c.name AS customer_name,
    a.street,
    a.city
FROM customers AS c
INNER JOIN addresses AS a
    ON a.customer_id = c.id;
```

får vi en tabellformad representation som inte finns som en egen lagrad tabell.

Vår lagringsmodell består av `customers` och `addresses`, men resultatet beskriver i praktiken **kundadresser**.

Det är ett annat perspektiv på samma information.

Senare i boken kommer vi att använda **Views** för att ge sådana representationer egna namn och göra dem återanvändbara.

Därmed kan vi skilja mellan hur informationen organiseras för lagring och hur vi modellerar den för olika frågor och användare.

## Reflektion

1. Varför kan det vara bättre att lagra kundernas namn och adresser i olika tabeller, även när vi ofta vill visa dem tillsammans?
2. Vad representerar en rad efter att vi har kopplat en kund till flera adresser?
3. Hur skiljer sig en ny tabell från ett nytt sätt att betrakta informationen i flera befintliga tabeller?

# 10. Subqueries

En fråga till en databas behöver inte alltid besvaras i ett enda steg.

Vi kanske vill hitta böcker som kostar mer än genomsnittet i vårt sortiment. Men då behöver vi först veta vad genomsnittet är. Eller så vill vi hitta kunder som har en adress i en viss stad, utan att själva visa adressuppgifterna.

Vi skulle kunna ställa två frågor efter varandra. Men SQL låter oss också använda **en fråga som en del av en annan fråga**.

Det kallas en *subquery*, på svenska en **underfråga**.

## Grund

### En Query inuti en annan

Vi använder bokhandelns produkter:

**products**

| id | title | price | stock | product_type_id |
|---|---|---|---|---|
| 1 | Vägen hem | 200.00 | 12 | 1 |
| 2 | Den blå staden | 150.00 | 0 | 1 |
| 3 | Vinterljus | 249.00 | 4 | 1 |
| 4 | Små historier | 120.00 | 25 | 2 |
| 5 | Mellan raderna | 180.00 | 9 | 2 |

Anta att vi vill hitta produkter som kostar mer än genomsnittet.

Vi kan börja med att beräkna genomsnittspriset:

```
SELECT AVG(price)
FROM products;
```

Resultatet är `179.80`.

Vi kan sedan använda resultatet i en annan fråga:

```
SELECT title, price
FROM products
WHERE price > 179.80;
```

Men då har vi skrivit in ett värde som kan förändras när sortimentet ändras.

I stället kan vi låta databasen beräkna värdet direkt:

```
SELECT title, price
FROM products
WHERE price > (
    SELECT AVG(price)
    FROM products
);
```

| title | price |
|---|---|
| Vägen hem | 200.00 |
| Vinterljus | 249.00 |
| Mellan raderna | 180.00 |

Den inre frågan beräknar genomsnittspriset. Den yttre frågan använder det som gräns för att välja produkter.

Vi har alltså beskrivit ett informationsbehov som bygger på ett annat.

## Subqueries som ger ett enda värde

I vårt första exempel gav underfrågan ett enda värde: genomsnittspriset.

En underfråga som används som ett enskilt värde kallas en **skalär subquery**.

Vi kan också visa genomsnittet tillsammans med varje produkt:

```
SELECT
    title,
    price,
    (SELECT AVG(price) FROM products) AS average_price
FROM products;
```

Här blir genomsnittspriset detsamma på varje resultatrad.

En skalär underfråga måste ge högst en rad och en kolumn. Om den inte ger någon rad blir resultatet `NULL`; ger den fler än en rad uppstår normalt ett fel.

## Subqueries med flera värden: IN

Ibland är resultatet från den inre frågan inte ett enda värde, utan flera.

Anta att vi vill hämta alla kunder som har en adress i Storfors.

Vi känner redan till dessa tabeller:

### customers

| id | name |
|---|---|
| 1 | Benjamin |
| 2 | Olof |
| 3 | Anaïs |
| 4 | Beatrice |
| 5 | Anna |

### addresses

| id | customer_id | street | city |
|---|---|---|---|
| 1 | 1 | Hemmagatan 1 | Hemmavan |
| 2 | 1 | Storgatan 4 | Storfors |
| 3 | 2 | Parkvägen 2 | Storfors |
| 4 | 3 | NULL | Malmö |

Först kan vi ta fram de kund-id som förekommer på adresser i Storfors:

```
SELECT customer_id
FROM addresses
WHERE city = 'Storfors';
```

Resultatet blir 1 och 2.

Det kan vi använda i en yttre fråga med `IN`:

```
SELECT name
FROM customers
WHERE id IN (
    SELECT customer_id
    FROM addresses
    WHERE city = 'Storfors'
);
```

| name |
| --- |
| Benjamin |
| Olof |

`IN` frågar om ett värde finns bland värdena som underfrågan ger tillbaka.

Notera att vi bara hämtar en rad per kund, även om kunden skulle ha flera adresser i Storfors. Vi undersöker *vilka kunder* som matchar, inte hur många adresser de har.

## Subqueries och JOIN

Samma informationsbehov kan vi uttrycka med `JOIN`:

```
SELECT DISTINCT c.name
FROM customers AS c
INNER JOIN addresses AS a
    ON a.customer_id = c.id
WHERE a.city = 'Storfors';
```

Varför `DISTINCT`?

En `JOIN` kan skapa flera resultatrader för samma kund om kunden har flera matchande adresser. En fråga med `IN` utgår däremot från kundraderna och testar om deras id finns i en samling värden.

Ingen av metoderna är automatiskt bäst. Vi väljer bland annat efter **vad vi vill att varje rad i resultatet ska representera**.

## EXISTS – finns det minst en matchning?

Ibland vill vi inte ha något värde från underfrågan alls. Vi vill bara veta om en matchande rad **finns**.

Då kan vi använda `EXISTS`:

```
SELECT c.name
FROM customers AS c
WHERE EXISTS (
    SELECT 1
    FROM addresses AS a
    WHERE a.customer_id = c.id
      AND a.city = 'Storfors'
);
```

Resultatet blir återigen Benjamin och Olof.

`EXISTS` blir sant om underfrågan ger minst en rad. Vilka värden som står i underfrågans `SELECT` spelar ingen roll för själva existenskontrollen. Därför används ofta `SELECT 1`.

Detta exempel skiljer sig från de tidigare: den inre frågan använder `c.id` från den yttre frågan.

Vi tittar närmare på det under *Avancerat*.

# Avancerat

## Korrelerade subqueries

En **korrelerad subquery** hänvisar till värden från den omgivande frågan.

```
SELECT c.name
FROM customers AS c
WHERE EXISTS (
    SELECT 1
    FROM addresses AS a
    WHERE a.customer_id = c.id
);
```

För varje kund undersöker vi om det finns minst en adress som refererar till just den kundens id.

Det kan vara hjälpsamt att tänka att underfrågan prövas mot varje kundrad. Men det är en **logisk beskrivning**, inte ett krav på hur databashanteraren faktiskt ska utföra frågan. En SQL-optimerare kan välja en annan exekveringsplan.

## NOT EXISTS – när något saknas

Vi kan vända på frågan och leta efter kunder utan registrerade adresser:

```
SELECT c.name
FROM customers AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM addresses AS a
    WHERE a.customer_id = c.id
);
```

| name |
| --- |
| Beatrice |
| Anna |

`NOT EXISTS` är användbart när vi söker efter *avsaknaden av en relation*.

Det kan till exempel senare användas för att hitta produkter som aldrig har förekommit på en orderrad, eller beställningar som saknar någon viss typ av information.

## NOT IN och NULL

Det är frestande att skriva om föregående exempel med `NOT IN`:

```
SELECT name
FROM customers
WHERE id NOT IN (
    SELECT customer_id
    FROM addresses
);
```

Om `addresses.customer_id` aldrig kan vara `NULL` fungerar detta som väntat.

Men om underfrågan innehåller `NULL` kan `NOT IN` ge överraskande resultat. SQL:s trevärdeslogik innebär att en jämförelse mot ett okänt värde kan bli *okänd* i stället för sann.

När vi vill uttrycka att **ingen matchande rad finns** är `NOT EXISTS` därför ofta ett tydligare val.

## Subqueries i FROM

En underfråga behöver inte stå i `WHERE`. Vi kan också använda den som en datakälla i `FROM`.

Anta att vi först vill räkna fram antalet produkter per produkttyp och därefter bara visa typer som har fler än två produkter.

```
SELECT
    summary.product_type_id,
    summary.product_count
FROM (
    SELECT
```

```
        product_type_id,
        COUNT(*) AS product_count
    FROM products
    GROUP BY product_type_id
) AS summary
WHERE summary.product_count > 2;
```

| product_type_id | product_count |
|---|---|
| 1 | 3 |

Här skapar den inre frågan ett **tabellformat resultat**, som den yttre frågan sedan behandlar som en datakälla.

Vi ger det tillfälliga resultatet aliaset `summary`.

Vi hade också kunnat lösa just detta med `HAVING`, som vi såg i föregående kapitel. Men exemplet visar hur vi kan bygga vidare på ett mellanresultat.

En sådan konstruktion kallas ibland en *derived table* eller härledd tabell.

## Subqueries för jämförelser inom en grupp

Vi kan också jämföra varje produkt med genomsnittspriset för dess egen produkttyp:

```
SELECT
    p.title,
    p.price,
    p.product_type_id
FROM products AS p
WHERE p.price > (
    SELECT AVG(p2.price)
    FROM products AS p2
    WHERE p2.product_type_id = p.product_type_id
);
```

Här beräknar den inre frågan ett genomsnitt för just den produkttyp som den aktuella produkten tillhör.

Detta är ännu ett exempel på en korrelerad subquery.

Jämförelsen svarar alltså inte på frågan ”vilka böcker är dyrare än genomsnittsboken?”, utan på frågan **”vilka böcker är dyrare än genomsnittet inom sin egen typ?”**

Det är två olika sätt att tolka informationen.

## Subquery, JOIN eller CTE?

Flera SQL-konstruktioner kan ibland beskriva samma informationsbehov, men de är inte alltid utbytbara utan att resultatet förändras.

**När vi vill kombinera information från flera tabeller är `JOIN` ofta det naturliga förstahandsvalet.**

När vi i stället vill undersöka om en relation finns passar `EXISTS` eller `IN` ofta bättre. En `JOIN` kan ge flera resultatrader för samma kund om kunden har flera adresser, medan `EXISTS` bara testar om minst en matchning finns. En skalär subquery är också ett naturligt val när vi vill jämföra med exempelvis ett genomsnittspris.

Det finns ingen allmän regel om att subqueries är långsammare än `JOIN`. Moderna databashanterare kan ofta omvandla och optimera olika skrivsätt till liknande exekveringsplaner. När prestandan verkligen spelar roll bör vi undersöka frågans exekveringsplan, exempelvis med `EXPLAIN`, i stället för att dra slutsatser från syntaxen.

**Vår tumregel:** Välj i första hand den konstruktion som tydligast uttrycker vad varje rad i resultatet ska betyda. Använd gärna `JOIN` för att kombinera data och `EXISTS` eller `IN` för att pröva förekomsten av relaterade värden.

Senare kommer vi också att arbeta med `WITH` och så kallade **CTE:er** (*Common Table Expressions*). De låter oss namnge mellanresultat och strukturera större frågor.

### Att bygga en fråga av andra frågor

Med subqueries kan vi stegvis uttrycka mer sammansatta informationsbehov utan att skapa nya lagrade tabeller.

Vi kan exempelvis fråga efter produkter vars pris ligger över ett genomsnitt, kunder som har en viss relation eller produkttyper som uppfyller ett sammanfattande villkor.

I varje steg beskriver vi **vilket resultat vi behöver från föregående steg**.

Det är nära besläktat med tanken bakom Views: vi kan arbeta med nya logiska representationer av redan lagrad information.

Skillnaden är att en subquery är en del av den fråga vi skriver just nu, medan en View kan få ett eget namn och återanvändas i andra frågor.

## Reflektion

1. När kan en fråga inuti en annan fråga vara lättare att förstå än en enda fråga med flera kopplade tabeller?
2. Vad är den begreppsliga skillnaden mellan att fråga vilka kunder som har en adress och att fråga hur många adresser varje kund har?
3. Varför kan det vara värdefullt att dela upp ett informationsbehov i flera logiska steg även när databasen skulle kunna lösa det på andra sätt?

# 11. SET-operationer

I en bokhandel kan vi ställa frågor som handlar om två olika grupper av information.

Vilka kunder har en adress i Malmö **eller** Storfors? Vilka har adresser i **båda** städerna? Och vilka har en adress i den ena staden **men inte** i den andra?

Vi har redan använt `JOIN` för att kombinera tabeller och subqueries för att låta en fråga använda resultatet av en annan.

Nu ska vi kombinera **resultaten från flera frågor**. SQL har särskilda mängdoperationer för detta: `UNION`, `INTERSECT` och `EXCEPT`.

## Grund

### Två Queries med samma slags resultat

Vi fortsätter med kunderna och deras adresser. För att kunna undersöka överlappningar lägger vi till några adresser.

**customers**

| id | name |
|---|---|
| 1 | Benjamin |
| 2 | Olof |
| 3 | Anaïs |
| 4 | Beatrice |
| 5 | Anna |

**addresses**

| id | customer_id | street | city |
|---|---|---|---|
| 1 | 1 | Hemmagatan 1 | Hemmavan |
| 2 | 1 | Storgatan 4 | Storfors |
| 3 | 2 | Parkvägen 2 | Storfors |
| 4 | 3 | NULL | Malmö |
| 5 | 1 | Strandgatan 7 | Malmö |
| 6 | 4 | Torget 3 | Malmö |
| 7 | 5 | Skolgatan 8 | Storfors |

Vi kan ta fram kundernas id för adresser i Storfors:

```
SELECT customer_id
FROM addresses
WHERE city = 'Storfors';
```

Resultat: 1, 2, 5.

Och kunder med adresser i Malmö:

```
SELECT customer_id
FROM addresses
WHERE city = 'Malmö';
```

Resultat: 3, 1, 4.

Båda frågorna ger samma **slags resultat**: en kolumn som innehåller kund-id. Därför kan vi kombinera dem med mängdoperationer.

## UNION – förena resultaten

Vi vill veta vilka kunder som har en adress i Storfors **eller** Malmö.

```
SELECT customer_id
FROM addresses
WHERE city = 'Storfors'

UNION

SELECT customer_id
FROM addresses
WHERE city = 'Malmö';
```

Resultatet innehåller kund-id 1, 2, 3, 4 och 5, var och en en gång.

`UNION` förenar resultat från flera frågor och tar som standard bort dubbletter.

Att Benjamin har adresser i båda städerna gör alltså inte att kund-id 1 visas två gånger.

## UNION ALL – behåll alla rader

Om vi i stället vill behålla alla resultatrader använder vi `UNION ALL`:

```
SELECT customer_id
FROM addresses
WHERE city = 'Storfors'

UNION ALL

SELECT customer_id
FROM addresses
WHERE city = 'Malmö';
```

Nu får vi sex rader. Kund-id 1 förekommer två gånger eftersom Benjamin har en adress i vardera staden.

Skillnaden är viktig:

- `UNION` ger ett resultat utan dubbletter.
- `UNION ALL` bevarar samtliga rader från frågorna.

Om vi behöver behålla information om antalet förekomster är `UNION ALL` det naturliga valet.

## INTERSECT – det som finns i båda resultaten

Vilka kunder har adresser i **både** Storfors och Malmö?

```
SELECT customer_id
FROM addresses
WHERE city = 'Storfors'

INTERSECT
```

```
SELECT customer_id
FROM addresses
WHERE city = 'Malmö';
```

Resultatet innehåller bara `1`, alltså Benjamin.

`INTERSECT` väljer de rader som förekommer i båda resultaten.

Vi kombinerar inte kolumner från två tabeller, som vid en `JOIN`. Vi jämför i stället två resultat med samma struktur.

## EXCEPT – det som finns i det ena men inte det andra

Vilka kunder har en adress i Storfors, men **inte** i Malmö?

```
SELECT customer_id
FROM addresses
WHERE city = 'Storfors'

EXCEPT

SELECT customer_id
FROM addresses
WHERE city = 'Malmö';
```

Resultatet innehåller `2` och `5`, alltså Olof och Anna.

`EXCEPT` väljer rader från det första resultatet som inte förekommer i det andra.

Ordningen mellan frågorna spelar därför roll. Om vi byter plats på dem får vi i stället kunder med adress i Malmö men inte i Storfors.

## Från id till kundnamn

Mängdoperationerna ovan ger kund-id. Men vi vill kanske se namn i stället.

Vi kan kombinera mängdoperationer med de konstruktioner vi redan kan:

```
SELECT name
FROM customers
WHERE id IN (
    SELECT customer_id
    FROM addresses
    WHERE city = 'Storfors'

    INTERSECT

    SELECT customer_id
    FROM addresses
    WHERE city = 'Malmö'
);
```

| name |
| --- |
| Benjamin |

Här får `INTERSECT` först fram kund-id som förekommer i båda städerna. Den yttre frågan hämtar sedan kundernas namn.

### Krav på Queries och deras resultat

För att kombinera två frågor med `UNION`, `INTERSECT` eller `EXCEPT` måste de ge **samma antal kolumner**. Kolumnernas datatyper måste dessutom vara förenliga så att resultaten kan kombineras.

Vi kan exempelvis kombinera två frågor som båda ger `customer_id`.

Däremot kan vi inte utan vidare förena en fråga med en kolumn och en annan fråga med tre kolumner.

Kolumnernas namn i det sammansatta resultatet hämtas vanligtvis från den första frågan.

## Avancerat

### Mängder och rader

I matematiken innehåller en mängd varje element högst en gång.

Men SQL arbetar ofta med resultat där samma värden kan förekomma på flera rader.

Detta brukar beskrivas som en skillnad mellan **mängdsemantik** och **multimängdssemantik** (*bag semantics*).

`UNION`, `INTERSECT` och `EXCEPT` tar normalt bort upprepade resultatrader. `UNION ALL` behåller dem.

Vissa SQL-dialekter har även `INTERSECT ALL` och `EXCEPT ALL`, som tar hänsyn till antalet förekomster. Stödet varierar mellan databassystem.

## Dubbletter handlar om hela raden

Anta att två frågor ger en kunds namn och stad:

```
SELECT c.name, a.city
FROM customers AS c
INNER JOIN addresses AS a
    ON a.customer_id = c.id;
```

Då betraktas `('Benjamin', 'Malmö')` och `('Benjamin', 'Storfors')` som **olika rader**, trots att kundnamnet är samma.

När `UNION` tar bort dubbletter jämförs hela resultatraden, inte en enskild kolumn som vi själva tycker är viktig.

Det betyder också att två olika kunder med samma namn kan se ut som en dubblett om vi bara väljer `name`. Om kundernas identitet spelar roll bör vi också välja exempelvis `id`.

## Ordningen i resultatet

En mängdoperation garanterar inte vilken ordning raderna visas i.

Vill vi sortera det sammansatta resultatet skriver vi en `ORDER BY` efter hela konstruktionen:

```
SELECT customer_id
FROM addresses
WHERE city = 'Storfors'

UNION

SELECT customer_id
FROM addresses
WHERE city = 'Malmö'

ORDER BY customer_id;
```

Här sorterar vi det **gemensamma resultatet**.

## Prioritet och parenteser

När vi kombinerar flera mängdoperationer måste vi tänka på i vilken ordning de utvärderas.

`INTERSECT` har enligt SQL-standarden högre prioritet än `UNION` och `EXCEPT`, men vi bör inte förutsätta att alla dialekter hanterar sammansatta uttryck identiskt.

Med parenteser eller namngivna mellanresultat kan vi göra avsikten tydligare.

Det är ofta bättre att dela upp en svår mängdoperation i flera steg än att förlita sig på att läsaren känner till operatorernas prioritet.

## När använder vi JOIN, subqueries och mängdoperationer?

De tre konstruktionerna löser delvis överlappande, men olika slags problem.

| Konstruktion | Naturlig användning |
|---|---|
| `JOIN` | Kombinera relaterad information från flera tabeller |
| Subquery | Använda ett Query-resultat som del av en annan fråga |
| `UNION` | Förena resultatrader från flera frågor |
| `INTERSECT` | Hitta resultatrader som finns i båda frågorna |

| Konstruktion | Naturlig användning |
|---|---|
| `EXCEPT` | Hitta resultatrader som finns i den första men inte den andra frågan |

Vi kan exempelvis använda `EXISTS` för att ta reda på om en kund har en adress i båda städerna. Resultatet kan bli detsamma som med `INTERSECT`.

Vi väljer då den konstruktion som tydligast beskriver vårt informationsbehov. Precis som för subqueries och `JOIN` bör vi inte anta att ett skrivsätt alltid är snabbare utan att undersöka exekveringsplanen.

## Dialektskillnader

`UNION` och `UNION ALL` har brett stöd.

`INTERSECT` och `EXCEPT` finns i SQL-standarden och stöds av bland annat PostgreSQL, SQLite och nyare MySQL-versioner. MySQL införde dessa operatorer i version 8.0.31.[13]

SQL Server stöder också `INTERSECT` och `EXCEPT`, men vissa andra delar av syntaxen och stödet för `ALL`-varianter skiljer sig åt.

## Från flera frågor till ett begrepp

När vi tar fram ”kunder med adresser i både Malmö och Storfors” har vi definierat en grupp som inte finns lagrad som en egen tabell.

Gruppen uppstår genom villkoren i våra frågor.

Vi kan alltså skapa **meningsfulla kategorier ur befintlig information**, utan att behöva lagra ytterligare en kolumn på kunderna för varje möjlig kategori.

Det knyter an till vår tidigare diskussion om datamodellering och Views: vi kan skapa nya sätt att betrakta informationen genom att formulera frågor om den.

## Reflektion

1. Varför kan det vara viktigt att behålla dubbletter i vissa resultat men ta bort dem i andra?
2. Vad är skillnaden mellan att kombinera tabeller med `JOIN` och att kombinera Query-resultat med `UNION`?
3. När vi skapar en grupp som ”kunder med adresser i båda städerna”, är det då en egenskap hos kunden eller ett resultat av hur vi väljer att undersöka informationen?

# 12. Att skapa tabeller

Hittills har vi arbetat med bokhandelns kunder, adresser och produkter som om tabellerna redan fanns.

Men om vi faktiskt bygger databasens tabeller, vilka uppgifter ska en produkt ha? Vilka värden ska vi tillåta? Vilka begränsningar och regler vill vi ha – vad händer om någon försöker registrera en produkt utan titel?

När vi skapar en tabell går vi från vår **idé om informationen** till en struktur för praktisk användning.

## Grund

### Från modell till tabell

Vi har beskrivit bokhandelns produkter med fyra attribut:

| Attribut | Betydelse | Datatyp |
|---|---|---|
| `id` | Identifierar produkten | `INTEGER` |
| `title` | Produktens titel | `VARCHAR(200)` |
| `price` | Produktens pris | `DECIMAL(10, 2)` |
| `stock` | Antalet exemplar i lager | `INTEGER` |

Nu kan vi skapa tabellen med `CREATE TABLE`:

```
CREATE TABLE products (
    id INTEGER,
    title VARCHAR(200),
    price DECIMAL(10, 2),
    stock INTEGER
);
```

Vi anger tabellens namn och därefter kolumnerna inom parentes. Varje kolumn har ett namn och en datatyp. Kommatecken skiljer kolumndefinitionerna åt.

Kommandot skapar själva **strukturen**, inte produkterna. Tabellen är fortfarande tom.

## En tabell behöver inte innehålla data

Vi kan kontrollera att en tom tabell finns genom att försöka läsa den:

```
SELECT *
FROM products;
```

Resultatet innehåller inga rader, men tabellen har fortfarande sina fyra kolumner.

Det är samma skillnad som mellan att bestämma vad en produkt *kan vara* och att faktiskt registrera en produkt.

Att lägga till själva produkterna gör vi med `INSERT` i nästa kapitel.

## Att välja rätt datatyper

Vi känner redan till `INTEGER`, `VARCHAR` och `DECIMAL`. Nu använder vi dem för att bestämma vilka värden kolumnerna ska kunna representera.

Exempelvis:

```
title VARCHAR(200)
price DECIMAL(10, 2)
```

Det betyder att titeln är text med högst `200` tecken och priset ett exakt decimaltal med tio siffrors precision och två decimalplatser.

Vi bör välja datatyp efter uppgiftens **betydelse**, inte bara efter hur den råkar skrivas. Ett postnummer kan exempelvis vara text trots att det består av siffror.

## NOT NULL – när en uppgift krävs

En produkt utan titel är kanske inte en giltig produkt i vår bokhandel.

Vi kan därför ange att kolumnen inte får sakna värde:

```
CREATE TABLE products (
    id INTEGER,
    title VARCHAR(200) NOT NULL,
    price DECIMAL(10, 2),
    stock INTEGER
);
```

`NOT NULL` säger att kolumnen måste innehålla ett värde som inte är `NULL`.

Det betyder däremot inte att texten är meningsfull. En tom sträng `''` kan fortfarande vara möjlig att lagra. Regeln hindrar ett saknat värde, inte varje tänkbart kvalitetsproblem.

## PRIMARY KEY – att identifiera en rad

Vi har tidigare använt `id` för att hitta en bestämd kund eller produkt.

När vi skapar tabellen kan vi ange att `id` är dess **primärnyckel**:

```
CREATE TABLE products (
    id INTEGER PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    price DECIMAL(10, 2),
    stock INTEGER
);
```

En primärnyckel identifierar varje rad. Dess värde ska vara unikt och får inte vara `NULL`.

Vi ska senare ägna ett helt kapitel åt nycklar och begränsningar. Här räcker det att se hur ett av våra viktigaste modelleringsbeslut blir en del av tabellen.

## DEFAULT – ett förvalt värde

Anta att vi registrerar en produkt men ännu inte har några exemplar av den i lager.

Vi kan ange ett standardvärde för `stock`:

```
CREATE TABLE products (
    id INTEGER PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    stock INTEGER NOT NULL DEFAULT 0
);
```

`DEFAULT 0` betyder att databasen använder värdet noll när vi skapar en rad utan att ange något lagersaldo.

Observera att `DEFAULT` inte hindrar andra värden. Regeln avgör bara vad som används när ett värde utelämnas.

## CHECK – tillåt bara rimliga värden

Datatypen `INTEGER` tillåter negativa tal. Men vår modell säger att lagersaldot inte får vara negativt.

Vi kan uttrycka det med `CHECK`:

```
CREATE TABLE products (
    id INTEGER PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    stock INTEGER NOT NULL DEFAULT 0,
    CHECK (price >= 0),
    CHECK (stock >= 0)
);
```

Om någon försöker registrera ett negativt lagersaldo ska databasen neka ändringen.

Vi har nu uttryckt tre olika beslut:

- `INTEGER`: värdet är ett heltal.

- `NOT NULL`: värdet måste finnas.
- `CHECK (stock >= 0)`: värdet får inte vara negativt.

Tillsammans gör de vår modell mer precis.

### Att skapa fler tabeller

Vi kan använda samma princip för bokhandelns kunder:

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    email VARCHAR(200),
    birthday DATE
);
```

Vi kräver ett namn, men tillåter att e-postadress och födelsedatum saknas.

Och för produkttyperna:

```
CREATE TABLE product_types (
    id INTEGER PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);
```

Produkttyperna representerar **referensdata**: en begränsad uppsättning kategorier som vi använder när vi beskriver produkter.

Produkterna kan sedan hänvisa till en typ med `product_type_id`. Exakt hur vi säkrar den relationen återkommer vi till i kapitlet om främmande nycklar.

## Avancerat

### Databasstruktur som ett kontrakt

När vi skapar tabellen beskriver vi inte bara hur vi tänker använda informationen. Vi anger också regler som databasen ska tillämpa på framtida förändringar.

Det fungerar som ett **kontrakt** mellan datamodellen och de program eller personer som skriver data.

Om priset inte får vara negativt behöver inte varje program uppfinna sin egen version av den grundläggande kontrollen.

Men databasen kan inte automatiskt avgöra alla verksamhetsfrågor. Vad ett rimligt pris är, om en bok bör säljas och vem som får ändra priset är andra slags beslut.

## Kolumnbegränsningar och tabellbegränsningar

Vi kan skriva vissa regler direkt efter en kolumn:

```
price DECIMAL(10, 2) NOT NULL
```

Andra regler kan stå som separata begränsningar:

```
CHECK (price >= 0)
```

En tabellbegränsning kan även omfatta flera kolumner. Föreställ dig att vi längre fram registrerar både ett startdatum och ett slutdatum för en kampanj:

```
CHECK (ends_on >= starts_on)
```

Då uttrycker regeln ett förhållande mellan två attribut.

## Beroenden mellan tabeller

Vi har organiserat produkttypens namn i `product_types` och låter produkten lagra `product_type_id`.

En möjlig definition är:

```
CREATE TABLE products (
    id INTEGER PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    stock INTEGER NOT NULL DEFAULT 0,
    product_type_id INTEGER,
```

```
    CHECK (price >= 0),
    CHECK (stock >= 0),
    FOREIGN KEY (product_type_id)
        REFERENCES product_types(id)
);
```

Nu uttrycker vi även **relationen** mellan produkterna och deras typer. En produkt kan inte hänvisa till en produkttyp som saknas i referenstabellen.

Vi behöver skapa `product_types` före `products` om tabellen ska referera till den direkt.

Hur främmande nycklar fungerar och vilka regler vi vill ha för relationer behandlar vi mer utförligt längre fram.

## Datatyper och SQL-dialekter

Exemplen i kapitlet är avsiktligt standardnära, men databassystemen skiljer sig åt.

Särskilt skapandet av automatiskt genererade id-värden varierar. MySQL använder ofta `AUTO_INCREMENT`, PostgreSQL har bland annat identitetskolumner och SQL Server använder `IDENTITY`.

Vi har därför inte låtit databasen generera id-värden automatiskt i exemplen. Vi återkommer till detta när vi lägger till data.[14]

Även `CHECK` stöds och tillämpas olika i äldre databassystem. Exempelvis började MySQL tillämpa `CHECK`-begränsningar från version 8.0.16.[15]

## Att förändra en befintlig struktur

Vad händer om vi vill lägga till `product_type_id` när tabellen redan finns?

Då behöver vi ändra strukturen i stället för att skapa tabellen på nytt. SQL har bland annat `ALTER TABLE`:

```
ALTER TABLE products
ADD COLUMN product_type_id INTEGER;
```

Vi går igenom förändringar av befintliga tabeller mer ordentligt i ett senare kapitel. Där tar vi också upp vad som händer när tabellen redan innehåller data.

## DROP TABLE – att ta bort strukturen

Vi kan också ta bort en tabell:

```
DROP TABLE products;
```

Detta tar normalt bort både tabellstrukturen och dess data.

Kommandot är därför något helt annat än att ta bort en enskild produkt. Vi ska skilja noga mellan att förändra **databasens struktur** och att förändra **raderna i en tabell**.

## Modellen är alltid ett urval

Vi har bestämt att en produkt har en titel, ett pris och ett lagersaldo.

Men en verklig bokhandel skulle kanske också behöva ISBN, författare, språk, upplaga, leverantörer och flera olika lagerplatser.

Modellen är alltså inte en fullständig beskrivning av allt som en produkt *kan vara*. Den är en beskrivning av den information vi för tillfället behöver kunna hantera på ett kontrollerat sätt.

När behoven ändras kan modellen behöva utvecklas. Det är en anledning till att vi skiljer mellan datamodellering, databasstruktur och applikationernas arbetssätt.

## Reflektion

1. Vad är skillnaden mellan att en kolumn har datatypen `INTEGER` och att vi kräver att dess värde ska vara större än eller lika med noll?
2. Vilka regler bör databasen själv upprätthålla, och vilka beslut kan behöva fattas någon annanstans?
3. När vi skapar tabellen `products`, har vi då beskrivit vad en produkt *är*, eller vad vi just nu behöver *veta om* en produkt?

# 13. Nycklar och relationer

En bokhandel kan ha flera kunder med samma namn, flera böcker med samma titel och flera adresser på samma gata. Hur vet vi då vilken kund, produkt eller adress vi menar?

Vi behöver kunna identifiera en bestämd post. Men vi behöver också kunna beskriva hur olika poster hör ihop: vilken kund en adress tillhör, eller vilken produkttyp en bok har.

En nyckel är inte bara ett tekniskt nummer. Den hjälper oss att uttrycka **identitet och samband** i vår modell.

## Grund

### Primärnyckeln identifierar en rad

Vi har tidigare arbetat med kunderna:

**customers**

| id | name |
|---|---|
| 1 | Benjamin |
| 2 | Olof |
| 3 | Anaïs |

Kolumnen `id` är vår **primärnyckel**, eller *primary key*.

Den identifierar varje rad i tabellen. Två kunder kan heta Olof, men de ska inte ha samma `id`.

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    email VARCHAR(200),
```

```
    birthday DATE
);
```

En primärnyckel ska vara **unik** och får inte vara `NULL`. När vi skriver:

```
SELECT *
FROM customers
WHERE id = 2;
```

menar vi den bestämda kundrad vars `id` är `2`, oavsett om andra kunder råkar ha samma namn.

## Varför räcker inte namnet?

Vi skulle kunna försöka använda kundens namn som identifikation, men namn är inte säkert unika. De kan dessutom förändras.

Samma problem gäller exempelvis boktitlar: olika böcker kan heta samma sak och en bok kan få en annan titel i en ny utgåva.

Därför använder vi ofta en separat identifierare, som `id`, i stället för ett beskrivande attribut.

Det är ett modelleringsbeslut: vi skiljer mellan **vem eller vad något är** och **hur vi beskriver det**.

## Främmande nycklar uttrycker samband

Vi har också tabellen `addresses`:

| id | customer_id | street | city |
|---|---|---|---|
| 1 | 1 | Hemmagatan 1 | Hemmavan |
| 2 | 1 | Storgatan 4 | Storfors |
| 3 | 2 | Parkvägen 2 | Storfors |

`customer_id` anger vilken kund adressen hör till. Det är en **främmande nyckel**, eller *foreign key*.

En främmande nyckel innehåller ett värde som refererar till en nyckel i en annan tabell.

```
CREATE TABLE addresses (
    id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    street VARCHAR(200),
    postal_code VARCHAR(20),
    city VARCHAR(100),
    FOREIGN KEY (customer_id)
        REFERENCES customers(id)
);
```

Detta uttrycker att värdet i `addresses.customer_id` måste referera till en existerande rad i `customers`.

Om en adress har `customer_id = 2` pekar den på Olofs kundrad, inte på adressen med `id = 2`.

## Referensintegritet

Tänk att vi försöker registrera en adress som har `customer_id = 99`, men att det inte finns någon kund med `id = 99`.

Utan en främmande nyckel skulle databasen kunna acceptera referensen, trots att sambandet inte går att följa.

Med `FOREIGN KEY` kan databasen kontrollera att relationen är giltig. Det kallas **referensintegritet**.

En främmande nyckel behöver alltså inte ha ett unikt värde. Benjamin kan ha två adresser, båda med `customer_id = 1`. Det är precis vad vår en-till-många-relation innebär.

## En relation mellan produkter och produkttyper

Vi har organiserat produktens typ i en egen tabell:

**product_types**

| id | name |
|---|---|
| 1 | Roman |
| 2 | Novellsamling |

Varje produkt innehåller då ett `product_type_id` som hänvisar till produkttypen.

```
CREATE TABLE products (
    id INTEGER PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    stock INTEGER NOT NULL DEFAULT 0,
    product_type_id INTEGER,
    CHECK (price >= 0),
    CHECK (stock >= 0),
    FOREIGN KEY (product_type_id)
        REFERENCES product_types(id)
);
```

Vi kan nu använda `JOIN` för att visa produkterna tillsammans med namnen på deras typer, utan att behöva lagra typnamnet på varje produkt.

Notera att `product_type_id` här får vara `NULL`: vi har inte skrivit `NOT NULL`. Vi tillåter alltså att en produkt ännu saknar registrerad typ.

## UNIQUE – en annan sorts unikhet

En tabell har en primärnyckel, men kan ha andra kolumner som också behöver innehålla unika värden.

Exempelvis kanske varje produkttyp ska ha ett eget namn:

```
CREATE TABLE product_types (
    id INTEGER PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);
```

`UNIQUE` förhindrar att samma värde förekommer flera gånger i kolumnen enligt databasens jämförelseregler.

Skillnaden är att **primärnyckeln identifierar raden**, medan `UNIQUE` kan skydda ytterligare värden från att upprepas.

Reglerna för `NULL` i en `UNIQUE`-kolumn skiljer sig mellan databassystem.[16]

### När vi skapar tabellerna spelar ordningen roll

Om `addresses` ska referera till `customers` behöver kundtabellen normalt finnas när vi definierar den främmande nyckeln.

En lämplig ordning är alltså:

1. Skapa `customers`.
2. Skapa `addresses` med en främmande nyckel till `customers`.

Detsamma gäller `product_types` före `products` när produkterna har en främmande nyckel till typerna.

Vi har därmed skapat en struktur som kan ta emot data och samtidigt kontrollera några av relationerna mellan den.

## Avancerat

### Vad händer när en relaterad rad tas bort?

Anta att Benjamin har registrerade adresser. Vad ska hända om hans kundrad tas bort?

Databasen kan inte gissa vad verksamheten vill.

När vi skapar den främmande nyckeln kan vi ange en regel, exempelvis:

```
FOREIGN KEY (customer_id)
    REFERENCES customers(id)
    ON DELETE RESTRICT
```

Det innebär att vi inte kan ta bort kunden så länge adresser refererar till honom.

Ett annat alternativ är:

```
FOREIGN KEY (customer_id)
    REFERENCES customers(id)
    ON DELETE CASCADE
```

Då tas även de adresser som refererar till kunden bort när kundraden tas bort.

Andra alternativ kan bland annat innebära att referensen sätts till `NULL` om kolumnen tillåter det.

Det här är **inte bara syntaxval**. Olika alternativ uttrycker olika regler om relationens betydelse, och `CASCADE` kan få stora konsekvenser när många poster hör ihop.[17]

## När flera kolumner tillsammans är en nyckel

En primärnyckel måste inte bestå av en enda kolumn.

Föreställ dig att vi har en tabell som anger vilka författare som hör till vilka produkter:

### product_authors

| product_id | author_id |
|---|---|
| 1 | 4 |
| 1 | 7 |
| 2 | 4 |

En produkt kan ha flera författare, och en författare kan vara knuten till flera produkter.

Här kan **kombinationen** av `product_id` och `author_id` vara unik:

```
CREATE TABLE product_authors (
    product_id INTEGER NOT NULL,
    author_id INTEGER NOT NULL,
    PRIMARY KEY (product_id, author_id),
    FOREIGN KEY (product_id)
        REFERENCES products(id),
    FOREIGN KEY (author_id)
        REFERENCES authors(id)
);
```

Vi förutsätter här att tabellen `authors` redan finns. Vi bygger ut den modellen ordentligt i nästa kapitel.

Den sammansatta primärnyckeln förhindrar att vi registrerar samma produkt–författare-koppling två gånger.

## Identitet är inte alltid ett löpnummer

Vi använder oftast `id` som ett heltal i exemplen eftersom det är lätt att läsa.

Men i verkliga system kan en identifierare exempelvis vara en UUID eller bestå av flera attribut.

En identifierare behöver inte heller alltid vara automatiskt genererad. Det avgörande är att vi kan lita på dess identitet och reglerna kring den.

Vi ska skilja mellan en **naturlig nyckel** – ett identifierande värde från verksamheten – och en **surrogatnyckel** som skapas för databasmodellen, exempelvis ett löpnummer.

Vilket alternativ som passar bäst beror på hur stabilt det identifierande värdet är och hur modellen ska användas.

## Främmande nyckel är inte samma sak som normalisering

Vi har nu lärt oss att uttrycka att en adress hör till en kund eller att en produkt har en produkttyp.

Men bara för att vi använder främmande nycklar betyder det inte att vår modell automatiskt är väl organiserad.

Vi kan fortfarande lagra samma information på flera ställen, välja attribut som hör till fel entitet eller skapa beroenden som gör uppdateringar svåra.

Främmande nycklar skyddar **relationernas giltighet**. Normalisering hjälper oss att undersöka **var informationen hör hemma** och vilka beroenden som finns mellan attribut.

Det är nästa steg.

### Egen information och information om någon annan

Tänk dig en produkt med följande attribut:

```
title: Vägen hem
author_name: Anna Svensson
product_type_name: Roman
```

Titeln beskriver själva produkten. Men `author_name` beskriver en författare och `product_type_name` beskriver en produkttyp.

Vi kan fråga: **Vilken företeelse äger egentligen den här informationen?**

Det är inte alltid så enkelt som att varje attribut bara har en självklar hemvist. Men frågan hjälper oss att upptäcka upprepningar och möjliga relationer.

I nästa kapitel använder vi denna tanke som ingång till datamodellering och normalformerna.

## Reflektion

1. Varför kan vi behöva en primärnyckel även när varje kund redan har ett namn?

2. Vad innebär det för verksamheten att välja `ON DELETE CASCADE` i stället för att förhindra borttagningen?
3. Vad är skillnaden mellan att en relation är giltig och att informationen är organiserad på ett bra sätt?

# 14. Datamodellering och normalisering

En bokhandel behöver veta vilka produkter den säljer, vilka författare som skrivit böckerna och vilka produkttyper böckerna tillhör.

Vi skulle kunna lagra all information på varje produktrad. Då blir den lätt att läsa på ett ställe. Men om samma författare förekommer på hundra böcker behöver vi också skriva samma författarinformation hundra gånger.

Vad händer när författaren byter namn? Eller om vi vill registrera en författare som ännu inte har någon produkt i sortimentet?

När vi modellerar data behöver vi därför fråga oss: **Vilken information hör till vilken företeelse?**

Normalisering syftar till att undvika onödig dubbellagring av data, skapa god dataintegritet och göra databasen lättare att förändra och bygga ut. Om vi lagrar ett sakförhållande på flera ställen kan vi behöva ändra det överallt – och riskerar att missa något.

Det finns flera **normalformer**. Varje normalform ställer ytterligare krav på hur informationen organiseras. Målet är inte att skapa så många tabeller som möjligt, utan att varje uppgift ska ha en tydlig plats och att samband ska kunna beskrivas utan onödig upprepning.

## Grund

### Egen information och information om något annat

Tänk dig att vi börjar med en enda tabell för böcker:

**products**

| id | title | price | author_name | author_email | product_type |
|---|---|---|---|---|---|
| 1 | Vägen hem | 200.00 | Anna Svensson | anna@authors.example | Roman |
| 2 | Den blå staden | 150.00 | Erik Lund | erik@authors.example | Roman |
| 3 | Vinterljus | 249.00 | Anna Svensson | anna@authors.example | Roman |
| 4 | Små historier | 120.00 | Erik Lund | erik@authors.example | Novellsamling |

Här blandar vi tre slags information:

- Produktens egen information: titel och pris.
- Information om en annan företeelse: författarens namn och e-postadress.
- En klassificering: produkttypen.

Tabellen fungerar för våra fyra böcker. Men vissa uppgifter upprepas.

Att skilja mellan **egen data och främmande data** är en användbar första tanke när vi förbättrar modellen. Det är dock inte en fullständig definition av normalisering. För att förstå varför vi ska dela upp informationen behöver vi också undersöka beroenden mellan värden.

## När information upprepas

Anna Svenssons namn och e-postadress förekommer på två produktrader.

Det kan skapa flera problem:

**Uppdateringsproblem:** Om Anna byter e-postadress måste vi ändra informationen på flera ställen. Missar vi ett av dem får databasen motstridiga uppgifter.

**Infogningsproblem:** Om vi vill registrera en ny författare innan vi har en bok att koppla till författaren finns ingen naturlig rad att lägga informationen på.

**Borttagningsproblem:** Om vi tar bort den sista boken av Erik Lund riskerar även den enda lagrade informationen om honom att försvinna.

Detta brukar kallas *anomalier*. De är tecken på att vår struktur kan behöva förändras.

## Ge författaren en egen identitet

Vi kan börja med att skapa en egen tabell för författare:

### authors

| id | name | email |
|---|---|---|
| 1 | Anna Svensson | anna@authors.example |
| 2 | Erik Lund | erik@authors.example |

Då behöver vi inte skriva författarens namn och e-postadress på varje produkt.

Men hur kopplar vi produkten till författaren?

Vi skulle kunna lagra `author_id` i `products`. Det fungerar om varje produkt alltid har exakt en författare.

I vår bokhandel vill vi däremot kunna hantera böcker med flera författare.

## En kopplingstabell för flera-till-flera

I stället skapar vi `product_authors`:

### product_authors

| product_id | author_id |
|---|---|
| 1 | 1 |
| 2 | 2 |
| 3 | 1 |

| product_id | author_id |
|---|---|
| 4 | 2 |

Den innehåller relationer mellan produkter och författare.

Om en bok skrivits av två författare registrerar vi helt enkelt två kopplingar till samma produkt.

Med SQL kan vi uttrycka tabellen så här:

```
CREATE TABLE product_authors (
    product_id INTEGER NOT NULL,
    author_id INTEGER NOT NULL,
    PRIMARY KEY (product_id, author_id),
    FOREIGN KEY (product_id)
        REFERENCES products(id),
    FOREIGN KEY (author_id)
        REFERENCES authors(id)
);
```

Nu kan en författare finnas oberoende av produkterna, samtidigt som en produkt kan ha flera författare.

Vi har alltså inte bara *flyttat kolumner*. Vi har gjort relationen mellan företeelserna tydligare.

## Referensdata har också en egen plats

Vi har tidigare skapat tabellen `product_types`:

| id | name |
|---|---|
| 1 | Roman |
| 2 | Novellsamling |

I `products` lagrar vi därför `product_type_id`, inte typnamnet på varje rad.

Produkttypen är här **referensdata**: en kategori som används för att beskriva produkterna.

En författare är däremot inte bara ett tillåtet kategorivärde. Författaren är en självständig företeelse med egna attribut och en egen identitet.

Det är en av anledningarna till att samma sorts relationsteknik kan användas för olika informationsbegrepp.

## Vår uppdelade modell

Den modell vi nu använder kan beskrivas så här:

```
products
    id
    title
    price
    stock
    product_type_id  → product_types.id

product_types
    id
    name

authors
    id
    name
    email

product_authors
    product_id       → products.id
    author_id        → authors.id
```

En produkt har en produkttyp, och en produkttyp kan beskriva flera produkter.

En produkt kan ha flera författare, och en författare kan vara kopplad till flera produkter.

Vi kan sedan använda `JOIN` för att visa informationen tillsammans:

```
SELECT
    p.title,
    a.name AS author_name
FROM products AS p
INNER JOIN product_authors AS pa
    ON pa.product_id = p.id
```

```
INNER JOIN authors AS a
    ON a.id = pa.author_id;
```

Att informationen är uppdelad betyder alltså inte att vi förlorat möjligheten att använda den som en helhet.

## Normalformerna – från enkla regler till tydligare struktur

Vi kan beskriva de första tre normalformerna med tre korta frågor:

| Normalform | Fråga |
|---|---|
| **1NF** | Innehåller varje cell ett enskilt värde, eller har vi grupperat flera värden i samma kolumn? |
| **2NF** | Hör alla icke-nyckelattribut till **hela** nyckeln, eller beskriver något bara en del av den? |
| **3NF** | Beror ett attribut direkt på nyckeln, eller beskriver det egentligen något som bestäms av ett annat attribut? |

Vi börjar med denna praktiska förståelse och undersöker sedan hur kraven kan beskrivas mer exakt.

## Första normalformen: en uppgift per värde

Anta att vi registrerar författare så här:

| product_id | title | authors |
|---|---|---|
| 1 | Vägen hem | Anna Svensson, Erik Lund |

Kolumnen `authors` innehåller här en kommaseparerad lista av författare.

Det verkar praktiskt, men det blir svårt att identifiera varje författare och kontrollera relationerna mellan produkter och författare.

Vi vill i stället representera författarna var för sig och lagra varje koppling som en egen rad i `product_authors`.

Det här illustrerar **första normalformen (1NF)**: varje cell ska innehålla ett **atomärt värde** – inte en lista av flera författare som vi förväntas behandla var för sig.

Vad som räknas som atomärt avgörs av datamodellen och vilka operationer vi behöver kunna utföra. En textsträng eller ett datum kan exempelvis vara ett enda värde även om det går att dela upp i tecken eller datumdelar.

## Andra normalformen: beroende på hela nyckeln

Tänk att vi skapar följande tabell:

| product_id | author_id | product_title | author_name |
|---|---|---|---|
| 1 | 1 | Vägen hem | Anna Svensson |
| 1 | 2 | Vägen hem | Erik Lund |

Vi antar att kombinationen `(product_id, author_id)` identifierar varje rad.

Men `product_title` beror bara på `product_id`. Och `author_name` beror bara på `author_id`.

Informationen hör alltså inte till *hela kombinationen*. Den hör till varsin företeelse.

Genom att flytta titel till `products` och namn till `authors` och behålla `product_authors` som en ren koppling kan vi undvika dessa **partiella beroenden**.

Det är grundidén bakom **andra normalformen (2NF)**: ett icke-nyckelattribut ska beskriva något som hör till **hela** nyckeln.

Som praktisk tumregel kan vi fråga oss om tabellen verkligen beskriver en och samma företeelse eller relation. Men 2NF handlar mer specifikt om **partiella beroenden av sammansatta kandidatnycklar**. En

tabell kan uppfylla 2NF och ändå innehålla information om andra företeelser som gör att den bryter mot 3NF.

### Tredje normalformen: beroenden via andra attribut

Anta att vi har:

| product_id | title | product_type_id | product_type_name |
|---|---|---|---|
| 1 | Vägen hem | 1 | Roman |
| 3 | Vinterljus | 1 | Roman |
| 4 | Små historier | 2 | Novellsamling |

Vi har `product_id` som primärnyckel.

Produkttypens namn bestäms av `product_type_id`. Det finns alltså ett beroende från produktens identifierare till typ-id, och vidare från typ-id till typnamnet.

Vi kan undvika upprepningen genom att lägga `product_type_name` i `product_types` och bara lagra `product_type_id` i `products`.

Det är grundtanken bakom **tredje normalformen (3NF)**: en uppgift ska inte vara indirekt beroende av nyckeln via ett annat icke-nyckelattribut.

Produktens typ-id beskriver vilken typ produkten tillhör. Typens namn beskriver däremot själva produkttypen, inte produkten. Vi lagrar därför namnet i `product_types`.

Detta är en användbar förenkling. Den mer exakta definitionen, som också tar hänsyn till kandidatnycklar, följer under *Avancerat*.

# Avancerat

## Funktionella beroenden

För att förstå normalisering mer noggrant behöver vi begreppet **funktionellt beroende**.

Vi skriver:

```
product_id → title, price, stock
```

Det betyder att ett bestämt `product_id` avgör vilka värden `title`, `price` och `stock` har i vår modell.

Vi kan också skriva:

```
author_id → name, email
```

Det betyder inte att en författare aldrig kan byta namn eller e-postadress. Det betyder att varje identifierad författare vid en given tidpunkt har ett bestämt värde för de attribut som vi modellerar.

Ett funktionellt beroende handlar alltså om de regler vi förutsätter för vår information, inte bara om vilka råkar vara de aktuella värdena i databasen.

## Kandidatnycklar och en mer precis syn på normalformerna

En **kandidatnyckel** är en minimal uppsättning attribut som kan identifiera en rad unikt.

En tabell kan ha flera kandidatnycklar. Den vi väljer som primärnyckel är bara en av dem.

I mer formell terminologi handlar 2NF om att varje attribut som inte ingår i någon kandidatnyckel är fullt funktionellt beroende av varje kandidatnyckel.

3NF kräver dessutom att för varje icke-trivialt funktionellt beroende $X \rightarrow A$ gäller att $X$ är en supernyckel eller att $A$ är ett *primattribut*, alltså ingår i någon kandidatnyckel.

Det innebär att den vanliga minnesregeln om ”beroende av nyckeln och inget annat än nyckeln” är användbar, men inte en fullständig definition.

## Boyce–Codd och högre normalformer

**Boyce–Codd-normalformen (BCNF)** skärper kravet på funktionella beroenden: varje determinant i ett icke-trivialt funktionellt beroende ska vara en supernyckel.

**Fjärde normalformen (4NF)** tar upp situationer där flera *oberoende flervärdesfakta* lagras tillsammans. Anta exempelvis att en produkt kan ha flera författare och samtidigt finnas i flera språkversioner, och att relationerna är oberoende av varandra. Om vi lagrar alla kombinationer av författare och språk i en enda tabell kan vi skapa onödig upprepning. Då kan separata relationer vara bättre.

**Femte normalformen (5NF)** handlar om relationer där **tre eller fler samband tillsammans** avgör vilka kombinationer som är giltiga.

Föreställ dig att bokhandeln har flera butiker och leverantörer. Vi vill beskriva vilka leverantörer som kan leverera vilka produkter till vilka butiker. Vi börjar med en tabell:

| supplier_id | product_id | store_id |
|---|---|---|
| 1 | 10 | 1 |
| 1 | 10 | 2 |
| 1 | 20 | 1 |

Varje rad beskriver en kombination av *leverantör, produkt och butik*.

Anta nu att verksamheten har en bestämd regel: **Om en leverantör kan leverera en produkt, leverantören samarbetar med en butik och butiken tar emot produkten, då gäller också kombinationen av alla tre.**

Då beskriver vår tabell egentligen tre slags samband:

- Vilka **produkter** varje **leverantör** kan leverera.
- Vilka **butiker** varje **leverantör** samarbetar med.
- Vilka **produkter** varje **butik** tar emot.

Vi skulle kunna lagra dessa samband i tre separata tabeller. En `JOIN` mellan samtliga tre kan då återskapa exakt de giltiga kombinationerna av leverantör, produkt och butik. Om vi behåller allt i samma tabell upprepar vi i stället samband som redan bestäms av de andra.

Det kallas ett **joinberoende**: en relation kan återskapas fullständigt genom att kombinera flera mindre relationer, utan att vi förlorar information eller skapar ogiltiga kombinationer.

Det avgörande är **verksamhetsregeln**. Om bokhandeln i stället avtalar varje enskild kombination av leverantör, produkt och butik separat, kan vi inte härleda alla kombinationer från de tre parvisa sambanden. Då skulle uppdelningen kunna skapa leveranser som aldrig avtalats.

5NF hjälper oss alltså att upptäcka när en komplex relation i själva verket kan beskrivas av flera mindre, oberoende relationer – **men bara när vi vet att uppdelningen är giltig för alla tillåtna datatillstånd**, inte bara för de exempelrader vi råkar ha just nu.

Det är alltså inte korrekt att 4NF bara betyder ”använd en kopplingstabell” eller att 5NF betyder ”ha flera kopplingar mellan samma entiteter”. Båda handlar om vilka beroenden som faktiskt gäller i informationen.

I de flesta praktiska databasprojekt är en genomtänkt modell som uppfyller 3NF en bra utgångspunkt, även om högre normalformer och medveten denormalisering ibland behöver övervägas.

För en mer utförlig men fortfarande pedagogisk förklaring av normalformerna rekommenderas William Kents *A Simple Guide to Five Normal Forms in Relational Database Theory*.[18]

## Normalisering är inte bara att dela upp tabeller

En modell med fler tabeller är inte automatiskt bättre normaliserad.

Om vi delar upp information utan att förstå beroendena kan vi skapa onödig komplexitet.

Vi kan också få problem om vi delar upp något som i verksamheten faktiskt är ett sammanhängande värde, eller om vi skapar otydliga identiteter och relationer.

Normalisering handlar om att **bevara informationens betydelse och beroenden** när vi organiserar den.

Ett bra mål är att en förändring av ett sakförhållande normalt ska behöva göras på ett enda logiskt ställe.

## När relationen har egna attribut

Föreställ dig att vi vill veta i vilken ordning författarna ska visas på en bok.

Då räcker det inte att bara veta vilka författare som hör till produkten. Vi behöver också beskriva **relationen**.

Vi kan lägga till `author_order` i `product_authors`:

```
CREATE TABLE product_authors (
    product_id INTEGER NOT NULL,
    author_id INTEGER NOT NULL,
```

```
    author_order INTEGER NOT NULL,
    PRIMARY KEY (product_id, author_id),
    FOREIGN KEY (product_id)
        REFERENCES products(id),
    FOREIGN KEY (author_id)
        REFERENCES authors(id),
    CHECK (author_order > 0),
    UNIQUE (product_id, author_order)
);
```

`author_order` beskriver inte författaren som person och inte heller produkten ensam. Det beskriver författarens plats **i förhållande till en viss produkt**.

Det är ett exempel på att även en relation kan ha egna meningsfulla attribut.

### Normalisering och historisk information

Anta att vi kopplar en orderrad direkt till produktens aktuella pris.

Om priset ändras i `products` skulle en senare fråga kunna ge intryck av att kunden betalade det nya priset vid den tidigare beställningen.

Därför kan vi behöva lagra **priset vid köptillfället** på orderraden, även om produktens aktuella pris redan finns i en annan tabell.

Det är inte nödvändigtvis onödig dubbellagring. Det är två olika sakförhållanden:

- Aktuellt pris på en produkt.
- Avtalat pris på en orderrad vid en viss tidpunkt.

Att lagra båda kan därför vara helt förenligt med en genomtänkt datamodell.

Normalisering betyder inte att alla värden som ser likadana ut måste lagras på exakt ett ställe. Vi behöver först förstå **vad de olika värdena betyder**.

### Normalisering och praktisk prestanda

Det kan finnas skäl att skapa särskilda, beräknade eller duplicerade representationer för rapportering och prestanda.

Det brukar kallas **denormalisering** när vi medvetet inför redundans eller förenklar en struktur för att stödja ett visst användningssätt.

Men först behöver vi veta vilken grundmodell och vilka beroenden vi utgår från. Annars är det svårt att avgöra vilka avvikelser vi gör och vilka problem de kan skapa.

I senare kapitel kommer vi att använda Views och andra tekniker för att erbjuda enklare representationer utan att alltid behöva förändra lagringsmodellen.

## Reflektion

1. Hur kan vi avgöra om en uppgift beskriver produkten själv, författaren eller relationen mellan dem?
2. När är upprepade värden ett problem, och när kan två lika värden representera olika sakförhållanden?
3. Varför bör vi förstå informationens beroenden innan vi bestämmer hur många tabeller modellen ska innehålla?

# 15. Att lägga till och förändra data

Hittills har vi skapat en modell för bokhandelns information. Vi har bestämt vad kunder, produkter och författare är, vilka uppgifter de har och hur de förhåller sig till varandra.

Men en bokhandel förändras hela tiden. Nya kunder registreras, produkter läggs till, priser justeras och vissa uppgifter behöver rättas.

Vi behöver därför kunna förändra **informationen i tabellerna**, utan att förändra själva tabellernas struktur. Samtidigt behöver vi förstå vilken verklig förändring varje instruktion representerar.

## Grund

### Att lägga till en rad med INSERT

Vi börjar med bokhandelns produkttyper. Tabellen har två rader:

**product_types**

| id | name |
|---|---|
| 1 | Roman |
| 2 | Novellsamling |

Vi vill lägga till en ny typ: Facklitteratur.

```
INSERT INTO product_types (id, name)
VALUES (3, 'Facklitteratur');
```

`INSERT INTO` anger vilken tabell vi vill lägga till data i. Inom parentes anger vi kolumnerna och efter `VALUES` anger vi motsvarande värden.

Nu innehåller tabellen också:

| id | name |
|---|---|
| 3 | Facklitteratur |

Vi har skapat **en ny rad**, inte en ny kolumn eller tabell.

## Att lägga till flera rader

Vi kan lägga till flera produkttyper i samma instruktion:

```
INSERT INTO product_types (id, name)
VALUES
    (4, 'Biografi'),
    (5, 'Barnbok');
```

Varje parentes beskriver en ny rad. Värdena måste motsvara kolumnerna i den ordning vi har angett dem.

## Att lägga till en produkt

Vi har också en tabell `products` med kolumnerna `id`, `title`, `price`, `stock` och `product_type_id`.

```
INSERT INTO products (
    id, title, price, stock, product_type_id
)
VALUES (
    6, 'Kunskapens vägar', 199.00, 8, 3
);
```

Nu har vi lagt till en produkt som tillhör produkttypen Facklitteratur.

Eftersom `product_type_id` är en främmande nyckel behöver produkttypen med `id = 3` redan finnas. Det är ett exempel på hur vår datamodell och dess regler påverkar vilka förändringar databasen tillåter.

## DEFAULT och värden vi utelämnar

När vi skapade `products` bestämde vi att `stock` har standardvärdet `0`:

```
stock INTEGER NOT NULL DEFAULT 0
```

Vi behöver därför inte ange lagersaldo när vi registrerar en produkt:

```
INSERT INTO products (
    id, title, price, product_type_id
)
VALUES (
    7, 'Om databaser', 225.00, 3
);
```

Den nya produkten får `stock = 0`.

Det är en viktig skillnad mellan att **utelämna en kolumn med ett standardvärde** och att uttryckligen ange värdet `NULL`.

`NULL` betyder att ett värde saknas, och en kolumn med `NOT NULL` tillåter inte det. Att en kolumn har `DEFAULT` gör inte `NULL` till ett giltigt värde.

## Att förändra befintliga data med UPDATE

Anta att bokhandeln får fem nya exemplar av `Vägen hem`. Det gamla lagersaldot är `12`.

Vi kan ändra det med `UPDATE`:

```
UPDATE products
SET stock = 17
WHERE id = 1;
```

`UPDATE` anger tabellen och `SET` beskriver det nya värdet. `WHERE` anger **vilka rader** som ska förändras.

Men vi behöver inte alltid känna till det nya värdet i förväg. Vi kan utgå från det gamla:

```
UPDATE products
SET stock = stock + 5
WHERE id = 1;
```

Det här skulle öka lagersaldot med fem utifrån det värde som finns när instruktionen utförs. **Kör alltså inte båda exemplen efter varandra** om syftet är att registrera en och samma leverans. Exemplen visar två alternativa sätt att göra uppdateringen.

## Att förändra flera kolumner

Vi kan ändra flera attribut samtidigt:

```
UPDATE products
SET
    price = 210.00,
    stock = 15
WHERE id = 1;
```

Här förändras både pris och lagersaldo på produkten med `id = 1`.

Vi bör välja `WHERE`-villkoret med omsorg. Primärnyckeln är ofta praktisk när vi avser en bestämd rad.

## Vad händer utan WHERE?

Betrakta detta:

```
UPDATE products
SET price = 100.00;
```

Här saknas `WHERE`.

Det betyder inte att databasen väljer en godtycklig produkt. Det betyder att **alla rader i tabellen uppdateras**.

Det kan förstås vara avsiktligt. Men om vi bara tänkt ändra priset på en produkt är det ett allvarligt misstag.

Ett enkelt arbetssätt är att först kontrollera urvalet:

```
SELECT id, title, price
FROM products
WHERE id = 1;
```

Och sedan använda samma villkor i uppdateringen.

## Att ta bort rader med DELETE

Om en produkt inte längre ska finnas kvar i vårt sortiment kan vi ta bort dess rad:

```
DELETE FROM products
WHERE id = 7;
```

`DELETE FROM` tar bort rader ur en tabell.

Precis som för `UPDATE` är `WHERE` avgörande:

```
DELETE FROM products;
```

Den senare instruktionen försöker ta bort **alla rader** i tabellen. Den tar däremot inte bort tabellens struktur.

Det skiljer `DELETE` från `DROP TABLE`, som vi mötte i föregående kapitel.

## Vilka förändringar är tillåtna?

Våra tidigare beslut om datatyper och begränsningar gäller också när vi skriver data.

Exempelvis ska denna instruktion nekas om vi har definierat `CHECK (stock >= 0)`:

```
UPDATE products
SET stock = -5
WHERE id = 1;
```

På samma sätt kan databasen neka en ändring som bryter mot en primärnyckel, en unikhetsregel eller en främmande nyckel.

Det är därför vi skapar **regler i datamodellen**, och inte bara kontroller i de program som använder databasen.

## Avancerat

### INSERT ... SELECT – skapa rader från ett annat resultat

Ibland vill vi inte skriva värdena manuellt. Vi vill lägga till rader utifrån en annan fråga.

Anta att vi har skapat en separat tabell för en sammanställning av produkter:

```
CREATE TABLE product_snapshot (
    product_id INTEGER,
    title VARCHAR(200),
    price DECIMAL(10, 2)
);
```

Vi kan fylla den med information från `products`:

```
INSERT INTO product_snapshot (
    product_id, title, price
)
SELECT id, title, price
FROM products
WHERE stock > 0;
```

Här är det `SELECT` som bestämmer vilka rader som läggs till. Det är ett exempel på hur vi kan återanvända våra tidigare kunskaper om urval och beräkningar när vi förändrar data.

Tabellen `product_snapshot` är bara ett fristående exempel. Vi inför den inte som en permanent del av bokhandelns grundmodell.

### Att kontrollera vad som förändrades

Efter en `UPDATE` eller `DELETE` vill vi ofta kontrollera hur många rader som faktiskt påverkades.

Många databasklienter visar antalet påverkade rader. Vissa SQL-dialekter kan dessutom returnera de förändrade raderna direkt, exempelvis med `RETURNING` eller `OUTPUT`.[19]

Detta är särskilt användbart när ett program vill avgöra om en uppdatering verkligen hittade den avsedda posten.

## När information måste bevaras

Att ta bort en produkt ur det aktuella sortimentet betyder inte nödvändigtvis att vi bör radera all information om den.

Om produkten tidigare har sålts behöver vi kanske fortfarande kunna se vad kunden köpte.

Vi kan då skilja mellan:

- Att **radera** produktens rad ur databasen.
- Att **markera** produkten som inaktiv, exempelvis med en kolumn `is_active`.

Det senare kallas ibland *soft delete* eller logisk radering.

Men att markera något som inaktivt är inte automatiskt bättre än att radera det. Det påverkar hur vi skriver våra frågor, hur vi bevarar historik och vilka uppgifter vi faktiskt behöver spara.

## När ett pris förändras

Anta att en bok kostar 200 kronor när en kund beställer den.

Nästa vecka ändrar bokhandeln priset till 225 kronor.

```
UPDATE products
SET price = 225.00
WHERE id = 1;
```

Det är helt rimligt att produktens **aktuella pris** förändras.

Men om vi senare tittar på en gammal beställning måste vi fortfarande kunna veta vilket pris kunden faktiskt betalade.

Därför kan en orderrad behöva spara `unit_price` – priset som gällde vid just den beställningen.

Vi har alltså två olika sakförhållanden: produktens pris *nu* och den genomförda försäljningens pris *då*.

Att bevara båda är inte ett misslyckande med normaliseringen. Det är ett uttryck för att informationen betyder olika saker.

## Flera förändringar som hör ihop

Föreställ dig att en kund köper två exemplar av en bok.

Då kanske vi behöver både registrera beställningen och minska lagersaldot.

Vad händer om beställningen registreras, men uppdateringen av lagersaldot misslyckas?

Databaser kan hantera flera förändringar som en sammanhängande **transaktion**:

```
START TRANSACTION;

UPDATE products
SET stock = stock - 2
WHERE id = 1 AND stock >= 2;

-- Ytterligare förändringar som hör till köpet

COMMIT;
```

Det här är bara en illustration, **inte ett färdigt och säkert orderflöde**. Ett verkligt system måste bland annat kontrollera att uppdateringen påverkade rätt rad och hantera fel innan transaktionen avslutas.

Om något går fel kan vi i en pågående transaktion använda `ROLLBACK` i stället för `COMMIT` för att ångra de förändringar som hör till transaktionen.[20]

Vi går igenom transaktioner, samtidighet och isolering på djupet i ett senare kapitel.

## Samtidiga uppdateringar

Anta att två anställda samtidigt försöker ändra lagersaldot.

Om båda först läser värdet 12 och sedan skriver tillbaka var sitt uträknade resultat riskerar en av ändringarna att gå förlorad.

En instruktion som:

```
UPDATE products
SET stock = stock + 5
WHERE id = 1;
```

låter databasen utföra beräkningen på det lagrade värdet inom själva uppdateringen. Det är ofta bättre än att först läsa värdet i en applikation och sedan skicka tillbaka ett nytt tal.

Men även då behöver vi förstå transaktioner och samtidighet när flera operationer tillsammans ska vara korrekta.

## Att ändra data är att ändra ett påstående

När vi skriver:

```
UPDATE customers
SET email = 'olof.new@example.com'
WHERE id = 2;
```

byter vi inte bara några tecken i en tabell.

Vi förändrar vad databasen **påstår** om kunden Olof.

Det kan vara en rättelse av felaktig information, eller en korrekt beskrivning av att något faktiskt har förändrats.

Skillnaden kan spela roll om vi senare behöver förstå historiken. En korrigering och en verklig förändring är inte alltid samma sak, även om båda råkar använda `UPDATE`.

## Reflektion

1. Vad är skillnaden mellan att ändra en produkts aktuella pris och att ändra prisuppgiften för en redan genomförd beställning?
2. När är det rimligt att radera en rad, och när behöver vi i stället bevara historiken?
3. Varför bör flera förändringar som beskriver en och samma verksamhetshändelse ibland genomföras som en transaktion?

# 16. Att förändra databasens struktur

När vi först skapade bokhandelns databas bestämde vi vilka uppgifter en produkt, kund och författare skulle ha.

Men verksamheten står inte stilla. Vi kanske upptäcker att vi behöver spara ISBN för böcker, skilja mellan produkter som är aktiva och inaktiva eller registrera fler leveransuppgifter.

Då räcker det inte att lägga till eller ändra rader med `INSERT` och `UPDATE`. Vi behöver förändra **själva strukturen** för den information vi kan lagra.

En datamodell är alltså inte något vi konstruerar en gång för alla. Den behöver kunna utvecklas utan att vi förlorar information eller gör befintliga system obrukbara.

## Grund

### Att förändra data eller att förändra strukturen

I förra kapitlet ändrade vi uppgifter med exempelvis:

```
UPDATE products
SET price = 225.00
WHERE id = 1;
```

Det förändrar värdet i en befintlig kolumn.

Men om vi vill börja lagra ISBN för våra böcker behöver vi en **ny kolumn**. Då använder vi `ALTER TABLE`.

## Att lägga till en kolumn

Anta att vår tabell `products` innehåller:

| id | title | price | stock |
|---|---|---|---|
| 1 | Vägen hem | 200.00 | 12 |
| 2 | Den blå staden | 150.00 | 0 |
| 3 | Vinterljus | 249.00 | 4 |

Vi vill börja registrera ISBN:

```
ALTER TABLE products
ADD COLUMN isbn VARCHAR(20);
```

Kolumnen finns nu i tabellen, även för produkter vi redan registrerat.

| id | title | isbn |
|---|---|---|
| 1 | Vägen hem | NULL |
| 2 | Den blå staden | NULL |
| 3 | Vinterljus | NULL |

Vi har inte förlorat de gamla raderna. Men databasen känner ännu inte till deras ISBN.

Det illustrerar skillnaden mellan att **göra plats för en uppgift** och att **faktiskt känna till dess värde**.

## Att fylla i informationen

När vi har kontrollerat vilka ISBN som hör till vilka produkter kan vi lägga till dem med `UPDATE`:

```
UPDATE products
SET isbn = '9781234567897'
WHERE id = 1;
```

ISBN är här ett illustrativt värde. Vi använder text eftersom ett ISBN är en identifierare, inte en kvantitet som vi ska räkna med.

När en ny kolumn införs kan vi alltså behöva kombinera två olika slags förändringar:

1. Ändra **strukturen** med `ALTER TABLE`.
2. Ändra **innehållet** med `UPDATE`.

### Att lägga till ett standardvärde

Anta att vi vill markera om produkter ingår i det aktiva sortimentet.

Vi kan lägga till en kolumn med ett förvalt värde:

```
ALTER TABLE products
ADD COLUMN is_active BOOLEAN NOT NULL DEFAULT TRUE;
```

Tanken är att våra befintliga produkter ska betraktas som aktiva tills vi bestämmer något annat.

Vi behöver då fundera över om det är en rimlig regel för **alla redan registrerade produkter**, inte bara för nya produkter.

Olika databassystem kan hantera tillägg av kolumner, standardvärden och befintliga rader på olika sätt, särskilt när tabellerna är stora.[21]

### Att ändra en kolumns definition

Ibland behöver vi ändra en datatyp eller tillåta längre värden.

Om `title` från början definierats som `VARCHAR(100)` kanske det inte räcker för alla boktitlar.

I MySQL kan vi exempelvis skriva:

```
ALTER TABLE products
MODIFY COLUMN title VARCHAR(200) NOT NULL;
```

Detta förändrar kolumnens definition, inte dess namn eller vilket slags uppgift den beskriver.

Syntaxen varierar mellan olika SQL-dialekter.[22]

### Att ta bort en kolumn

Om vi inte längre behöver ett attribut kan vi ta bort dess kolumn:

```
ALTER TABLE products
DROP COLUMN isbn;
```

Men då försvinner också de lagrade ISBN-värdena ur tabellen.

Det är därför en strukturell förändring kan vara mer omfattande än den ser ut. Vi förändrar inte bara vilka frågor vi kan ställa i framtiden; vi kan också ta bort information som redan finns.

Detta är ett **alternativt exempel**, inte något vi bör göra direkt efter att ha infört ISBN.

### Att byta namn på en kolumn

Vi kan också vilja byta namn på en kolumn, exempelvis från `title` till `product_title`:

```
ALTER TABLE products
RENAME COLUMN title TO product_title;
```

Många moderna databassystem stöder denna syntax, men äldre versioner och vissa dialekter använder andra konstruktioner.[23]

En namnändring kan påverka alla SQL-frågor och program som använder det gamla namnet.

## Avancerat

### Vad är en databasmigration?

En **migration** är en kontrollerad förändring av databasens struktur och ibland även dess data.

Vi kan beskriva förändringen i en SQL-fil:

```
-- 001_add_isbn.sql
ALTER TABLE products
ADD COLUMN isbn VARCHAR(20);
```

Nästa ändring kan finnas i en annan fil:

```
-- 002_add_is_active.sql
ALTER TABLE products
ADD COLUMN is_active BOOLEAN NOT NULL DEFAULT TRUE;
```

Filerna dokumenterar hur databasen har utvecklats. Ett migrationsverktyg kan hålla reda på vilka ändringar som redan genomförts.

Migrationsskript bör normalt versionshanteras tillsammans med den kod och dokumentation som är beroende av databasens struktur.

## Utöka, överför, begränsa

Att göra en stor förändring direkt kan vara riskabelt om flera applikationer använder databasen.

En vanlig strategi är att ändra modellen i flera mindre steg:

**1. Utöka.** Lägg till en ny kolumn utan att omedelbart ta bort den gamla.

**2. Överför.** Fyll den nya kolumnen med kontrollerad information och uppdatera programmen så att de använder den.

**3. Begränsa.** När vi vet att den nya strukturen fungerar kan vi införa striktare regler och så småningom ta bort det gamla.

Detta brukar kallas *expand and contract*.

Strategin gör det möjligt att låta äldre och nyare programversioner fungera under en övergångsperiod, om förändringen planeras för det.

### Att lägga till en regel på befintliga data

Anta att vi vill kräva att varje bok har ett ISBN.

Det är inte tillräckligt att direkt lägga till `NOT NULL` om tabellen redan innehåller produkter där ISBN saknas.

Vi kan behöva:

- Först lägga till kolumnen utan kravet.
- Ta reda på vilka värden som saknas och fylla i dem.
- Kontrollera att informationen är tillräckligt korrekt.
- Först därefter göra kolumnen obligatorisk, om den verkligen ska gälla för alla produkter.

Och innan vi gör detta behöver vi kanske fråga oss om **alla produkter är böcker**. Om vår bokhandel senare säljer vykort eller presentartiklar kan ett obligatoriskt ISBN på tabellen `products` vara fel modellering.

En strukturell förändring kan alltså avslöja att vi behöver tänka om kring datamodellen.

### Att ändra en relation

Anta att vi från början bestämt att varje produkt bara kan ha en författare, genom en kolumn `author_id` i `products`.

När vi senare behöver stöd för flera författare blir lösningen inte bara att öka utrymmet i en kolumn.

Vi behöver förändra **relationen** i modellen, exempelvis med kopplingstabellen `product_authors` som vi redan introducerat.

En sådan migration kan innebära att vi först skapar den nya tabellen, för över befintliga relationer, ändrar applikationens frågor och först därefter tar bort den gamla kolumnen.

Här ser vi skillnaden mellan att förändra en kolumns tekniska form och att förändra **hur vi förstår informationens samband**.

## Går det att ångra en migration?

Vissa databasändringar kan ångras med en omvänd SQL-instruktion. Men det betyder inte att förlorad information automatiskt kommer tillbaka.

Om vi tar bort en kolumn som innehåller data kan det krävas en säkerhetskopia eller en separat överföring för att återställa värdena.

Det finns dessutom dialektskillnader i hur schemaändringar fungerar tillsammans med transaktioner. Exempelvis innebär många DDL-operationer i MySQL implicita commits, medan PostgreSQL ofta kan hantera sådana ändringar transaktionellt.[24]

En genomtänkt migrationsplan omfattar därför både hur förändringen ska utföras och hur vi hanterar ett misslyckande.

## Schemaförändringar och verksamhetens tillgänglighet

På en liten övningsdatabas kan vi oftast ändra en tabell direkt.

I ett system med många användare och stora tabeller kan vissa schemaändringar ta tid, kräva låsning eller påverka pågående frågor och uppdateringar.

Därför behöver vi ibland planera förändringar för att undvika driftstopp, eller använda databasens särskilda möjligheter till mindre störande schemaändringar.

Det är ytterligare ett skäl att inte se `ALTER TABLE` som enbart ett sätt att redigera en tabellbeskrivning.

## En View kan skydda ett gränssnitt

Föreställ dig att vi har en applikation som förväntar sig kolumnen `title`, men att vi vill byta namn på den i vår lagringsmodell.

En möjlig strategi är att erbjuda ett fortsatt stabilt gränssnitt med en **View**:

```
CREATE VIEW product_overview AS
SELECT
    id,
    product_title AS title,
    price
FROM products;
```

Här antar vi att den underliggande kolumnen redan heter `product_title`.

Program som frågar vyn `product_overview` kan då fortfarande få en kolumn med namnet `title`.

Detta är bara ett exempel på hur en View kan användas för att skilja den **interna lagringsmodellen** från det **gränssnitt andra system använder**. Det kan fortfarande krävas planering av beroenden och rättigheter.

Vi fördjupar oss i Views i nästa kapitel.

## Att förändra en modell är också att förändra ett avtal

När vi lägger till ett attribut utökar vi vad databasen kan säga om verkligheten.

När vi tar bort en kolumn upphör databasen kanske att kunna beskriva ett tidigare sakförhållande.

Och när vi förändrar relationerna mellan tabellerna kan vi förändra vilka kombinationer av information systemet betraktar som giltiga.

En databasstruktur är därför inte bara en teknisk ritning. Den fungerar också som ett **avtal om informationens betydelse**, mellan verksamheten, databasen och de program som använder den.

## Reflektion

1. Vad är skillnaden mellan att lägga till en ny kolumn och att faktiskt känna till värdena som ska stå i den?
2. Varför kan det vara säkrare att först utöka en databasmodell och senare ta bort äldre delar, i stället för att ändra allt på en gång?
3. När vi ändrar en tabell, förändrar vi då bara dess tekniska struktur – eller ibland också vårt sätt att förstå verksamheten?

# 17. Views och återanvändbara Queries

Vi har byggt bokhandelns databas med flera tabeller. Kunder, adresser, produkter, produkttyper och författare lagras var för sig, med relationer mellan dem.

Men den som arbetar i bokhandeln vill kanske inte tänka på hur informationen är uppdelad. Hen vill se vilka produkter som finns i lager, vilka författare de har eller vilka kunder som har adresser i en viss stad.

Vi har redan lärt oss att formulera sådana frågor med SQL. Men om vi behöver samma representation på många ställen kan det vara praktiskt att ge den ett eget namn.

En **View**, eller vy, låter oss beskriva **hur information ska betraktas**, utan att behöva skapa en ny vanlig tabell för den.

## Grund

### Från Query till View

Vi börjar med två tabeller vi redan känner:

**products**

| id | title | price | stock | product_type_id |
|---|---|---|---|---|
| 1 | Vägen hem | 200.00 | 12 | 1 |
| 2 | Den blå staden | 150.00 | 0 | 1 |
| 3 | Vinterljus | 249.00 | 4 | 1 |
| 4 | Små historier | 120.00 | 25 | 2 |

| id | title | price | stock | product_type_id |
|---|---|---|---|---|
| 5 | Mellan raderna | 180.00 | 9 | 2 |

### product_types

| id | name |
|---|---|
| 1 | Roman |
| 2 | Novellsamling |

Vi vet hur vi kan hämta produktens titel tillsammans med namnet på dess typ:

```
SELECT
    p.id,
    p.title,
    p.price,
    p.stock,
    t.name AS product_type
FROM products AS p
INNER JOIN product_types AS t
    ON p.product_type_id = t.id;
```

denna Query fungerar, men vi kanske vill återanvända den i flera rapporter och applikationer.

Då kan vi skapa en View:

```
CREATE VIEW product_overview AS
SELECT
    p.id,
    p.title,
    p.price,
    p.stock,
    t.name AS product_type
FROM products AS p
INNER JOIN product_types AS t
    ON p.product_type_id = t.id;
```

Nu har vi gett denna Query namnet `product_overview`.

## Att använda en View

Vi kan läsa från vyn nästan som från en tabell:

```
SELECT *
FROM product_overview;
```

| id | title | price | stock | product_type |
|---|---|---|---|---|
| 1 | Vägen hem | 200.00 | 12 | Roman |
| 2 | Den blå staden | 150.00 | 0 | Roman |
| 3 | Vinterljus | 249.00 | 4 | Roman |
| 4 | Små historier | 120.00 | 25 | Novellsamling |
| 5 | Mellan raderna | 180.00 | 9 | Novellsamling |

Vi kan också filtrera och sortera resultatet:

```
SELECT title, stock
FROM product_overview
WHERE product_type = 'Roman'
  AND stock > 0
ORDER BY title;
```

Vyn är alltså inte ett färdigt utskrivet resultat, utan en **namngiven Query** som vi kan fortsätta arbeta med.

## Lagrar en View data?

I en vanlig View lagrar databasen i huvudsak **definitionen av frågan**, inte en separat kopia av resultatraderna.

Om priset på `Vägen hem` ändras i `products` kommer nästa fråga mot `product_overview` normalt att visa det nya priset.

Vi har med andra ord inte skapat ännu en plats där priset behöver uppdateras.

Det här skiljer en vanlig View från en kopierad rapporttabell.

## En View behöver inte kombinera tabeller

En View kan också vara ett urval från en enda tabell.

Vi kan exempelvis definiera en vy för produkter som finns i lager:

```
CREATE VIEW available_products AS
SELECT id, title, price, stock
FROM products
WHERE stock > 0;
```

Och sedan använda den:

```
SELECT title, price
FROM available_products
ORDER BY price;
```

Om lagersaldot förändras påverkas även vilka produkter som visas genom vyn.

## Views för sammanfattningar

I kapitel 8 arbetade vi med `GROUP BY`. Även sådana resultat kan vi ge ett namn:

```
CREATE VIEW stock_by_product_type AS
SELECT
    t.id AS product_type_id,
    t.name AS product_type,
    COUNT(*) AS product_count,
    SUM(p.stock) AS total_stock
FROM product_types AS t
INNER JOIN products AS p
    ON p.product_type_id = t.id
GROUP BY t.id, t.name;
```

Vi har nu en vy där varje rad beskriver **en produkttyp**, inte en enskild produkt.

```
SELECT *
FROM stock_by_product_type;
```

| product_type_id | product_type | product_count | total_stock |
|---|---|---|---|
| 1 | Roman | 3 | 16 |
| 2 | Novellsamling | 2 | 34 |

Det är ett annat sätt att betrakta samma information.

### Att ta bort en View

Vi kan ta bort definitionen med:

```
DROP VIEW available_products;
```

Det tar bort vyn, **inte** produkterna i `products`.

Vi bör ändå vara försiktiga: andra frågor eller program kan vara beroende av vyn.

## Avancerat

### Lagringsmodell och logisk modell

Vår databas är organiserad så att varje företeelse har en tydlig plats. En produkt har sitt pris i `products` och en produkttyp har sitt namn i `product_types`.

Det är en modell som passar för **lagring och integritet**.

Men en anställd kanske vill se produktnamn, typnamn, pris och lagersaldo tillsammans. En rapport kanske vill ha en rad per produkttyp.

Views låter oss därför definiera **logiska modeller för olika informationsbehov** ovanpå samma lagringsmodell.

Vi kan tänka på tre nivåer:

| Nivå | Fråga |
|---|---|
| Lagringsmodell | Hur organiserar och skyddar vi informationen? |
| View | Vilken representation behöver vi för ett visst syfte? |
| Användare eller applikation | Vilka frågor behöver vi kunna besvara? |

Det betyder inte att en View alltid är en komplett modell i sig. Men den kan fungera som ett namngivet, återanvändbart perspektiv på vår information.

## När varje rad får en ny betydelse

I `products` representerar en rad en produkt.

I `product_overview` representerar varje rad fortfarande en produkt, men med uppgifter från flera tabeller.

I `stock_by_product_type` representerar varje rad i stället en **produkttyp med sammanfattad lagerinformation**.

Det är värt att fundera över vad vi faktiskt har definierat. Vi har inte bara sparat ett stycke SQL. Vi har gett ett namn åt **vilken sorts information resultatet beskriver**.

Det anknyter till vår tidigare diskussion om datatyper och datamodellering: vi kan skapa meningsfulla strukturer utan att varje struktur behöver motsvara en egen, lagrad entitet.

## Views som stabila gränssnitt

I förra kapitlet diskuterade vi hur databasstrukturer förändras över tid.

Anta att vi byter namn på en intern kolumn från `title` till `product_title`. Applikationer som använder den gamla kolumnen kan då sluta fungera.

Om de i stället frågar en View kan vi i vissa fall bevara ett stabilt gränssnitt:

```
CREATE VIEW product_catalog AS
SELECT
    id,
    product_title AS title,
    price
FROM products;
```

Här förutsätter vi att den underliggande kolumnen redan har bytt namn.

Vyn kan erbjuda det kolumnnamn applikationen förväntar sig, även när lagringsmodellen ser annorlunda ut.

Detta kräver planering: vi måste hantera beroenden, behörigheter och hur vyn uppdateras under migrationen. Vyn förändrar inte automatiskt sig själv när tabellen ändras.

## Views som avgränsning av information

En View kan också visa bara de kolumner som behövs i ett visst sammanhang.

Anta att bokhandelns interna kundtabell innehåller namn, e-postadress och födelsedatum, men att en viss rapport bara behöver namn och kund-id.

```
CREATE VIEW customer_names AS
SELECT id, name
FROM customers;
```

Vi har därmed definierat en mer begränsad representation.

Det kan vara en del av en säkerhets- och behörighetsmodell om användaren får åtkomst till vyn men inte till den underliggande tabellen. **Vyn ensam är däremot ingen garanti för att känslig information skyddas.** Databassystemets rättigheter och vyernas exekveringsregler måste också vara korrekt konfigurerade.[25]

## Kan vi använda UPDATE på en View?

Vissa Views går att uppdatera, så att en ändring genom vyn påverkar den underliggande tabellen.

En enkel vy som väljer kolumner från en tabell kan i många databassystem vara uppdateringsbar.

Men en vy med `GROUP BY` eller aggregeringar, som `stock_by_product_type`, är normalt inte direkt uppdateringsbar. Vad skulle det exempelvis innebära att ändra gruppens `total_stock` till `100`? Vilka produktposter skulle vi förändra?

Även en `JOIN`-vy kan ha begränsningar. De exakta reglerna varierar mellan SQL-dialekter.[26]

Vi bör därför se en View som en **läsmodell i första hand**, om vi inte uttryckligen har bestämt och testat hur uppdateringar ska fungera.

## Views och prestanda

En vanlig View lagrar normalt inte resultatet i förväg. När vi använder den behöver databashanteraren fortfarande behandla frågan som vyn beskriver.

Databasen kan i många fall optimera genom att kombinera vydefinitionen med vår yttre fråga. Men en View gör inte automatiskt en komplicerad fråga snabbare.

En del databassystem stöder också **materialiserade vyer**, där ett frågeresultat lagras och uppdateras eller förnyas enligt särskilda regler.

Det kan förbättra prestanda vid upprepade analyser, men ger oss en ny fråga: **Hur aktuellt är det lagrade resultatet?**

Stödet skiljer sig mellan SQL-system.[27]

## Views, subqueries och CTE:er

Vi har nu flera sätt att skapa nya representationer av information:

| Teknik | Hur vi använder den |
|---|---|
| Subquery | Ett frågeresultat används inuti en annan fråga |
| View | Vi ger en fråga ett bestående namn i databasen |

| Teknik | Hur vi använder den |
|---|---|
| CTE | Vi namnger ett mellanresultat inom en enskild SQL-sats |

En View är alltså inte bara en subquery med ett namn. Den blir ett **databasobjekt** som andra frågor kan referera till, med egna beroenden och eventuellt egna åtkomsträttigheter.

I nästa kapitel går vi vidare med CTE:er och undersöker hur vi kan göra stora frågor mer begripliga genom att namnge olika steg.

### En modell är också ett perspektiv

Vi kan beskriva samma bokhandel på många sätt.

För lagring behöver vi veta vilka kunder, produkter och adresser som finns och hur de hör ihop.

För personalen i butiken behöver vi kanske visa vilka produkter som finns tillgängliga.

För ledningen behöver vi kanske se sammanfattningar av lager och försäljning.

Dessa perspektiv behöver inte konkurrera med varandra. De kan bygga på en gemensam modell där informationens betydelse och relationer är tydliga.

**En View är därför både ett tekniskt verktyg och ett uttryck för vilket informationsperspektiv vi vill erbjuda.**

## Reflektion

1. Vad är skillnaden mellan en tabell som lagrar produkter och en View som presenterar en produktöversikt?
2. Varför kan en View vara användbar när databasens interna struktur behöver förändras?

3. När vi skapar en View som sammanfattar lagersaldo per produkttyp, vad har vi då egentligen modellerat?

# 18. Att dela upp Queries i logiska steg

När vi formulerar en Query kan vi ofta börja med en enkel fråga: vilka produkter finns i lager, eller vilka produkter kostar mer än genomsnittet?

Men ibland behöver vi flera steg för att få fram ett meningsfullt resultat. Vi kanske först vill beräkna lagervärdet för varje produkt, sedan sammanfatta det per produkttyp och slutligen välja ut de produkttyper som representerar störst värde.

Det går att skriva allt i en enda stor SQL-konstruktion. Men det är inte alltid det bästa sättet att **förstå vad vi gör**.

Med **Common Table Expressions (CTE:er)** kan vi ge olika delar av en Query namn och låta dem bygga vidare på varandra.

## Grund

### Common Table Expressions – WITH

Vi börjar med bokhandelns produkter:

**products**

| id | title | price | stock | product_type_id |
|---|---|---|---|---|
| 1 | Vägen hem | 200.00 | 12 | 1 |
| 2 | Den blå staden | 150.00 | 0 | 1 |
| 3 | Vinterljus | 249.00 | 4 | 1 |
| 4 | Små historier | 120.00 | 25 | 2 |
| 5 | Mellan raderna | 180.00 | 9 | 2 |

Vi har tidigare beräknat lagervärdet för en produkt genom att multiplicera `price` med `stock`:

```
SELECT
    title,
    price * stock AS inventory_value
FROM products;
```

Om vi vill använda samma beräkning som ett eget mellanresultat kan vi skriva:

```
WITH product_values AS (
    SELECT
        title,
        price * stock AS inventory_value
    FROM products
)
SELECT *
FROM product_values;
```

| title | inventory_value |
|---|---|
| Vägen hem | 2400.00 |
| Den blå staden | 0.00 |
| Vinterljus | 996.00 |
| Små historier | 3000.00 |
| Mellan raderna | 1620.00 |

`WITH` inleder definitionen. `product_values` är namnet vi valt för mellanresultatet och parenteserna innehåller den Query som skapar det.

Efter definitionen följer vår huvudsakliga Query. Den använder `product_values` som om det vore en tabell.

En CTE skapar normalt inte någon bestående tabell eller View. Namnet finns tillgängligt **inom den SQL-sats där det definieras**.

## Filtrera ett mellanresultat

Vi kan fortsätta arbeta med det beräknade värdet:

```
WITH product_values AS (
    SELECT
        title,
        price * stock AS inventory_value
    FROM products
)
SELECT title, inventory_value
FROM product_values
WHERE inventory_value > 2000
ORDER BY inventory_value DESC;
```

| title | inventory_value |
|---|---|
| Små historier | 3000.00 |
| Vägen hem | 2400.00 |

Nu blir det tydligt att vi gör två olika saker:

1. Beräknar lagervärdet för varje produkt.
2. Väljer vilka beräknade värden vi är intresserade av.

Det är en liten skillnad i kod, men en viktig skillnad för läsbarheten.

## Flera namngivna steg

Nu vill vi sammanfatta lagervärdet per produkttyp.

Vi kan dela upp arbetet i två CTE:er:

```
WITH product_values AS (
    SELECT
        product_type_id,
        price * stock AS inventory_value
    FROM products
),
type_values AS (
    SELECT
        product_type_id,
        SUM(inventory_value) AS total_value
    FROM product_values
    GROUP BY product_type_id
)
SELECT product_type_id, total_value
FROM type_values
ORDER BY total_value DESC;
```

| product_type_id | total_value |
|---|---|
| 2 | 4620.00 |
| 1 | 3396.00 |

Den första CTE:n beräknar värden per produkt. Den andra sammanfattar dessa värden per produkttyp.

En CTE kan alltså använda en tidigare CTE inom samma `WITH`-del.

Här har vi byggt upp ett större informationsbehov som **en kedja av begripliga steg**.

## Kombinera CTE med JOIN

Nu vill vi också visa produkttypernas namn.

### product_types

| id | name |
|---|---|
| 1 | Roman |
| 2 | Novellsamling |

```
WITH product_values AS (
    SELECT
        product_type_id,
        price * stock AS inventory_value
    FROM products
),
type_values AS (
    SELECT
        product_type_id,
        SUM(inventory_value) AS total_value
    FROM product_values
    GROUP BY product_type_id
)
SELECT
    t.name AS product_type,
    v.total_value
FROM type_values AS v
INNER JOIN product_types AS t
    ON t.id = v.product_type_id
ORDER BY v.total_value DESC;
```

| product_type | total_value |
|---|---|
| Novellsamling | 4620.00 |
| Roman | 3396.00 |

Vi känner redan till alla delarna: uttryck, aggregering och `JOIN`. CTE:erna hjälper oss bara att **organisera dem**.

### CTE eller subquery?

I kapitel 10 lärde vi oss att en subquery kan användas som en del av en annan Query.

Vi kan exempelvis skriva:

```
SELECT product_type_id, SUM(inventory_value) AS total_value
FROM (
    SELECT
        product_type_id,
        price * stock AS inventory_value
    FROM products
) AS product_values
GROUP BY product_type_id;
```

Det fungerar också.

En CTE ger oss möjlighet att flytta ut det namngivna mellanresultatet till början av satsen, och kan göra det enklare att följa en Query med många steg.

Vi väljer den form som gör informationsbehovet tydligast – inte utifrån en regel om att den ena alltid skulle vara bättre.

## Avancerat

### En CTE är inte nödvändigtvis en tillfällig tabell

Det kan vara hjälpsamt att föreställa sig att databasen först skapar `product_values` och sedan läser från det.

Men det är en **logisk beskrivning**, inte nödvändigtvis hur frågan körs.

En databashanterare kan välja att slå samman CTE:n med resten av frågan, beräkna den som ett separat mellanresultat eller använda andra optimeringar.

CTE:er gör alltså inte automatiskt en Query snabbare eller långsammare. Beteendet varierar mellan databassystem, versioner och enskilda Queries.[28]

## Återanvända samma mellanresultat

En namngiven CTE kan refereras från flera delar av samma SQL-sats.

Vi kanske vill jämföra varje produkt med genomsnittligt lagervärde:

```
WITH product_values AS (
    SELECT
        title,
        price * stock AS inventory_value
    FROM products
),
average_value AS (
    SELECT AVG(inventory_value) AS average_inventory_value
    FROM product_values
)
SELECT
    p.title,
    p.inventory_value,
    a.average_inventory_value
FROM product_values AS p
CROSS JOIN average_value AS a
WHERE p.inventory_value > a.average_inventory_value;
```

`average_value` ger en rad med ett värde, så en `CROSS JOIN` gör värdet tillgängligt på varje produktrad.

Vi ska inte anta att detta alltid innebär att databasen fysiskt räknar `product_values` exakt en gång. Det avgörs av frågeoptimeraren.

## Rekursiva CTE:er

Hittills har varje CTE byggt på redan definierade tabeller eller mellanresultat.

Men vissa problem är **rekursiva**. Tänk exempelvis på en hierarki av produktkategorier:

```
Böcker
├── Skönlitteratur
│   ├── Romaner
│   └── Noveller
└── Facklitteratur
    └── Databaser
```

En kategori kan ha en överordnad kategori. Hierarkin kan ha flera nivåer, och vi kanske inte vet i förväg hur många.

Vi kan modellera den med en tabell:

### categories

| id | name | parent_id |
|---|---|---|
| 1 | Böcker | NULL |
| 2 | Skönlitteratur | 1 |
| 3 | Facklitteratur | 1 |
| 4 | Romaner | 2 |
| 5 | Noveller | 2 |
| 6 | Databaser | 3 |

Detta är ett separat exempel på en möjlig framtida kategorimodell; den ersätter inte vår tidigare enkla tabell `product_types`.

Vi kan sedan använda en **rekursiv CTE**:

```
WITH RECURSIVE category_tree AS (
    SELECT
        id,
        name,
        parent_id,
```

```
        0 AS depth
    FROM categories
    WHERE parent_id IS NULL

    UNION ALL

    SELECT
        c.id,
        c.name,
        c.parent_id,
        ct.depth + 1
    FROM categories AS c
    INNER JOIN category_tree AS ct
        ON c.parent_id = ct.id
)
SELECT name, depth
FROM category_tree
ORDER BY depth, id;
```

Resultat:

| name | depth |
|---|---|
| Böcker | 0 |
| Skönlitteratur | 1 |
| Facklitteratur | 1 |
| Romaner | 2 |
| Noveller | 2 |
| Databaser | 2 |

## Hur fungerar rekursionen?

En rekursiv CTE har två delar:

**Startdel:** Den första `SELECT`-satsen väljer var vi ska börja, här de kategorier som inte har någon överordnad kategori.

**Rekursiv del:** Den andra `SELECT`-satsen hittar fler rader genom att använda resultatet från tidigare steg.

`UNION ALL` kombinerar delarnas resultat.

Vi kan tänka oss att databasen arbetar nivå för nivå tills det inte finns fler underkategorier att hitta. Detta är en förenklad förklaring av den logiska processen.

Det är viktigt att rekursionen faktiskt kan avslutas. Om vår hierarki innehåller cykler, exempelvis en kategori som indirekt pekar tillbaka på sig själv, behöver vi skydda oss mot obegränsad upprepning. Databassystem har olika säkerhetsgränser och möjligheter att hantera sådana situationer.[29]

## Rekursion är inte bara för hierarkier

Samma princip kan användas för att följa relationer i flera led, exempelvis:

- Kategorier och underkategorier.
- Organisationsstrukturer.
- Beroenden mellan komponenter.
- Förbindelser i ett nätverk.

Men en rekursiv CTE är inte alltid bästa lösningen för varje grafproblem. Vi behöver fortfarande avgöra vad relationerna betyder, om cykler tillåts och vilka vägar som är relevanta.

## CTE, View och lagrad tabell

Vi har nu tre olika sätt att arbeta med ett resultat som liknar en tabell:

| Konstruktion | Var resultatet eller definitionen finns |
|---|---|
| **CTE** | Namngivet inom en enda SQL-sats |
| **View** | En sparad definition som kan användas av flera Queries |
| **Tabell** | En lagringsstruktur med egna rader och regler |

En CTE är praktisk när vi vill göra en enskild Query mer begriplig.

En View passar när flera användare eller applikationer behöver återanvända samma logiska representation.

En tabell passar när informationen faktiskt behöver lagras som egna poster, med egen identitet och integritetsregler.

Valet handlar alltså inte bara om hur vi skriver SQL, utan också om **hur länge och för vilket syfte representationen ska finnas**.

### Att namnge ett steg är att formulera en tanke

I vår Query om lagervärden valde vi namnen `product_values` och `type_values`.

Dessa namn berättar vad informationen betyder på olika nivåer: först produktens lagervärde, sedan produkttypens sammanlagda lagervärde.

Det är möjligt att skriva SQL som ger rätt resultat men är svårt att förstå. Men det är också möjligt att skriva SQL så att strukturen förklarar resonemanget bakom resultatet.

**En välskriven Query är inte bara korrekt. Den hjälper oss att förstå vilka informationssteg som leder fram till svaret.**

## Reflektion

1. När blir en Query lättare att förstå om vi delar upp den i flera namngivna steg?
2. Vad är skillnaden mellan en CTE och en View, även om båda kan se ut som en tabell när vi använder dem?
3. När kan en rekursiv CTE hjälpa oss att beskriva ett samband som sträcker sig över flera nivåer?

# 19. Fönsterfunktioner

I kapitel 8 lärde vi oss att sammanfatta data. Med `GROUP BY` kan vi exempelvis ta reda på hur många böcker vi har i varje produkttyp eller vad deras sammanlagda lagervärde är.

Men ibland vill vi både se **varje enskild produkt** och samtidigt jämföra den med andra produkter.

Vi kanske vill veta hur en boks pris förhåller sig till genomsnittspriset i samma kategori, eller vilken plats den har i en prisordning. Då behöver vi inte nödvändigtvis slå ihop produkterna till en rad per grupp.

**Fönsterfunktioner** (*window functions*) låter oss göra beräkningar över en samling relaterade rader och ändå behålla de individuella raderna i resultatet.

## Grund

### Från grupper till fönster

Vi börjar med bokhandelns produkter:

**products**

| id | title | price | stock | product_type_id |
|---|---|---|---|---|
| 1 | Vägen hem | 200.00 | 12 | 1 |
| 2 | Den blå staden | 150.00 | 0 | 1 |
| 3 | Vinterljus | 249.00 | 4 | 1 |
| 4 | Små historier | 120.00 | 25 | 2 |
| 5 | Mellan raderna | 180.00 | 9 | 2 |

Med `GROUP BY` kan vi beräkna genomsnittspriset per produkttyp:

```
SELECT
    product_type_id,
    AVG(price) AS average_price
FROM products
GROUP BY product_type_id;
```

| product_type_id | average_price |
|---|---|
| 1 | 199.666667 |
| 2 | 150.000000 |

Här har vi fått **en rad per produkttyp**. Produktens titel och enskilda pris finns inte kvar som egna rader i resultatet.

Men tänk om vi vill visa varje produkt tillsammans med genomsnittet för just dess produkttyp?

## OVER – beräkna utan att slå ihop rader

Vi kan använda `AVG` som en fönsterfunktion:

```
SELECT
    title,
    price,
    product_type_id,
    AVG(price) OVER (
        PARTITION BY product_type_id
    ) AS average_price
FROM products
ORDER BY product_type_id, id;
```

| title | price | product_type_id | average_price |
|---|---|---|---|
| Vägen hem | 200.00 | 1 | 199.666667 |
| Den blå staden | 150.00 | 1 | 199.666667 |
| Vinterljus | 249.00 | 1 | 199.666667 |
| Små historier | 120.00 | 2 | 150.000000 |
| Mellan raderna | 180.00 | 2 | 150.000000 |

Antalet rader är detsamma som i `products`, men varje rad har fått ett extra beräknat värde.

`OVER` talar om att funktionen ska beräknas över ett **fönster av rader**, i stället för att resultatet ska sammanfattas till en rad per grupp.

`PARTITION BY product_type_id` delar in raderna i olika fönster: ett för varje produkttyp.

Vi kan tänka oss att varje produkt får med sig ett mått som beskriver den grupp den tillhör.

## Att jämföra produkten med sin grupp

När genomsnittet finns tillgängligt på varje rad kan vi enkelt jämföra:

```
SELECT
    title,
    price,
    price - AVG(price) OVER (
        PARTITION BY product_type_id
    ) AS difference_from_average
FROM products
ORDER BY product_type_id, id;
```

En produkt med ett positivt värde ligger över genomsnittspriset för sin produkttyp. Ett negativt värde betyder att produkten ligger under genomsnittet.

Nu har vi inte bara beräknat ett mått. Vi har använt det som en **jämförelsepunkt för varje enskild produkt**.

## ROW_NUMBER – numrera rader

Fönsterfunktioner kan också ordna rader i en följd.

```
SELECT
    title,
    price,
    ROW_NUMBER() OVER (
        ORDER BY price DESC, id
    ) AS price_position
FROM products
ORDER BY price_position;
```

| title | price | price_position |
|---|---|---|
| Vinterljus | 249.00 | 1 |
| Vägen hem | 200.00 | 2 |
| Mellan raderna | 180.00 | 3 |
| Den blå staden | 150.00 | 4 |
| Små historier | 120.00 | 5 |

`ROW_NUMBER()` ger varje rad ett eget nummer.

Observera att `ORDER BY` **inne i** `OVER` anger i vilken ordning numren tilldelas. Den avslutande `ORDER BY` anger däremot hur resultatet ska presenteras.

Vi har lagt till `id` som andra sorteringskolumn för att få en entydig ordning om två produkter kostar lika mycket.

## Numrera inom varje grupp

Vi kan kombinera `PARTITION BY` med `ORDER BY`:

```
SELECT
    title,
    product_type_id,
    price,
    ROW_NUMBER() OVER (
        PARTITION BY product_type_id
        ORDER BY price DESC, id
    ) AS position_in_type
FROM products
ORDER BY product_type_id, position_in_type;
```

Nu börjar numreringen om för varje produkttyp.

Det är samma princip som när vi delade upp beräkningen av genomsnittspriser, men här använder vi fönstret till att skapa en **ordning inom varje grupp**.

### RANK och DENSE_RANK

`ROW_NUMBER()` ger unika radnummer även när flera produkter har samma pris.

I vissa sammanhang vill vi i stället att produkter med samma pris ska dela placering.

Då kan vi använda `RANK()`:

```
SELECT
    title,
    price,
    RANK() OVER (ORDER BY price DESC) AS price_rank,
    DENSE_RANK() OVER (ORDER BY price DESC) AS dense_price_rank
FROM products;
```

Skillnaden blir synlig om flera produkter har samma pris:

| Pris | RANK | DENSE_RANK |
|---|---|---|
| 249 | 1 | 1 |
| 200 | 2 | 2 |
| 200 | 2 | 2 |
| 180 | 4 | 3 |

`RANK()` lämnar ett glapp efter en delad placering. `DENSE_RANK()` gör inte det.

Tabellen ovan är ett **illustrativt exempel med två lika priser**, inte det exakta resultatet från vår nuvarande produktlista.

## Avancerat

### Fönstret kan också bestå av en del av en grupp

Hittills har `PARTITION BY` avgränsat en grupp, och `ORDER BY` har skapat en ordning inom den.

Men vissa fönsterberäkningar kan också använda en **fönsterram** (*window frame*): vilka av de ordnade raderna som ingår i just den aktuella radens beräkning.

Vi kan exempelvis beräkna en löpande summa av lagerantalet:

```
SELECT
    id,
    title,
    stock,
    SUM(stock) OVER (
        ORDER BY id
        ROWS BETWEEN UNBOUNDED PRECEDING
             AND CURRENT ROW
    ) AS running_stock
FROM products
ORDER BY id;
```

| id | title | stock | running_stock |
|---|---|---|---|
| 1 | Vägen hem | 12 | 12 |
| 2 | Den blå staden | 0 | 12 |
| 3 | Vinterljus | 4 | 16 |
| 4 | Små historier | 25 | 41 |
| 5 | Mellan raderna | 9 | 50 |

`ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` betyder att beräkningen omfattar alla tidigare rader **i fönstrets ordning**, inklusive den aktuella raden.

Vi skriver ramen uttryckligen för att tydliggöra vad som räknas med.

## ROWS och RANGE

Det kan verka självklart att `ORDER BY` bestämmer vilka rader som räknas som tidigare och senare, men SQL skiljer mellan olika sätt att avgränsa ett fönster.

Med `ROWS` räknar vi **fysiska positioner i den ordnade följden av resultatrader**.

Med RANGE kan rader med samma sorteringsvärde behandlas som en grupp av likvärdiga rader.

Därför kan standardramen för en aggregerande fönsterfunktion med ORDER BY ge oväntade resultat när flera rader har samma sorteringsvärde.

När vår avsikt är en löpande beräkning rad för rad använder vi gärna en uttrycklig ROWS-ram och en entydig sorteringsordning.[30]

## LAG och LEAD – jämföra med andra rader

Ibland vill vi jämföra ett värde med raden före eller efter i en ordnad följd.

```
SELECT
    id,
    title,
    price,
    LAG(price) OVER (
        ORDER BY price, id
    ) AS previous_price
FROM products
ORDER BY price, id;
```

LAG(price) hämtar priset från föregående rad i den angivna ordningen. För den första raden finns inget föregående pris, så resultatet blir normalt NULL.

Motsvarigheten LEAD(price) hämtar värdet från nästa rad.

Sådana funktioner blir särskilt användbara när raderna beskriver exempelvis försäljning över tid, mätvärden eller andra händelser i följd.

## Filtrera på ett fönsterresultat

Anta att vi bara vill visa den dyraste produkten i varje produkttyp.

Det kan vara frestande att skriva:

```
-- Fungerar inte som avsett
SELECT
    title,
    product_type_id,
    ROW_NUMBER() OVER (
        PARTITION BY product_type_id
        ORDER BY price DESC, id
    ) AS position_in_type
FROM products
WHERE position_in_type = 1;
```

Problemet är att `WHERE` filtrerar rader **innan** fönsterfunktionerna beräknas. Aliaset `position_in_type` finns inte tillgängligt där.

Här får vi användning av CTE:er från kapitel 18:

```
WITH ranked_products AS (
    SELECT
        title,
        product_type_id,
        price,
        ROW_NUMBER() OVER (
            PARTITION BY product_type_id
            ORDER BY price DESC, id
        ) AS position_in_type
    FROM products
)
SELECT
    title,
    product_type_id,
    price
FROM ranked_products
WHERE position_in_type = 1
ORDER BY product_type_id;
```

| title | product_type_id | price |
|---|---|---|
| Vinterljus | 1 | 249.00 |
| Mellan raderna | 2 | 180.00 |

Vi beräknar först placeringen. Därefter kan den yttre Queryn filtrera på resultatet.

Det visar varför det är användbart att kunna kombinera **CTE:er och fönsterfunktioner**.

## Fönsterfunktioner och NULL

Precis som vanliga aggregeringar påverkas fönsterfunktioner av `NULL`.

`AVG(price)` räknar normalt bara med rader där `price` inte är `NULL`, medan `COUNT(*)` räknar rader oavsett om någon kolumn saknar värde.

`LAG` och `LEAD` kan däremot ge `NULL` därför att det saknas en föregående eller efterföljande rad.

Vi måste alltså fortfarande skilja mellan **ett saknat värde i informationen** och **avsaknaden av en jämförelserad**.

## Är fönsterfunktioner alltid effektivare?

Fönsterfunktioner kan ofta ersätta flera separata Queries, självkopplingar eller korrelerade subqueries.

Men de kräver ibland sortering och hantering av stora mängder rader. Resultatet beror på hur frågan ser ut, hur datan är fördelad, vilka index som finns och hur databashanteraren optimerar den.

Vi väljer därför inte en fönsterfunktion enbart för att den verkar avancerad, utan för att den uttrycker den beräkning vi faktiskt behöver.

Fönsterfunktioner stöds i moderna versioner av bland annat MySQL, PostgreSQL, SQLite och SQL Server, men detaljer och utökade funktioner varierar.[31]

## Vilken rad beskriver vi?

Med `GROUP BY` förvandlar vi en samling produktposter till en sammanfattning: exempelvis **en rad per produkttyp**.

Med fönsterfunktioner behåller vi i stället **en rad per produkt**, men gör det möjligt att beskriva produkten i relation till en större samling.

Det är en grundläggande skillnad i **vilken information varje rad representerar**.

En produkt kan ha ett eget pris, tillhöra en produkttyp med ett genomsnittspris och ha en viss placering bland andra produkter. Alla tre är meningsfulla uppgifter – men de beskriver olika perspektiv på samma information.

## Reflektion

1. När är `GROUP BY` lämpligare än en fönsterfunktion, och när behöver vi bevara en rad för varje enskild produkt?
2. Varför kan två Queries med `RANK()` och `ROW_NUMBER()` ge olika sätt att beskriva samma placering?
3. Vad händer med vår tolkning av en beräkning när vi ändrar vilka rader som ingår i fönstret?

# 20. Transaktioner och samtidighet

När en kund köper en bok händer flera saker. Vi behöver registrera beställningen, komma ihåg vad kunden köpte och minska lagersaldot.

Men vad händer om vi lyckas registrera beställningen och sedan misslyckas med att minska lagret? Eller om två kunder samtidigt försöker köpa det sista exemplaret?

Var och en av våra SQL-satser kan vara korrekt, men **helheten kan ändå bli fel**.

En **transaktion** hjälper oss att hantera flera förändringar som en sammanhängande händelse. Samtidigt behöver databasen kunna hantera att många användare arbetar med informationen parallellt.

## Grund

### En beställning är mer än en rad

Vi har sedan tidigare tabellen `products`:

| id | title | price | stock |
|---|---|---|---|
| 1 | Vägen hem | 200.00 | 12 |
| 2 | Den blå staden | 150.00 | 0 |

Vi utökar nu bokhandelns modell med beställningar:

**orders**

| id | customer_id | ordered_at |
|---|---|---|
| 101 | 2 | 2026-10-12 10:30:00 |

**order_items**

| id | order_id | product_id | quantity | unit_price |
|---|---|---|---|---|
| 1001 | 101 | 1 | 2 | 200.00 |

En rad i `orders` beskriver beställningen. En rad i `order_items` beskriver en produkt i den beställningen. Värdet `unit_price` bevarar priset vid köptillfället, även om produktens aktuella pris förändras.

Tabellerna är ett förenklat exempel. Vi förutsätter här att de har skapats med lämpliga nycklar och främmande nycklar.

## Flera SQL-satser, en händelse

Anta att Olof beställer två exemplar av `Vägen hem`.

Det innebär åtminstone tre förändringar:

1. En ny beställning registreras.
2. Två exemplar registreras på beställningen.
3. Produktens lagersaldo minskas med två.

Om vi utför SQL-satserna var för sig kan några av förändringarna lyckas även om en senare förändring misslyckas.

Med en transaktion kan vi i stället bestämma att de ska **antingen genomföras tillsammans eller inte alls**.

## START TRANSACTION och COMMIT

I MySQL kan vi påbörja en transaktion med `START TRANSACTION`:

```
START TRANSACTION;

INSERT INTO orders (id, customer_id, ordered_at)
VALUES (102, 2, '2026-10-12 11:00:00');

INSERT INTO order_items (
    id, order_id, product_id, quantity, unit_price
)
VALUES (1002, 102, 1, 2, 200.00);
```

```
UPDATE products
SET stock = stock - 2
WHERE id = 1 AND stock >= 2;

COMMIT;
```

`COMMIT` avslutar transaktionen och gör dess förändringar bestående.

Det här exemplet visar syntaxen och tanken, men är **inte ett färdigt säkert orderflöde**. Innan vi gör `COMMIT` måste vi bland annat kontrollera att lageruppdateringen verkligen påverkade exakt en rad. Om den inte gjorde det ska vi avbryta transaktionen, inte spara en order som inte kan levereras.

Vi förutsätter också att identifierarna är lediga och att beställningens data har kontrollerats.

## ROLLBACK – att ångra en transaktion

Anta att vi upptäcker ett fel innan transaktionen har avslutats.

Då kan vi skriva:

```
START TRANSACTION;

UPDATE products
SET stock = stock - 2
WHERE id = 1 AND stock >= 2;

-- Vi upptäcker att beställningen inte ska genomföras.

ROLLBACK;
```

`ROLLBACK` återställer de förändringar som utförts i den pågående transaktionen, under förutsättning att operationerna omfattas av transaktionsstödet.

Den tar inte automatiskt tillbaka en redan committad transaktion. Efter `COMMIT` behöver vi i regel en **ny förändring** för att korrigera det som hänt.

Det är därför viktigt att skilja mellan att *ångra en ännu inte avslutad händelse* och att *rätta en redan registrerad händelse*.

## Vad är ACID?

Fyra vanliga egenskaper hos databastransaktioner sammanfattas med **ACID**:

| Egenskap | Betydelse |
|---|---|
| **Atomicity – atomäritet** | En transaktion behandlas som en helhet: alla dess förändringar genomförs eller ingen av dem. |
| **Consistency – konsistens** | En transaktion ska föra databasen från ett giltigt tillstånd till ett annat, enligt de regler som faktiskt finns definierade och upprätthålls. |
| **Isolation – isolering** | Samtidiga transaktioner ska inte kunna störa varandra på sätt som den valda isoleringsnivån förbjuder. |
| **Durability – beständighet** | När en transaktion har bekräftats ska dess ändringar överleva ett systemfel, inom de garantier systemets konfiguration erbjuder. |

Vi har redan mött integritetsregler som `PRIMARY KEY`, `FOREIGN KEY` och `CHECK`. De är exempel på regler som hjälper oss att upprätthålla **konsistens**.

Men ACID garanterar inte automatiskt att alla verksamhetsregler är rätt modellerade. Om vi tillåter en affärsmässigt orimlig förändring genom en tekniskt giltig SQL-sats kan databasen ändå genomföra den.

## När flera arbetar samtidigt

Anta att det bara finns **ett exemplar** kvar av en bok och att två kunder försöker köpa det samtidigt.

Om båda applikationerna först läser `stock = 1` och sedan var för sig beslutar att ett köp är möjligt, finns risk att båda registrerar en beställning.

Ett bättre sätt att reservera ett exemplar kan vara att göra kontrollen och förändringen i samma `UPDATE`:

```
UPDATE products
SET stock = stock - 1
WHERE id = 1
  AND stock >= 1;
```

Efteråt måste vi kontrollera att **exakt en rad** uppdaterades. Om ingen rad uppdaterades fanns inte tillräckligt lager vid tillfället för uppdateringen.

En villkorlig `UPDATE` är användbar därför att kontrollen och ändringen av raden sker tillsammans i databasen. Men orderflödet behöver fortfarande en transaktion som kopplar lagerändringen till beställningen.

## Avancerat

### Isolering är inte bara av eller på

En databas kan ha flera pågående transaktioner samtidigt. **Isolering** beskriver vilka effekter en transaktion får observera från andra transaktioner.

SQL beskriver flera isoleringsnivåer:

| Isoleringsnivå | Grundidé |
|---|---|
| **READ UNCOMMITTED** | Tillåter mycket svag isolering, där smutsiga läsningar kan förekomma. |
| **READ COMMITTED** | En läsning ska inte se förändringar från andra transaktioner som ännu inte committats. |
| **REPEATABLE READ** | Ger starkare skydd för upprepade läsningar inom en transaktion. |
| **SERIALIZABLE** | Ska ge ett resultat som motsvarar någon seriell ordning av transaktionerna. |

Det här är en orientering, inte en fullständig specifikation. Samma benämning kan innebära något olika praktiskt beteende i olika SQL-system.[32]

## Tre typer av samtidighetsproblem

**Smutsig läsning (*dirty read*).** En transaktion läser en förändring som en annan transaktion ännu inte har committat. Om den senare gör `ROLLBACK` läste vi information som aldrig blev bestående.

**Icke-upprepbar läsning (*non-repeatable read*).** Vi läser en rad två gånger i samma transaktion, men får olika värden därför att en annan transaktion har ändrat och committat raden mellan läsningarna.

**Fantomläsning (*phantom read*).** Vi upprepar en Query som söker efter rader med ett visst villkor och får en förändrad mängd matchande rader, exempelvis därför att andra transaktioner har lagt till matchande poster.

Det finns dessutom problem som **förlorade uppdateringar (*lost updates*)** och **skrivskew (*write skew*)**. Dessa fångas inte fullständigt av den enkla uppräkningen av de tre klassiska läsanomalierna.

## Förlorade uppdateringar

Anta att två administrativa program samtidigt läser samma lagersaldo: `12`.

Det ena programmet vill lägga till fem exemplar och det andra vill minska lagret med två.

Om programmen räknar var för sig och sedan skriver tillbaka `17` respektive `10` kan den ena förändringen skriva över den andra.

Vi förväntade oss `15` men kan få `17` eller `10`.

Att utföra förändringen i SQL med:

```
UPDATE products
SET stock = stock + 5
WHERE id = 1;
```

är därför ofta säkrare än att läsa ett tal, räkna ut ett nytt värde i applikationen och sedan skriva tillbaka det. Databasen kan hantera låsning av uppdateringen.

Men detta löser inte automatiskt alla samtidighetsproblem i ett flerstegsflöde.

## Låsning och radversionering

För att hantera samtidighet kan databashanterare använda bland annat **låsning** och **radversionering**, ofta kallad *MVCC* (*Multi-Version Concurrency Control*).

Med låsning kan en transaktion tillfälligt hindra andra från att göra förändringar som skulle skapa konflikt.

Med radversionering kan läsare i många situationer få en konsekvent bild av tidigare committade versioner, i stället för att vänta på en pågående skrivning.

Det betyder inte att MVCC gör låsning eller konflikter obehövliga. En databas kan kombinera flera tekniker.

## SELECT … FOR UPDATE

I MySQL och PostgreSQL kan vi låsa utvalda rader för en kommande förändring:

```
START TRANSACTION;

SELECT stock
FROM products
WHERE id = 1
FOR UPDATE;
```

```
-- Kontrollera saldot och genomför de ändringar som hör ihop.
COMMIT;
```

`FOR UPDATE` används när vi behöver läsa något och sedan fatta beslut om en förändring inom samma transaktion.

Vilka lås som tas och vad andra transaktioner får göra under tiden varierar med databassystem, lagringsmotor, index och isoleringsnivå.[33]

För ett enkelt lageravdrag kan vår tidigare **villkorliga `UPDATE`** ofta vara ett bättre och enklare val, eftersom vi då inte behöver separera kontrollen från ändringen.

## Ett säkrare köp – transaktionens logik

Vi kan nu formulera ett mer genomtänkt orderflöde:

1. Starta en transaktion.
2. Försök att minska lagersaldot med `UPDATE ... WHERE stock >= quantity`.
3. Kontrollera att **exakt en rad påverkades**; annars gör `ROLLBACK` och meddela att lagret inte räcker.
4. Registrera orderhuvud och orderrader med rätt pris vid köptillfället.
5. Gör `COMMIT` endast om **alla** nödvändiga operationer lyckades.

Själva kontrollen av antalet påverkade rader och felhanteringen utförs ofta av applikationen eller genom databasspecifika funktioner och lagrade procedurer.

Om flera olika produkter köps samtidigt behöver vi tänka igenom i vilken ordning lagerposter uppdateras och hur vi hanterar konflikter.

## Deadlocks och återförsök

Anta att två transaktioner försöker uppdatera två produkter i motsatt ordning.

Den första låser produkt A och väntar på B. Den andra låser B och väntar på A.

Ingen kan fortsätta. Detta kallas en **deadlock**.

Databashanteraren kan då upptäcka situationen och avbryta en av transaktionerna så att den andra kan fortsätta.

Applikationen behöver kunna hantera felet och ibland **försöka om hela transaktionen**.

Det är viktigt att skilja på att försöka om en enskild SQL-sats och att försöka om hela verksamhetshändelsen. Vid återförsök behöver vi också tänka på om en operation kan ha utförts tidigare, exempelvis om nätverksanslutningen försvann innan applikationen fick svar.

## Transaktioner, DDL och systemgränser

Vi har sett att `COMMIT` och `ROLLBACK` hjälper oss att styra ändringar av data.

Men inte alla operationer omfattas av exakt samma transaktionsgarantier. Vissa schemaändringar, exempelvis `ALTER TABLE`, kan orsaka implicita commits i MySQL. I andra databassystem finns ofta ett bredare stöd för transaktionell DDL.[24]

Och en databastransaktion styr normalt inte automatiskt vad som händer **utanför databasen**.

Om ett köp också ska skicka ett mejl, debitera ett betalsystem eller beställa leverans är det inte säkert att dessa externa handlingar kan ångras bara för att databasen gör `ROLLBACK`.

Det leder till nya arkitektoniska frågor om hur vi samordnar händelser mellan olika system.

### Vad betyder det att en händelse är klar?

En SQL-sats kan ha körts utan syntaxfel och ändå ingå i en felaktig verksamhetshändelse.

På samma sätt kan flera korrekta SQL-satser tillsammans skapa inkonsekvent information om vi inte bestämmer **vilka förändringar som måste höra ihop**.

Transaktionen är därför inte bara en teknisk funktion för att undvika fel. Den är också ett sätt att uttrycka **var gränsen går för en sammanhängande handling i verksamheten**.

Vi måste förstå vad som räknas som en färdig beställning innan vi kan bestämma när den ska committas.

## Reflektion

1. Vilka förändringar i bokhandelns orderflöde måste genomföras tillsammans, och vilka kan ske senare?
2. Varför räcker det inte att varje enskild SQL-sats är korrekt för att helheten ska bli tillförlitlig?
3. Hur förändras våra krav på databasmodellen när flera användare kan läsa och förändra samma information samtidigt?

# 21. Index och Query-prestanda

När vi skriver en Query beskriver vi vilken information vi vill ha. Vi säger inte nödvändigtvis exakt **hur** databashanteraren ska hitta den.

Det spelar kanske ingen större roll när bokhandeln har fem produkter och några kunder. Men vad händer om den får hundratusentals produkter, miljontals beställningar och många användare som söker samtidigt?

Samma Query kan fortfarande ge rätt resultat – men ta mycket olika lång tid att utföra.

För att förstå varför behöver vi skilja mellan **Queryns betydelse** och **databasens arbete med att utföra den**.

## Grund

### Hur hittar databasen en rad?

Vi börjar med vår bekanta tabell `products`:

| id | title | price | stock | product_type_id |
|---|---|---|---|---|
| 1 | Vägen hem | 200.00 | 12 | 1 |
| 2 | Den blå staden | 150.00 | 0 | 1 |
| 3 | Vinterljus | 249.00 | 4 | 1 |
| 4 | Små historier | 120.00 | 25 | 2 |
| 5 | Mellan raderna | 180.00 | 9 | 2 |

Vi vill hämta en bestämd produkt:

```
SELECT *
FROM products
WHERE id = 3;
```

Databasen skulle kunna läsa varje rad och kontrollera om `id` är `3`.

Det kallas ofta en **tabellskanning** (*table scan*). Med fem rader är det knappast ett problem. Med fem miljoner rader kan det däremot bli dyrt.

Ett **index** ger databashanteraren en annan möjlighet att hitta rätt rader.

## Vad är ett index?

Vi kan jämföra ett databasindex med ett sakregister i en bok.

Om vi söker efter ordet *transaktion* kan vi slå upp det i registret och få en hänvisning till rätt sidor, i stället för att läsa hela boken från början.

Ett databasindex fungerar inte exakt som ett bokregister, men grundidén är densamma: **en extra struktur hjälper oss att hitta information utan att undersöka allt**.

Många vanliga databasindex bygger på trädstrukturer, ofta B-träd eller närbesläktade varianter. Exakt utformning varierar mellan databassystemen.

## Primärnyckeln har ofta redan ett index

Vi har tidigare bestämt att `id` är primärnyckel i `products`.

I många SQL-system skapas eller används ett index automatiskt för primärnyckeln.

Det innebär att vår Query med `WHERE id = 3` ofta redan kan använda ett index.

Vi ska därför **inte automatiskt skapa ännu ett index på `id`**.

Hur primärnyckeln lagras och vilka index som finns kan vi undersöka i databassystemets metadata.[34]

## Skapa ett index med CREATE INDEX

Anta att bokhandelns personal ofta söker efter produkter utifrån titeln:

```
SELECT id, title, price
FROM products
WHERE title = 'Vägen hem';
```

Vi kan skapa ett index på `title`:

```
CREATE INDEX idx_products_title
ON products (title);
```

Nu finns det en indexstruktur som databashanteraren **kan välja** att använda för sökningar på titlar.

Att skapa ett index innebär inte att varje relevant Query alltid använder det. Databashanteraren väljer en exekveringsplan utifrån bland annat tabellstorlek, statistik och vad den uppskattar blir effektivast.

Med vår lilla övningstabell kan en fullständig skanning fortfarande vara enklare.

## Flera kolumner i ett index

Anta att vi ofta vill hitta tillgängliga produkter av en viss typ, sorterade efter pris:

```
SELECT title, price
FROM products
WHERE product_type_id = 1
  AND stock > 0
ORDER BY price;
```

Vi kan överväga ett sammansatt index:

```
CREATE INDEX idx_products_type_price
ON products (product_type_id, price);
```

Indexet är ordnat med `product_type_id` som första indexkolumn och `price` som andra.

Det kan vara användbart när vi filtrerar på produkttyp och därefter vill läsa produkterna i prisordning.

Men vårt villkor `stock > 0` kan fortfarande kräva ytterligare arbete. Ett index löser inte automatiskt alla delar av en Query.

## Ordningen i ett sammansatt index

Ordningen mellan indexkolumnerna spelar roll.

Ett index på:

```
(product_type_id, price)
```

är inte generellt likvärdigt med ett index på:

```
(price, product_type_id)
```

Det första är ofta mer användbart om vi börjar med att söka på en bestämd produkttyp.

Vi kan tänka på det som ett register sorterat först efter produkttyp, sedan efter pris inom varje produkttyp.

Det finns undantag och olika optimeringar i olika system, men ordningen är en viktig del av indexets utformning.

## Hur vet vi om ett index används?

För att undersöka hur databashanteraren planerar att köra en Query kan vi använda `EXPLAIN`:

```
EXPLAIN
SELECT id, title, price
FROM products
WHERE title = 'Vägen hem';
```

I MySQL kan resultatet bland annat ge information om vilka tabeller som läses, vilka index som skulle kunna användas och vilket index som valdes.

Vi kan också undersöka hur en Query planeras när vi filtrerar och sorterar:

```
EXPLAIN
SELECT title, price
FROM products
WHERE product_type_id = 1
ORDER BY price;
```

**En exekveringsplan** beskriver vilka operationer databasen tänker använda. Det är en möjlighet att förstå *hur* databasen försöker besvara Queryn, inte bara *vad* Queryn betyder.

Vi redovisar inget fast resultat från `EXPLAIN` här, eftersom det beror på databas, version, statistik, index och mängden data.[35]

## Avancerat

### Att uppskatta kostnaden för en Query

En databashanterare kan ofta välja mellan flera vägar till samma resultat.

Den kan läsa hela tabellen, följa ett index, kombinera olika index eller välja mellan flera sätt att genomföra en `JOIN`.

Optimeraren försöker uppskatta kostnaden för alternativen utifrån tillgänglig information, exempelvis:

- Hur många rader en tabell innehåller.
- Hur stor andel av raderna som sannolikt uppfyller ett villkor.
- Vilka index som finns och hur de är organiserade.
- Kostnaden för att läsa data, sortera och kombinera rader.

Det är viktigt att förstå att en plan kan bygga på **uppskattningar**, inte fullständig förhandskunskap om resultatet.

## Selektivitet – hur mycket väljer vi ut?

Anta att nästan alla produkter har `stock > 0`.

Ett index på `stock` kanske inte hjälper särskilt mycket för:

```
SELECT *
FROM products
WHERE stock > 0;
```

Om Queryn ändå ska hämta nästan hela tabellen kan det vara effektivare att läsa den direkt.

Men ett index på en kolumn som ofta används för att hitta **en mycket liten andel** av raderna kan vara värdefullt.

Det brukar beskrivas med begreppet **selektivitet**.

Samtidigt räcker det inte att en kolumn har många olika värden för att ett index alltid ska vara bra. Vi måste se till de faktiska sökningarna.

## Index hjälper även vid JOIN

Vi har kopplat produkter till produkttyper med `product_type_id`.

```
SELECT
    p.title,
    t.name AS product_type
FROM products AS p
INNER JOIN product_types AS t
    ON p.product_type_id = t.id;
```

Index på kolumner som används i relationsvillkor kan hjälpa databashanteraren att hitta matchande rader.

Men det betyder inte att varje `JOIN` alltid bör använda index. Om stora delar av tabellerna ändå behöver kombineras kan databasen välja andra strategier.

Index på främmande nycklar är därför ofta användbara, men vi ska inte utgå från att de skapas automatiskt i alla SQL-system.[36]

## Covering index – när indexet räcker

Titta på denna Query:

```
SELECT product_type_id, price
FROM products
WHERE product_type_id = 1
ORDER BY price;
```

Om vi har ett sammansatt index på `(product_type_id, price)` kan indexet i vissa system innehålla allt som behövs för att besvara Queryn.

Då behöver databashanteraren kanske inte läsa själva tabellraderna separat. Detta brukar kallas ett **covering index**, eller en index-only-liknande läsning.

Men en sådan optimering beror på indexets lagring, databassystemet och ibland synlighetsinformation om raderna.

## Varför fungerar inte alltid en indexerad sökning?

Ett index på `title` kan vara användbart när vi jämför en titel med ett exakt värde:

```
WHERE title = 'Vägen hem'
```

Det kan ibland också hjälpa för ett prefix:

```
WHERE title LIKE 'Vägen%'
```

Men en sökning med ett inledande jokertecken:

```
WHERE title LIKE '%vägen%'
```

kan ofta inte utnyttja ett vanligt B-trädsindex på samma sätt.

Vilket index som kan användas påverkas också av kollationer och databasens hantering av text.

Det här är ett exempel på att **hur vi uttrycker ett villkor** kan påverka vilka sökvägar databasen har tillgång till.

## När vi använder funktioner i villkor

Anta att vi vill jämföra ett omvandlat värde:

```
SELECT *
FROM products
WHERE LOWER(title) = 'vägen hem';
```

Ett vanligt index på `title` är inte nödvändigtvis direkt användbart för villkoret `LOWER(title)`.

Vissa databassystem erbjuder uttrycksindex eller andra lösningar för sådana fall. Men vi behöver undersöka vilket stöd den faktiska databasen har.

Det är alltså inte så att SQL-funktioner är problematiska i sig, utan att de kan förändra **vilket uttryck databasen behöver söka efter**.

## Index har en kostnad

Varje index tar lagringsutrymme.

När vi lägger till en produkt eller ändrar ett indexerat värde kan databasen också behöva uppdatera indexstrukturen.

Därför påverkar index inte bara läsningar utan även `INSERT`, `UPDATE` och `DELETE`.

Det skapar en avvägning:

| Fler index kan ge | Men innebär också |
|---|---|
| Snabbare sökningar för vissa Queries | Mer lagringsutrymme |
| Effektivare vissa `JOIN` och sorteringar | Mer arbete vid förändringar av data |
| Bättre stöd för vissa åtkomstmönster | Fler strukturer att underhålla och förstå |

Vi bör alltså inte skapa index på varje kolumn bara för säkerhets skull.

## Att undersöka en verklig prestandafråga

Anta att personalen upplever att en produktsökning blivit långsam.

Vi kan arbeta systematiskt:

1. **Beskriv problemet.** Vilken Query är långsam, hur ofta körs den och hur mycket data läser den?
2. **Studera planen.** Använd `EXPLAIN` och undersök tabellskanningar, sorteringar, index och uppskattade radantal.
3. **Formulera en hypotes.** Skulle exempelvis ett index på en viss kombination av kolumner hjälpa?
4. **Prova och mät.** Jämför samma Query och liknande data före och efter ändringen.
5. **Värdera helheten.** Förbättrades läsningen utan att skrivningar, lagringskostnad eller andra Queries påverkades orimligt mycket?

I flera databassystem går det också att mäta faktisk exekvering och jämföra uppskattade med verkliga radantal. Sådana mätningar behöver göras med omsorg: att köra en Query kan belasta systemet, och mätresultat påverkas av cache och parallella aktiviteter.[37]

## Prestanda handlar också om modellering

Vi har tidigare lärt oss att tabeller ska beskriva tydliga företeelser och relationer.

Nu kan vi se att datamodellen påverkar vilka Queries som behöver skrivas och vilka index som är användbara.

En modell med tydliga nycklar och relationer gör det ofta lättare att förstå hur information ska kombineras. Men en väl normaliserad modell är inte automatiskt optimal för varje analytiskt informationsbehov.

I vissa system kan vi behöva komplettera den operativa modellen med Views, särskilda sammanställningar eller andra representationsformer.

Det betyder inte att vi ska överge våra principer om dataintegritet. Det betyder att vi måste förstå **vilken användning en viss modell är byggd för**.

### Rätt svar och rätt arbetsinsats

Två Queries kan ge exakt samma resultat men kräva olika mycket arbete av databashanteraren.

Och samma Query kan köras med olika exekveringsplaner när mängden data förändras.

Vi behöver därför kunna resonera på två nivåer:

**Vad betyder vår Query?** Vilken information beskriver resultatet?

**Hur kan databasen ta fram det?** Vilka strukturer, operationer och resurser behövs?

Att förstå båda nivåerna hjälper oss att skapa system som inte bara ger korrekta svar, utan också fungerar när verksamheten växer.

## Reflektion

1. Varför kan ett index vara mycket användbart för en Query men nästan värdelöst för en annan?

2. Varför skapar vi inte bara så många index som möjligt?
3. Vad är skillnaden mellan att förbättra en Querys logiska tydlighet och att förbättra dess faktiska exekveringsprestanda?

# 22. SQL i applikationer

Hittills har vi skrivit SQL direkt mot bokhandelns databas. Vi har formulerat Queries, förändrat data och undersökt hur databashanteraren arbetar.

Men kunderna kommer inte att skriva SQL när de handlar i vår bokhandel. De använder kanske en webbplats eller en app. Personalen använder ett administrativt system.

Hur kommer då deras handlingar fram till databasen?

Mellan användaren och databasen finns ett program som behöver **översätta verksamhetens handlingar till SQL**, tolka svaren och hantera de fel som kan uppstå.

## Grund

### Från användarhandling till Query

Anta att en kund söker efter en bok med ordet *vägen* i titeln.

Vi kan beskriva händelsen så här:

1. Kunden anger en söktext.
2. Applikationen tar emot söktexten.
3. Applikationen skickar en parametriserad Query till databasen.
4. Databasen returnerar de matchande raderna.
5. Applikationen presenterar böckerna för kunden.

Det är viktigt att skilja på **användarens avsikt**, **applikationens logik** och **databasens ansvar**.

## En SQL-anslutning

För att använda SQL från ett program behöver vi normalt en **databasdrivrutin**: programkod som förstår hur vårt programs miljö och databashanteraren kommunicerar.

I C# används ofta ADO.NET tillsammans med en drivrutin för den aktuella databasen.

I våra exempel använder vi MySQL och NuGet-paketet `MySqlConnector`.[38]

En möjlig anslutning ser ut så här:

```
using MySqlConnector;

var connectionString =
    "Server=localhost;Database=bookstore;User
ID=bookstore_app;Password=...;";

await using var connection = new MySqlConnection(connectionString);
await connection.OpenAsync();
```

Nu kan applikationen kommunicera med databasen.

Exemplet visar grundprincipen. I en verklig applikation ska lösenord och andra hemligheter **inte skrivas direkt i källkoden**, utan hämtas från exempelvis säker konfiguration eller miljövariabler.

## Att köra en Query

Vi vill hämta bokhandelns tillgängliga produkter:

```
SELECT id, title, price
FROM products
WHERE stock > 0
ORDER BY title;
```

Från C# kan vi skicka samma Query till databasen:

```
await using var command = new MySqlCommand("""
    SELECT id, title, price
```

```
    FROM products
    WHERE stock > 0
    ORDER BY title;
    """, connection);

await using var reader = await command.ExecuteReaderAsync();

while (await reader.ReadAsync())
{
    int id = reader.GetInt32("id");
    string title = reader.GetString("title");
    decimal price = reader.GetDecimal("price");

    Console.WriteLine($"{id}: {title} – {price} kr");
}
```

En Query som returnerar rader kan behandlas med en **data reader**, som låter applikationen läsa resultatet rad för rad.

Vi har inte behövt förändra SQL-syntaxen för att anropa databasen från ett annat programmeringsspråk. Däremot behöver programmet hantera anslutningen och tolka resultatet.

## Parametrar – värden är inte SQL-kod

Nu vill vi söka efter en produkt med ett bestämt id.

Det kan vara frestande att bygga en Query genom att sätta ihop text:

```
// Osäkert mönster – bygg inte SQL från användarinmatning så här.
string sql = $"SELECT title FROM products WHERE id = {userInput}";
```

Problemet är att ett värde från användaren då riskerar att bli en del av **själva SQL-instruktionen**.

Vi ska i stället använda **parametriserade Queries**:

```
await using var command = new MySqlCommand("""
    SELECT title, price
    FROM products
    WHERE id = @productId;
    """, connection);

command.Parameters.AddWithValue("@productId", productId);
```

```
await using var reader = await command.ExecuteReaderAsync();

if (await reader.ReadAsync())
{
    Console.WriteLine(reader.GetString("title"));
}
```

Här antar vi att `productId` redan är ett heltal som applikationen har tagit emot och validerat.

Parametern `@productId` är en **plats för ett värde**. Värdet skickas separat från Queryns struktur, så att användarinmatning inte ska kunna omtolkas som SQL-syntax.

Detta är ett grundläggande skydd mot **SQL-injektion**.

## SQL-injektion

SQL-injektion uppstår när ett program låter opålitlig text påverka konstruktionen av en SQL-sats.

Problemet är inte att databasen missförstår SQL. Problemet är att applikationen har byggt en annan instruktion än den avsåg.

Parametrar hjälper oss att skilja **data från instruktioner**.

Men parametrar kan normalt användas för *värden*, inte för godtyckliga tabellnamn, kolumnnamn eller SQL-nyckelord. Om användaren ska kunna välja sorteringskolumn behöver applikationen till exempel välja mellan en **fördefinierad lista av tillåtna alternativ**, inte lägga in användarens text direkt i `ORDER BY`.

## Att förändra data från ett program

När bokhandeln ändrar ett lagersaldo använder applikationen en annan typ av kommando:

```
await using var command = new MySqlCommand("""
    UPDATE products
    SET stock = stock + @quantity
```

```
    WHERE id = @productId;
    """, connection);

command.Parameters.AddWithValue("@quantity", 5);
command.Parameters.AddWithValue("@productId", 1);

int changedRows = await command.ExecuteNonQueryAsync();
```

`ExecuteNonQueryAsync()` används för satser som inte förväntas ge ett vanligt radresultat, exempelvis `UPDATE` och `DELETE`.

Returvärdet ger oss information om påverkade rader, men **vad som räknas som en påverkad rad kan bero på drivrutinens och databasens inställningar**. Vid ett kritiskt lageravdrag behöver vi därför förstå exakt vad returvärdet betyder.

## Avancerat

### Vem ansvarar för vad?

Applikationen och databasen samarbetar, men deras ansvar är inte identiska.

| Del | Exempel på ansvar |
|---|---|
| **Användargränssnitt** | Samla in söktext, visa produkter och felmeddelanden |
| **Applikationslogik** | Tolka önskemål, genomföra arbetsflöden och kontrollera behörighet |
| **Databas** | Lagra information, upprätthålla definierade integritetsregler, besvara Queries och hantera transaktioner |

Gränserna är inte absoluta. En verksamhetsregel kan behöva kontrolleras på flera nivåer: i gränssnittet för tydlig återkoppling, i applikationen för att styra processen och i databasen för att skydda informationens integritet.

En databaskoppling får däremot inte ersätta en genomtänkt behörighetsmodell.

## Transaktioner från applikationen

I kapitel 20 såg vi varför beställning och lageravdrag behöver höra ihop.

Ett program kan starta en transaktion och koppla flera SQL-satser till den.

Följande förenklade exempel visar bara ett lageravdrag med kontroll av resultatet:

```
await using var transaction =
    await connection.BeginTransactionAsync();

try
{
    await using var command = new MySqlCommand("""
        UPDATE products
        SET stock = stock - @quantity
        WHERE id = @productId AND stock >= @quantity;
        """, connection, transaction);

    command.Parameters.AddWithValue("@quantity", 2);
    command.Parameters.AddWithValue("@productId", 1);

    int changedRows = await command.ExecuteNonQueryAsync();

    if (changedRows != 1)
    {
        throw new InvalidOperationException("Otillräckligt
lagersaldo.");
    }

    // Här skulle fler förändringar som hör till köpet utföras
    // inom samma transaktion.

    await transaction.CommitAsync();
}
catch
{
    await transaction.RollbackAsync();
    throw;
}
```

Detta är fortfarande **inte en komplett beställningsprocess**. I ett verkligt köp behöver vi dessutom registrera beställningen, säkerställa rätt priser och hantera fel som uppstår under eller efter commit.

Det avgörande är att varje kommando som hör till transaktionen använder **samma anslutning och samma transaktion**.

Vid allvarliga anslutningsfel kan en rollback i sig misslyckas. Applikationen behöver också kunna hantera osäkerhet om huruvida en commit hann genomföras.

## Att översätta rader till objekt

I ett objektorienterat program vill vi kanske inte arbeta direkt med kolumnnamn överallt.

Vi kan ha en klass:

```
public sealed class Product
{
    public int Id { get; init; }
    public string Title { get; init; } = "";
    public decimal Price { get; init; }
}
```

Och skapa objekt utifrån databasens resultat:

```
var product = new Product
{
    Id = reader.GetInt32("id"),
    Title = reader.GetString("title"),
    Price = reader.GetDecimal("price")
};
```

Det här brukar kallas **mappning** mellan relationsdata och applikationens objekt.

Men det betyder inte att en tabell alltid motsvarar exakt en klass, eller att varje JOIN ska skapa ett nytt objekt av samma typ.

Tabeller är modeller för lagring och relationer. Objekt är modeller för hur ett program organiserar sin logik. De kan likna varandra, men behöver inte ha identisk struktur.

## ORM – ett annat sätt att arbeta

Ett **Object-Relational Mapper (ORM)** kan hjälpa programmet att läsa och förändra databasens information via objekt och uttryck i programmeringsspråket.

I .NET-världen är **Entity Framework Core** ett vanligt exempel.

Med ett ORM kan applikationen i många fall formulera frågor med exempelvis LINQ, och verktyget översätter dem till SQL.

Men SQL har inte försvunnit. Databasen behöver fortfarande exekvera en Query, och vi behöver fortfarande förstå relationer, index, transaktioner och vilka SQL-satser som faktiskt genereras.

För vissa situationer är direkt SQL tydligare. För andra kan ett ORM minska mängden upprepande kod.

## Att undvika för många databasfrågor

Anta att applikationen först hämtar hundra produkter och sedan gör en separat Query för att hämta produkttypen för **varje produkt**.

Då kan vi få hundra extra Queries för information som kanske hade kunnat hämtas med en JOIN.

Det kallas ofta **N+1-problemet**: en Query för listan och ytterligare en per rad.

Problemet kan uppstå både med handskriven SQL och med ORM-verktyg, exempelvis genom hur relaterade objekt laddas.

Vi behöver därför tänka på **hur många gånger programmet kommunicerar med databasen**, inte bara hur en enskild Query ser ut.

## Anslutningar och connection pooling

Att öppna en fysisk databasanslutning kan innebära nätverksarbete och autentisering.

Därför använder många drivrutiner **connection pooling**: färdiga anslutningar kan återanvändas när ett program behöver dem.

I C# kan vi ändå skriva `await using` och avsluta vår användning av anslutningsobjektet. Med pooling innebär det normalt att anslutningen återlämnas till poolen i stället för att varje gång behöva etableras från början.

Men en anslutningspool är en begränsad resurs. Vi ska inte låta läsare, transaktioner eller anslutningar vara öppna längre än nödvändigt.

## Felhantering och tillförlitlighet

Applikationen behöver skilja mellan olika slags fel:

- **Ogiltiga värden:** exempelvis en prisuppgift som inte får vara negativ.
- **Integritetsfel:** exempelvis en främmande nyckel som hänvisar till en post som saknas.
- **Samtidighetskonflikter:** exempelvis en deadlock eller ett lager som hann ta slut.
- **Anslutningsproblem:** exempelvis tidsgränser eller avbruten nätverkskontakt.

En viktig skillnad är om ett fel ska leda till att vi **rättar indata**, **avbryter arbetet** eller **försöker om en hel transaktion**.

Vi ska också vara försiktiga med vad vi loggar. Databasfel kan innehålla information om tabeller, parametrar eller andra detaljer som inte bör visas för slutanvändaren eller sparas okontrollerat.

## Säkerheten börjar före SQL-satsen

Parametriserade Queries skyddar mot en viktig klass av angrepp, men de ersätter inte andra kontroller.

Applikationen behöver verifiera att den som begär en åtgärd faktiskt har rätt att göra den.

Och databasanslutningen bör använda ett konto som bara har de rättigheter applikationen behöver. Ett vanligt användarflöde ska exempelvis inte ansluta till produktionsdatabasen som administratör.

Vi utvecklar detta i nästa kapitel om säkerhet och åtkomst.

## Vad tillhör databasen – och vad tillhör programmet?

Vi kan lägga mycket logik i applikationen, i databasen eller i en kombination av båda.

Men valet bör bygga på **vilket ansvar logiken har**.

Databasen lämpar sig väl för att skydda sådant som alltid måste vara sant om informationen: nycklar, relationer och viktiga begränsningar.

Applikationen behöver förstå arbetsflödet: vad en kund försöker göra, vilka steg som ska utföras och vilken återkoppling kunden behöver.

Det kan finnas verksamhetsregler som behöver kontrolleras i båda miljöerna av olika skäl.

**Vi ska inte låta databasen bli hela applikationen. Men vi ska heller inte göra databasen till en passiv förvaringsplats utan ansvar för sina data.**

## Reflektion

1. Varför är en parametriserad Query något annat än en SQL-sträng som satts ihop av programmet?
2. Vilka kontroller hör bäst hemma i databasen, och vilka behöver utföras i applikationen?
3. Vad kan gå fel om ett program behandlar varje databasoperation för sig, trots att användaren uppfattar dem som en enda handling?

# 23. Säkerhet och åtkomst

Bokhandelns databas innehåller information som många kan behöva använda, men inte alla behöver kunna göra samma saker med den.

Kunder ska kunna se vilka böcker som finns. Personal kan behöva ändra lagersaldon. En administratör kan behöva hantera användarkonton och databasens struktur. Men det betyder inte att alla dessa roller ska få läsa och förändra all information.

En viktig del av databassäkerhet är därför att bestämma **vem som får göra vad, med vilken information och under vilka förutsättningar**.

## Grund

### Att identifiera och tillåta

Vi behöver skilja mellan två frågor:

- **Autentisering:** Vem är det som försöker ansluta?
- **Auktorisation:** Vad får den identifierade aktören göra?

Autentisering kan exempelvis ske med användarnamn och lösenord, certifikat eller en identitetstjänst. Auktorisation handlar om vilka rättigheter den identiteten har.

Vi kan också skilja mellan **databasanvändaren** som en applikation ansluter med och **slutanvändaren** som är inloggad i applikationen.

Om hundra kunder använder bokhandelns webbplats behöver det inte innebära hundra olika databaskonton. Webbapplikationen kan använda en särskild databasidentitet och själv kontrollera vilka kunduppgifter den enskilde kunden får se.

## Användare och rättigheter

I MySQL kan vi skapa ett konto för bokhandelns applikation:

```
CREATE USER 'bookstore_app'@'localhost'
IDENTIFIED BY 'ett-starkt-unikt-losenord';
```

Det här är ett syntaxexempel. Ett riktigt lösenord ska hanteras säkert och inte sparas i versionshanterade skript.

I MySQL ingår även anslutningsvärden som `localhost` i hur ett konto identifieras. Andra databassystem organiserar användare och anslutningsbehörigheter annorlunda.[39]

Att ett konto finns betyder inte att det automatiskt har rätt att läsa och förändra bokhandelns tabeller.

## GRANT – att tilldela rättigheter

Vi kan ge kontot rätt att läsa information om produkterna:

```
GRANT SELECT
ON bookstore.products
TO 'bookstore_app'@'localhost';
```

`GRANT` tilldelar en viss rättighet till en identitet.

Nu kan kontot läsa `products`, men den här instruktionen ger inte rätt att ändra pris, ta bort produkter eller skapa nya tabeller.

Om ett administrativt system också behöver ändra lagersaldon kan vi ge en annan identitet en mer avgränsad rättighet:

```
GRANT SELECT, UPDATE
ON bookstore.products
TO 'bookstore_staff'@'localhost';
```

Det förutsätter att kontot `bookstore_staff` redan finns. Även detta exempel är förenklat: `UPDATE` på hela tabellen ger möjlighet att ändra *alla* kolumner, inte bara lagersaldot.

## Minsta privilegium

Principen om **minsta privilegium** (*least privilege*) innebär att en identitet bara får de rättigheter den faktiskt behöver.

Det är ofta ett bättre utgångsläge att börja med få rättigheter och lägga till dem efter behov än att ge alla full åtkomst.

Om webbapplikationen bara behöver läsa produktkatalogen bör den till exempel inte kunna:

```
DROP TABLE products;
```

Även om applikationen aldrig avser att utföra en sådan operation är det värdefullt att databasen kan neka den.

Vi skyddar alltså inte bara mot avsiktliga angrepp, utan också mot programmeringsfel och oavsiktliga handlingar.

## REVOKE – att ta tillbaka rättigheter

Vi kan återkalla en tidigare tilldelad rättighet:

```
REVOKE UPDATE
ON bookstore.products
FROM 'bookstore_staff'@'localhost';
```

Efter detta har identiteten inte längre denna direkta `UPDATE`-rättighet på tabellen.

Men det är viktigt att komma ihåg att rättigheter också kan komma från andra tilldelningar, exempelvis roller eller rättigheter på en överordnad nivå. `REVOKE` av en enda tilldelning betyder därför inte alltid att alla möjligheter till samma operation försvinner.

## Roller

I stället för att hantera varje kontos rättigheter separat kan vi i många databassystem använda **roller**.

I MySQL 8 kan vi exempelvis skapa en roll för dem som behöver läsa bokhandelns produkter:

```
CREATE ROLE 'catalog_reader';

GRANT SELECT
ON bookstore.products
TO 'catalog_reader';

GRANT 'catalog_reader'
TO 'bookstore_app'@'localhost';
```

Detta förutsätter att applikationskontot redan finns. För att rollens rättigheter ska vara aktiva måste även rollaktiveringen hanteras enligt databasens regler, till exempel genom en standardroll eller genom att aktivera rollen i sessionen.[40]

En roll är alltså ett sätt att samla **vilka handlingar en viss funktion behöver kunna utföra**.

## Avancerat

### Behörighet på flera nivåer

Vi kan tänka oss flera nivåer av åtkomst:

| Nivå | Exempel |
|---|---|
| Databasserver | Får identiteten ansluta till databashanteraren? |
| Databas eller schema | Får identiteten använda en viss del av databasen? |
| Tabell eller View | Får identiteten läsa eller ändra ett visst databasobjekt? |
| Kolumn | Får identiteten läsa eller ändra vissa attribut? |
| Rad eller informationspost | Vilka enskilda poster får en användare se? |

Alla SQL-system hanterar inte dessa nivåer på samma sätt. Särskilt **radbaserad åtkomstkontroll** varierar mellan databassystem.

I en webbapplikation måste vi dessutom avgöra vilka rättigheter den inloggade kunden har, även om många kunder delar samma databasanslutning.

## En View som begränsat informationsgränssnitt

I kapitel 17 såg vi hur en View kan visa en del av informationen från en tabell.

Anta att kundtabellen innehåller både namn, e-postadress och födelsedatum, men att en intern namnlista bara behöver identifierare och namn:

```
CREATE VIEW customer_names AS
SELECT id, name
FROM customers;
```

Vi kan ge en särskild roll rätt att läsa just denna View:

```
GRANT SELECT
ON bookstore.customer_names
TO 'name_reader';
```

Det förutsätter att rollen redan är skapad.

Om rollen inte också har åtkomst till den underliggande tabellen kan vyn utgöra ett avgränsat gränssnitt. Men säkerheten beror på **hur databassystemet kontrollerar behörigheter när vyn körs** och vilka andra rättigheter som finns.[41]

Vi ska alltså inte anta att en View skyddar information bara för att den visar färre kolumner.

## Radbaserad åtkomst

Anta att varje kund ska få se sina egna beställningar, men inte andras.

En applikation kan formulera:

```
SELECT id, ordered_at
FROM orders
WHERE customer_id = @customerId;
```

Parametern `@customerId` är här ett exempel på en parametriserad Query från applikationskod, inte en fristående SQL-variabel som fungerar likadant i alla databassystem.

Det avgörande är **varifrån kund-id kommer**.

Om applikationen låter användaren ange vilket kund-id som helst utan kontroll kan kunden kanske läsa någon annans beställningar.

Applikationen behöver därför härleda identiteten från en betrodd inloggningsprocess och kontrollera rätten till den efterfrågade informationen.

Vissa databassystem har dessutom mekanismer för radbaserade säkerhetsregler, exempelvis PostgreSQL:s *Row-Level Security* och SQL Servers *Row-Level Security*. MySQL saknar en direkt motsvarande generell inbyggd funktion, vilket gör applikationskontroller och väl utformade databasgränssnitt särskilt viktiga i sådana miljöer.[42]

## SQL-injektion är inte bara en fråga om indata

Vi mötte SQL-injektion i kapitel 22.

Parametriserade Queries gör att en användares text inte kan tolkas som SQL-instruktioner där en parameter används som värde.

Men även en applikation med parametriserade Queries behöver begränsa **vad databasidentiteten har rätt att göra**.

Om ett angrepp ändå ger tillgång till en applikations databasanslutning bör kontot inte ha fler privilegier än nödvändigt.

Det är ett exempel på **försvar på flera nivåer** (*defence in depth*): flera oberoende skydd som minskar konsekvenserna om ett av dem misslyckas.

## Säker information är mer än rättigheter

Även om åtkomstkontrollen fungerar behöver vi tänka på hur informationen hanteras.

Exempelvis:

- **Överföring:** Är anslutningen till databasen krypterad, och kontrollerar klienten serverns identitet?
- **Lagring:** Skyddas databaser, säkerhetskopior och lagringsmedier på ett lämpligt sätt?
- **Hemligheter:** Hur förvaras lösenord, anslutningssträngar och andra åtkomstuppgifter?
- **Loggning:** Vilka händelser behöver kunna spåras, och hur undviker vi att loggarna avslöjar känslig information?
- **Säkerhetskopiering:** Går informationen att återställa efter fel, angrepp eller oavsiktlig radering?

Kryptering, åtkomstkontroll och säkerhetskopiering löser olika problem. Ingen enskild mekanism ersätter de andra.

## Kryptering och hashning – två olika syften

När vi lagrar känslig information behöver vi först avgöra **om originalvärdet någonsin ska kunna läsas igen**.

**Kryptering** omvandlar ett värde till en form som kan återställas med rätt nyckel. Det kan behövas för uppgifter som en behörig verksamhetsprocess senare måste kunna läsa.

**Hashning** beräknar däremot ett värde som inte är avsett att kunna omvandlas tillbaka till originalet. För lösenord använder vi särskilda,

avsiktligt resurskrävande **lösenordshashningsalgoritmer** – inte en vanlig snabb hashfunktion.

I MySQL finns exempelvis funktionen `SHA2()`:

```
SELECT SHA2('ett exempelvärde', 256);
```

Det är ett exempel på **vanlig hashning**, bland annat användbar i sammanhang där vi behöver ett kontrollvärde. **Det är inte ett lämpligt sätt att lagra lösenord.** En snabb hashfunktion som SHA-256 gör det möjligt att pröva stora mängder lösenordsgissningar på kort tid.

För lösenord använder vi hellre etablerade bibliotek med exempelvis **Argon2id** eller andra moderna, anpassade lösenordshashningsalgoritmer.[43]

## Att skapa och verifiera ett lösenordshashvärde

Anta att bokhandelns applikation har konton för personalen.

När en användare skapar ett lösenord behöver programmet:

1. Ta emot lösenordet via en skyddad anslutning.
2. Låta ett lösenordshashningsbibliotek skapa ett hashvärde med ett **unikt salt** och lämpliga kostnadsparametrar.
3. Lagra den resulterande lösenordsrepresentationen i databasen – **inte lösenordet i klartext**.

Vi kan illustrera skillnaden med pseudokod; funktionsnamnen nedan är **inte inbyggda SQL-funktioner**:

```
stored_hash = PasswordHasher.Hash(password)
is_valid = PasswordHasher.Verify(password_attempt, stored_hash)
```

Vid inloggning verifierar programmet det angivna lösenordet mot det lagrade värdet. Det behöver inte, och ska inte försöka, dekryptera lösenordet.

Salt är inte detsamma som en hemlig krypteringsnyckel. Ett slumpmässigt, unikt salt hjälper oss bland annat att undvika identiska hashvärden för identiska lösenord och försvårar förberäknade attacker. Etablerade bibliotek brukar själva generera och lagra salt tillsammans med algoritm och kostnadsparametrar i en strukturerad hashrepresentation.

Det är också en skillnad mellan **lösenordshashning** och att använda en hash för exempelvis filintegritet eller jämförelse av innehåll. Vi ska välja metod efter vilket problem vi vill lösa.

## Att kryptera lagrad information

MySQL har också funktioner för symmetrisk kryptering, exempelvis `AES_ENCRYPT()` och `AES_DECRYPT()`. Det visar att kryptering i vissa miljöer kan utföras av databashanteraren.

Men valet av funktion räcker inte för att skapa ett säkert system. Vi behöver bland annat hantera **nycklar, krypteringsläge, initialiseringsvärden, integritetsskydd och åtkomst** på ett genomtänkt sätt. Vi bör inte bygga en egen kryptografisk lösning kring enstaka SQL-funktioner.

I många applikationer är det lämpligt att utföra krypteringen med ett välunderhållet bibliotek **innan värdet skickas till databasen**. Då kan databasen lagra exempelvis krypterad binärdata tillsammans med den metadata som behövs för att senare kunna dekryptera den:

| customer_id | encrypted_note | key_version |
|---|---|---|
| 1 | *(krypterad binärdata)* | 2 |

Exemplet beskriver principen, inte en fullständig säker datamodell. Beroende på algoritm behöver vi också hantera exempelvis *nonce/IV* och autentiseringstagg. Nyckeln ska i möjligaste mån förvaras **separat från den krypterade informationen**, exempelvis i en avsedd nyckelhanteringstjänst.[44]

Kryptering av enskilda fält är inte heller samma sak som kryptering av hela databasens lagringsfiler eller säkerhetskopior. De skyddar mot delvis olika hot.

## När algoritmen eller nyckeln behöver bytas

Kryptografiska skydd behöver kunna förändras. En algoritm kan få kända svagheter, en nyckel kan misstänkas vara röjd eller säkerhetskraven kan förändras.

För **reversibelt krypterade data** kan vi, om vi fortfarande har tillgång till nyckeln och originalinformationen behöver bevaras:

1. Identifiera vilken algoritm och nyckelversion som användes.
2. Dekryptera uppgiften med den gamla nyckeln i en kontrollerad miljö.
3. Kryptera om den med den nya algoritmen eller nyckeln.
4. Kontrollera resultatet och avveckla det gamla skyddet när det är säkert att göra det.

Under en övergång kan systemet behöva kunna läsa både gamla och nya format. Därför är **versionsinformation** en viktig del av datamodellen.

I vissa system används *envelope encryption*: själva data krypteras med en datanyckel, som i sin tur skyddas med en separat nyckel. Då kan man ibland **kryptera om datanyckeln** med en ny överordnad nyckel utan att kryptera om alla datavärden. Om själva datanyckeln eller krypteringsalgoritmen måste bytas kan även innehållet behöva krypteras om.[45]

## När lösenordens skydd behöver ersättas

För lösenordshashar fungerar inte metoden ovan: vi kan inte dekryptera en hash och sedan skapa ett nytt hashvärde av det ursprungliga lösenordet.

En vanlig migreringsmetod är därför att **uppgradera hashvärdet vid en lyckad inloggning**:

1. Läs den lagrade lösenordsrepresentationen och identifiera dess algoritm och parametrar.
2. Verifiera användarens inmatade lösenord enligt den gamla metoden.
3. Skapa en ny lösenordshash med den moderna metoden.
4. Ersätt den gamla representationen.

Men vad gör vi med användare som inte loggar in?

Om det gamla skyddet är för svagt, eller misstänks ha komprometterats, kan vi behöva **ogiltigförklara de gamla autentiseringsuppgifterna** och kräva att berörda användare skapar nya lösenord genom en säker återställningsprocess.

Vi ska också skilja mellan att skapa **en ny hash av samma lösenord** och att faktiskt **byta lösenord**. Om äldre hashvärden redan har stulits kan ett algoritmbyte inte göra de stulna värdena ogjorda; angriparen kan fortfarande försöka knäcka dem. Vid kompromettering kan nya lösenord, återkallade sessioner och andra incidentåtgärder behövas.

Den här typen av förändringar bör planeras redan när autentiseringslösningen byggs. Systemet behöver exempelvis kunna hantera mer än ett lösenordsformat under en övergång utan att behålla osäkra format längre än nödvändigt.

## Informationsvärdet och skyddets livslängd

Det finns en viktig skillnad mellan **informationen vi behöver bevara** och **metoden vi använder för att skydda den**.

En krypterad kunduppgift kan fortfarande ha ett giltigt användningsområde även om krypteringsalgoritmen blivit föråldrad. Då vill vi kanske bevara informationen men byta skydd.

För ett lösenord är syftet inte att bevara den ursprungliga texten. Vi behöver i stället bevara eller återupprätta användarens **möjlighet att legitimera sig på ett säkert sätt**.

Det innebär att ett väl utformat system ibland måste kunna **migrera information**, och ibland måste kunna **kassera gamla autentiseringsuppgifter**.

Båda är exempel på att säkerhet också handlar om hur informationssystem kan förändras över tid.

## Personuppgifter och dataminimering

I bokhandelns kundregister finns namn, e-postadress och kanske födelsedatum.

Men behöver vi verkligen kundens födelsedatum för att sälja en bok?

Frågan handlar inte enbart om vilka data vi *kan* lagra, utan också om **varför vi lagrar dem och hur länge de behövs**.

Dataminimering innebär att vi inte samlar in mer personuppgifter än vad som är nödvändigt för ett berättigat ändamål.

Den principen påverkar datamodelleringen redan innan vi börjar diskutera `GRANT` och `REVOKE`.

Den kan också skapa en viktig spänning: vissa uppgifter behöver bevaras för exempelvis bokföring och historik, medan andra bör kunna rättas eller raderas enligt tillämpliga regler. Sådana beslut kräver både verksamhetskunskap och juridisk bedömning.

## Loggning och spårbarhet

Det kan vara viktigt att i efterhand förstå vem som ändrade ett pris eller tog bort en post.

Men en vanlig `UPDATE` säger inte automatiskt *vem* som begärde ändringen eller *varför*.

För spårbarhet kan vi behöva logga administrativa handlingar, använda särskilda historiktabeller eller stöd för databasgranskning (*auditing*).

Även här behöver vi skilja mellan databasens tekniska identitet och applikationens slutanvändare. Om alla ändringar kommer från `bookstore_app` räcker det inte att bara logga databasens kontonamn för att veta vilken anställd som gjorde ändringen.

En logg ska dessutom skyddas från obehörig ändring och inte innehålla fler personuppgifter än nödvändigt.

## Tillgänglighet är också säkerhet

Informationssäkerhet brukar ofta beskrivas genom tre grundläggande mål:

| Mål | Vad det betyder |
|---|---|
| **Konfidentialitet** | Information ska inte avslöjas för obehöriga. |
| **Integritet** | Information ska vara korrekt och skyddad mot otillåten eller oavsiktlig förändring. |
| **Tillgänglighet** | Behöriga användare ska kunna komma åt information när de behöver den. |

En databas som är perfekt skyddad mot läsning men inte kan användas av verksamheten är inte en fungerande lösning.

Säkerhetsarbetet innebär därför avvägningar mellan skydd, funktion, administration och återställningsförmåga.

## Säkerhetsregler som en del av datamodellen

Vi har tidigare diskuterat datamodellen som ett avtal om vad informationen betyder.

Nu kan vi lägga till en dimension: modellen och dess omgivande behörighetsregler uttrycker också **vem som får veta något och vem som får förändra det**.

En tabell kan innehålla korrekt information och en Query kan vara tekniskt giltig, men det betyder inte att alla ska kunna köra den.

Därför behöver vi skilja mellan tre frågor:

**Är informationen korrekt representerad?**

**Kan databasen besvara vår Query?**

**Har den som frågar rätt att få svaret?**

Alla tre kan vara nödvändiga för att ett informationssystem ska fungera ansvarsfullt.

## Reflektion

1. Varför räcker det inte att kontrollera användarens behörighet i webbgränssnittet om databaskontot samtidigt har full åtkomst?
2. Vad är skillnaden mellan att begränsa information med en View och att faktiskt kontrollera vem som får läsa den?
3. Hur påverkas vår datamodell av frågan om vilka uppgifter vi bör lagra – inte bara vilka uppgifter vi tekniskt kan lagra?

# 24. Datakvalitet och tillförlitlighet

Bokhandelns databas kan innehålla värden som är tekniskt tillåtna men ändå felaktiga.

En kund kan ha ett giltigt formaterat postnummer som tillhör en annan ort. En produkt kan ha ett pris som råkat bli tio gånger för högt. En beställning kan sakna information vi behöver för att hantera en reklamation, trots att alla tabellernas nycklar och relationer är korrekta.

Vi har tidigare arbetat med datatyper, begränsningar och normalisering. De hjälper oss att skydda informationen. Men **att databasen accepterar ett värde betyder inte att värdet beskriver verkligheten korrekt**.

Därför behöver vi också förstå och undersöka **datakvalitet**.

## Grund

### Vad menar vi med datakvalitet?

Datakvalitet handlar om i vilken utsträckning informationen fungerar för sitt avsedda användningsområde.

Det betyder att kvalitet inte bara kan bedömas utifrån tabellen. Vi behöver veta **vad uppgifterna ska användas till**.

Anta att en kundadress saknar lägenhetsnummer. För en enkel sammanställning av kunder per stad kanske det inte spelar någon roll. För att leverera ett paket kan det vara avgörande.

Vi kan därför behöva skilja mellan två frågor:

- **Följer informationen våra tekniska regler?**
- **Är informationen tillräckligt tillförlitlig för det vi vill göra?**

Båda är viktiga, men de besvarar inte samma sak.

## Några kvalitetsdimensioner

Ett vanligt sätt att resonera om datakvalitet är att beskriva olika **kvalitetsdimensioner**.

| Dimension | Fråga vi behöver ställa |
|---|---|
| **Fullständighet** (*Completeness*) | Finns de uppgifter som behövs? |
| **Korrekthet** (*Accuracy*) | Stämmer uppgifterna med verkligheten eller en tillförlitlig källa? |
| **Konsekvens** (*Consistency*) | Stämmer uppgifter överens mellan olika poster, tabeller och system? |
| **Aktualitet** (*Timeliness*) | Är informationen tillräckligt ny eller uppdaterad för användningen? |
| **Unikhet** (*Uniqueness*) | Beskriver flera poster av misstag samma företeelse? |
| **Giltighet** (*Validity*) | Följer värdena regler för format, typ eller tillåtna värden? |

Det finns flera ramverk och varianter av dessa dimensioner. I boken använder vi dem som **sätt att ställa bättre frågor om informationen**, inte som en enda universell definition av kvalitet.

## Giltighet är inte korrekthet

I tabellen `products` har vi ett pris:

```
price DECIMAL(10, 2) NOT NULL
```

Vi kan också ha en begränsning:

```
CHECK (price >= 0)
```

Då kan databasen neka negativa priser.

Men om boken `Vägen hem` egentligen ska kosta `200` kronor och någon registrerar `2 000` kronor kommer båda reglerna ändå att vara uppfyllda.

**Priset är tekniskt giltigt men sakligt felaktigt.**

Databasen kan kontrollera vissa regler själv. För att veta vilket pris som faktiskt gäller kan vi behöva jämföra med en prislista, ett beslut eller ett annat tillförlitligt underlag.

## Fullständighet – vad saknas?

Vi kan undersöka om kunder saknar e-postadress:

```
SELECT id, name
FROM customers
WHERE email IS NULL;
```

Men det räcker inte alltid att kontrollera `NULL`.

En tom text kan också sakna användbart innehåll:

```
SELECT id, name
FROM customers
WHERE email IS NULL
   OR TRIM(email) = '';
```

Det här är ett exempel på en **kvalitetskontroll**: en Query som försöker identifiera poster som inte uppfyller ett visst behov.

Samtidigt är det inte självklart att varje kund måste ha e-postadress. För en kund som handlar över disk kanske informationen inte behövs.

Fullständighet måste därför alltid bedömas **i relation till ett ändamål**.

## Giltighet – att följa en regel

Anta att vi vill hitta produkter med orimliga lagersaldon:

```
SELECT id, title, stock
FROM products
WHERE stock < 0;
```

Om modellen har en fungerande `CHECK (stock >= 0)` bör den här Queryn inte hitta några sådana rader.

Men kontrollen kan ändå vara användbar när vi arbetar med äldre data, importerar data från andra system eller undersöker en miljö där samma begränsning inte har funnits hela tiden.

Att en kontroll ger **noll träffar** visar att just den regeln inte brutits i de undersökta raderna. Det bevisar inte att all data är korrekt.

## Unikhet – när två poster beskriver samma sak

Vi har en primärnyckel `id` som garanterar att två kunder inte får samma identifierare.

Men det garanterar inte att samma person inte registrerats två gånger:

| id | name | email |
|---|---|---|
| 1 | Benjamin | benjamin@example.com |
| 6 | Benjamin | benjamin@example.com |

Vi kan leta efter upprepade e-postadresser:

```
SELECT
    email,
    COUNT(*) AS occurrences
FROM customers
WHERE email IS NOT NULL
GROUP BY email
HAVING COUNT(*) > 1;
```

Detta kan indikera dubbletter. Men det är inte ett säkert bevis för att två rader beskriver samma person. Flera personer kan i vissa verksamheter dela en e-postadress, och en person kan ha flera adresser.

**Vi måste först bestämma vad det innebär att två poster är samma företeelse.**

### Konsekvens – när uppgifter motsäger varandra

Anta att bokhandelns administrativa system och webbplatsen har var sin prislista.

Om samma produkt anges med två olika priser kanske vi har ett kvalitetsproblem.

Men först behöver vi fråga om priserna verkligen ska vara identiska. Det kan finnas kampanjer, olika valutor eller avtalade kundpriser.

Konsekvens betyder därför inte att samma text eller tal alltid måste förekomma överallt, utan att informationen **följer de samband och regler som faktiskt gäller**.

Normalisering och främmande nycklar kan förebygga vissa problem genom att minska onödig dubbellagring. Men de kan inte ensamma avgöra om två verksamhetssystem ger samma bild av verkligheten.

## Avancerat

### Att mäta datakvalitet med SQL

Det kan vara användbart att uttrycka en kvalitetskontroll som ett mått.

Anta att vi vill mäta hur stor andel av kunderna som har en registrerad, icke-tom e-postadress:

```
SELECT
    COUNT(*) AS total_customers,
    SUM(
        CASE
            WHEN email IS NOT NULL
             AND TRIM(email) <> ''
            THEN 1
            ELSE 0
        END
    ) AS customers_with_email
FROM customers;
```

Vi kan också beräkna en andel:

```
SELECT
    100.0 * SUM(
        CASE
            WHEN email IS NOT NULL
             AND TRIM(email) <> ''
            THEN 1
            ELSE 0
        END
    ) / NULLIF(COUNT(*), 0) AS email_completeness_percent
FROM customers;
```

`NULLIF(COUNT(*), 0)` undviker division med noll om tabellen är tom.

Vi har nu gjort en kvalitetsdimension **mätbar**, men måttet behöver fortfarande tolkas. Om e-post inte krävs för alla kunder kan 100 procent vara ett olämpligt mål.

## Korrekthet behöver ofta en referens

Vissa fel kan vi inte upptäcka enbart genom att studera samma tabell.

Anta att `products.price` innehåller 200 kronor. Är det rätt pris?

Vi kan inte avgöra det utan att känna till vad vi jämför med.

Vi kanske har en kontrollerad prislista från inköps- eller affärssystemet. Då kan vi jämföra registrerade priser med referensvärden.

Det kan uttryckas med en Query som kombinerar två tabeller:

```
SELECT
    p.id,
    p.title,
    p.price AS registered_price,
    r.approved_price
FROM products AS p
INNER JOIN approved_prices AS r
    ON r.product_id = p.id
WHERE p.price <> r.approved_price;
```

Här är `approved_prices` en **tänkt referenstabell**, inte en del av vår befintliga bokhandelsmodell.

Även den tillförlitliga källan kan förstås innehålla fel eller ha en annan uppdateringstidpunkt. Därför behöver vi veta **vilket system eller beslut som är auktoritativt för respektive uppgift**.

## Aktualitet och tidens betydelse

Vi kan ha en korrekt adress som ändå är för gammal för att användas.

Det innebär att **korrekthet vid registreringstillfället** inte alltid är samma sak som **aktualitet vid användningstillfället**.

Vi kan därför komplettera vissa uppgifter med metadata om när de senast kontrollerades eller uppdaterades:

```
customer_id | address | verified_at
------------+---------+--------------------
          2 | ...     | 2026-09-15 09:30:00
```

Det är ett konceptuellt exempel, inte en ändring vi redan gjort i `addresses`.

En Query kan identifiera uppgifter som behöver kontrolleras på nytt. Men ett sent datum innebär inte att uppgiften är korrekt, och ett gammalt datum innebär inte automatiskt att den är felaktig.

Aktualitet handlar om **hur snabbt verkligheten kan förändras och hur känslig användningen är för förändringen**.

## Flera dimensioner kan stå i konflikt

Vi kanske kan förbättra fullständigheten genom att kräva att alla kunder fyller i fler fält.

Men fler obligatoriska fält kan också leda till att användare fyller i godtyckliga värden för att komma vidare.

Vi kan få **mer data**, utan att få **bättre data**.

På samma sätt kan omfattande validering ge bättre skydd mot vissa fel men också hindra legitima uppgifter som inte passar ett alltför snävt format.

Kvalitetsarbete innebär därför att vi behöver värdera flera dimensioner tillsammans, utifrån verksamhetens behov.

## Förebygga, upptäcka och rätta

Vi kan skilja mellan tre slags kvalitetsarbete:

**Förebygga:** Använd datatyper, nycklar, begränsningar och välformulerade processer för inmatning.

**Upptäcka:** Kör kvalitetskontroller som hittar saknade värden, misstänkta dubbletter, brutna regler och oväntade förändringar.

**Rätta:** Undersök orsakerna och korrigera data utifrån en tillförlitlig källa eller verksamhetsregel.

Det är frestande att automatiskt ändra alla värden som avviker. Men en avvikelse är inte alltid ett fel.

Vi bör därför hålla isär **upptäckt** och **åtgärd**, särskilt när många poster kan påverkas.

## Datakvalitet i hela informationsflödet

Anta att en leverantör skickar en fil med produktuppgifter.

Vi importerar filen, uppdaterar bokhandelns databas och publicerar sedan produkterna på webbplatsen.

Fel kan uppstå i flera led:

- Uppgifterna kan vara fel redan hos leverantören.
- Importen kan tolka ett tal eller datum på fel sätt.
- Två produktidentifierare kan blandas ihop.
- Applikationen kan visa en gammal version av informationen.

Därför räcker det inte alltid att köra en kontroll i den slutliga databastabellen.

Vi behöver förstå **var data kommer ifrån, vilka förändringar de genomgått och vilka system som ansvarar för dem**.

Detta kallas ofta *data lineage* – informationens ursprung och väg genom olika bearbetningar.

## Tillförlitlighet och spårbarhet

När vi hittar en orimlig uppgift behöver vi kunna besvara frågor som:

Vem eller vilket system registrerade den? När ändrades den senast? Vilket underlag användes? Kan vi se det tidigare värdet?

Det kan kräva historik, loggning och metadata, men också rutiner för hur avvikelser rapporteras och beslutas.

I säkerhetskapitlet diskuterade vi loggning som ett sätt att upptäcka obehöriga förändringar. Här får spårbarhet ytterligare en betydelse: **att kunna förstå och förbättra informationens kvalitet**.

## Var hör kvalitetsreglerna hemma?

Vi har flera möjligheter:

| Plats | Exempel |
|---|---|
| **Databasens struktur** | `NOT NULL`, `CHECK`, `UNIQUE` och främmande nycklar |
| **Applikationen** | Hjälp vid inmatning och kontroller av verksamhetens arbetsflöde |
| **Datapipelines och importer** | Kontroller av format, förväntade värden och avvikande mängder |
| **Återkommande analyser** | Queries och rapporter som upptäcker kvalitetsproblem över tid |

En regel som alltid måste gälla bör, när det är möjligt och lämpligt, också skyddas av databasen.

Andra regler behöver mer kontext än vad en enskild tabell känner till. De kan vara beroende av andra system, avtal eller verksamhetsbeslut.

Att samma kvalitetsprincip kontrolleras på flera ställen behöver inte vara onödig dubblering. Kontrollerna kan ha **olika syften**.

## Att ett värde finns är inte samma sak som att vi vet något

Vi kan fylla varje kolumn i bokhandelns databas, sätta primärnycklar på varje tabell och få alla integritetskontroller att passera.

Ändå kan vi sakna tillförlitlig kunskap om kunderna, produkterna och beställningarna.

En databas kan garantera att värden har en viss **struktur**. Den kan också upprätthålla regler för vilka relationer och kombinationer som är tillåtna.

Men den kan inte ensam garantera att våra uppgifter är **sanna, aktuella och relevanta för det vi vill förstå eller göra**.

Det kräver att vi kopplar data till verksamhetens betydelser, källor och ansvar.

## Reflektion

1. Hur kan ett värde vara giltigt enligt databasens regler men ändå ha låg datakvalitet?
2. När är två likadana kundposter en oönskad dubblett, och när kan de beskriva olika saker?
3. Varför behöver vi förstå både informationens ursprung och dess användningsområde för att bedöma hur tillförlitlig den är?

# 25. SQL för analys och dataplattformar

När en kund köper en bok behöver bokhandelns system genomföra beställningen korrekt. Det ska kontrollera lagret, registrera ordern och bevara priset kunden betalade.

Men bokhandeln behöver också kunna ställa andra frågor: Vilka böcker säljer bäst? Hur utvecklas omsättningen från månad till månad? Vilka produkttyper växer snabbast?

Det är fortfarande samma verksamhet vi vill förstå, men nu använder vi informationen för ett annat syfte.

Vi går från **att registrera vad som händer** till **att undersöka vad som har hänt**.

## Grund

### Operativ data och analytisk data

Vår bokhandel har nu bland annat dessa tabeller:

- `customers` – kunder.
- `products` och `product_types` – sortiment och produkttyper.
- `orders` – beställningar.
- `order_items` – produkter och priser vid en beställning.

Tabellerna är utformade för att representera verksamhetens företeelser och relationer. När någon gör ett köp vill vi skapa och uppdatera ett litet antal poster på ett tillförlitligt sätt.

Detta brukar kallas **transaktionsorienterad datahantering**, ofta förkortat **OLTP** (*Online Transaction Processing*).

När vi i stället analyserar många beställningar samtidigt talar vi ofta om **OLAP** (*Online Analytical Processing*).

| OLTP – operativt arbete | OLAP – analys |
|---|---|
| Registrera ett köp | Summera försäljning per månad |
| Ändra en kundadress | Undersöka geografiska mönster |
| Uppdatera lagersaldo | Jämföra omsättning per produkttyp |
| Många små förändringar | Frågor över stora mängder historiska data |

Detta är olika **användningsmönster**, inte två helt åtskilda former av SQL. En mindre verksamhet kan utföra både operativa och analytiska Queries i samma databas.

## Från orderrader till omsättning

Vi har tidigare introducerat en förenklad tabell `order_items`:

| id | order_id | product_id | quantity | unit_price |
|---|---|---|---|---|
| 1001 | 101 | 1 | 2 | 200.00 |
| 1002 | 102 | 4 | 1 | 120.00 |
| 1003 | 102 | 5 | 2 | 180.00 |

Här beskriver `unit_price` priset **vid beställningen**, inte produktens aktuella pris.

Vi kan beräkna orderradernas försäljningsvärde:

```
SELECT
    order_id,
    product_id,
    quantity * unit_price AS line_amount
FROM order_items;
```

| order_id | product_id | line_amount |
|---|---|---|
| 101 | 1 | 400.00 |

| order_id | product_id | line_amount |
|---|---|---|
| 102 | 4 | 120.00 |
| 102 | 5 | 360.00 |

Om vi vill summera per order använder vi `GROUP BY`:

```
SELECT
    order_id,
    SUM(quantity * unit_price) AS order_amount
FROM order_items
GROUP BY order_id;
```

Vi återanvänder alltså SQL vi redan kan. Skillnaden ligger i **vilket perspektiv på informationen vi söker**.

Beräkningen är förenklad. En verklig omsättningsmodell behöver också ta ställning till exempelvis rabatter, moms, returer och orderstatus. Vi kan inte avgöra vad som ska räknas som omsättning enbart utifrån dessa tre kolumner.

## Analys över tid

Vi kan kombinera `orders` och `order_items` för att undersöka beställningsvärden per månad.

I MySQL kan vi skriva:

```
SELECT
    DATE_FORMAT(o.ordered_at, '%Y-%m') AS order_month,
    SUM(oi.quantity * oi.unit_price) AS ordered_value
FROM orders AS o
INNER JOIN order_items AS oi
    ON oi.order_id = o.id
GROUP BY DATE_FORMAT(o.ordered_at, '%Y-%m')
ORDER BY order_month;
```

Här grupperar vi beställningarna efter månad.

Det är viktigt att kalla måttet **beställt värde**, inte automatiskt bokförd omsättning. En order kan ha avbrutits eller returnerats och vi har ännu inte modellerat dessa händelser.

Funktionen `DATE_FORMAT()` hör till MySQL. Andra SQL-system använder exempelvis `DATE_TRUNC()` eller andra datumfunktioner.[46]

## Varför en egen analysmiljö?

Med några tusen beställningar kanske vi kan analysera direkt i bokhandelns operativa databas.

Men om vi får mycket data, komplexa sammanställningar och många samtidiga användare kan analysen behöva skiljas från systemet som registrerar köp.

En stor Query kan kräva omfattande skanningar, sorteringar och grupperingar. Den kan konkurrera om resurser med operativa transaktioner.

Vi kan därför föra över information till en särskild **analysmiljö**.

En sådan miljö kan vara utformad för att läsa stora datamängder effektivt, bevara historik och sammanföra information från flera system.

## ETL och ELT

När data flyttas och bearbetas möter vi ofta två begrepp:

**ETL – Extract, Transform, Load**

Vi hämtar data från källsystemen, bearbetar dem och läser in resultatet i analysmiljön.

**ELT – Extract, Load, Transform**

Vi hämtar data, läser in dem i en dataplattform och genomför bearbetningarna där.

Båda metoderna kan använda SQL för transformationer.

Skillnaden är inte att den ena metoden alltid är modernare eller bättre. Det handlar bland annat om var bearbetningen utförs och vilka resurser och krav miljön har.

### Ett första steg i en analytisk modell

Vi kan skapa en återanvändbar sammanställning över försäljning per produkt:

```
CREATE VIEW product_sales AS
SELECT
    oi.product_id,
    SUM(oi.quantity) AS units_ordered,
    SUM(oi.quantity * oi.unit_price) AS ordered_value
FROM order_items AS oi
GROUP BY oi.product_id;
```

Vyn ger oss ett **analytiskt perspektiv** på orderraderna.

Vi kan sedan kombinera den med `products` för att visa produktnamn:

```
SELECT
    p.title,
    s.units_ordered,
    s.ordered_value
FROM product_sales AS s
INNER JOIN products AS p
    ON p.id = s.product_id
ORDER BY s.ordered_value DESC;
```

Precis som i kapitel 17 kan en View skapa ett logiskt gränssnitt mot data. Men här ser vi också dess begränsning: resultatet är beroende av hur våra transaktionstabeller ser ut och vilka historiska uppgifter de faktiskt innehåller.

## Avancerat

### Dataplattformar och olika lagringsformer

En analysmiljö kan byggas på flera sätt.

Ett **datalager** (*data warehouse*) är vanligtvis organiserat för analytiska Queries, sammanställningar och rapportering.

En **datasjö** (*data lake*) kan samla stora mängder data i olika format, ofta inklusive filer som ännu inte har omvandlats till en gemensam relationsmodell.

En **lakehouse-arkitektur** försöker kombinera flexibla filbaserade lagringsformer med funktioner som gör informationen mer tillförlitlig och användbar för analys.

Dessa begrepp beskriver olika arkitekturval. De är inte garantier för att data blir korrekt eller att en Query går snabbt.

## Stjärnschema – en modell för analys

I en operativ databas organiserar vi ofta tabeller utifrån verksamhetens företeelser och deras relationer.

I en analytisk modell kan vi i stället utgå från **vad vi vill mäta**.

Tänk att vi vill analysera försäljningen.

Då kan vi ha en **faktatabell** med försäljningens mätvärden, exempelvis antal och orderradsvärde:

**fact_sales**

| order_item_id | date_id | product_id | quantity | ordered_value |
|---|---|---|---|---|
| 1001 | 20261012 | 1 | 2 | 400.00 |
| 1002 | 20261012 | 4 | 1 | 120.00 |

Och **dimensionstabeller** som beskriver produkten och datumet:

- `dim_product` – produktnamn, produkttyp och andra produktegenskaper.
- `dim_date` – datum, månad, kvartal och år.

Det brukar kallas ett **stjärnschema**.

Faktatabellens rader innehåller sådant vi vill mäta, medan dimensionerna ger oss olika sätt att gruppera och tolka mätvärdena.

Detta är en **alternativ analytisk modell**, inte nya tabeller vi lägger till direkt i vår operativa bokhandel.

## Vad representerar en rad?

När vi bygger en faktatabell behöver vi bestämma dess **kornighet** (*grain*): vad varje enskild rad representerar.

Ska en rad beskriva:

- En orderrad?
- En hel beställning?
- Försäljningen av en viss produkt under en viss dag?

Alla tre är möjliga modeller, men de ger olika beräkningsmöjligheter.

Om varje rad representerar en orderrad kan vi summera upp till ordernivå eller månadsnivå.

Om vi bara sparar totalsummor per månad förlorar vi möjligheten att senare undersöka enskilda orderrader utifrån samma tabell.

**Kornighet är alltså ett beslut om vilken detaljnivå vi bevarar**, inte bara ett val av antal kolumner.

## Egen data och främmande data

Vi har tidigare diskuterat att viss data beskriver den egna verksamhetens händelser, medan annan data kommer från externa källor.

Bokhandelns orderrader är ett exempel på information om egna transaktioner.

En förlagskatalog med författare, ISBN och boktitlar kan däremot vara information vi tar emot utifrån.

När vi sammanför dem behöver vi identifiera vilka uppgifter som ska betraktas som **auktoritativa**.

Om förlaget ändrar stavningen av en titel kanske vi bör uppdatera kataloginformationen. Men ett gammalt orderpris bör inte automatiskt ändras när katalogens aktuella produktpris förändras.

Analysmodellen behöver alltså bevara **betydelsen och ursprunget** för uppgifterna, även när de förs samman.

## Att bevara historiska förändringar

Anta att en bok i oktober kategoriseras som `Roman` men senare flyttas till en annan produkttyp.

När vi analyserar försäljningen i oktober behöver vi bestämma om den ska visas enligt:

- Produktens **nuvarande** kategori.
- Den kategori som gällde **när försäljningen skedde**.

Båda kan vara relevanta, men de besvarar olika frågor.

I datalager kan man hantera sådana förändringar med olika strategier för **långsamt föränderliga dimensioner**, ofta kallade *Slowly Changing Dimensions* (SCD).

En enkel strategi är att ersätta det gamla värdet. En annan är att bevara flera versioner med giltighetsperioder.

Det här är ett exempel på hur **tid blir en del av datamodellens betydelse**.

## Transformationer behöver kvalitetskontroller

Vi kan inte utgå från att en datapipeline producerar korrekta data bara för att varje SQL-sats körts utan fel.

Exempel på relevanta kontroller är:

- Har alla orderrader fått en giltig produktreferens?
- Har vi råkat läsa in samma händelser flera gånger?
- Stämmer antalet rader och summan av relevanta mätvärden mellan källa och mål?
- Saknas datum eller viktiga dimensioner?

Vi behöver också veta hur vi ska hantera sena uppdateringar, rättelser och återkörningar.

En transformation bör om möjligt vara **idempotent**: om samma inläsning upprepas av misstag ska den inte skapa dubbla försäljningshändelser.

Hur vi uppnår det kan bero på stabila händelseidentifierare, unika nycklar, inläsningslogik och transaktioner.

## Att mäta utan att missförstå

Anta att vi tar fram en rapport över de produkter som har högst försäljningsvärde.

Det låter enkelt, men först måste vi bestämma vad resultatet betyder.

Ska vi räkna:

- Lagda beställningar eller genomförda försäljningar?
- Priser före eller efter rabatt?

- Värden inklusive eller exklusive moms?
- Returer som negativa försäljningar eller som separata händelser?

SQL kan summera de värden vi har lagrat. Men SQL kan inte avgöra **vilken verksamhetsdefinition vi borde använda**.

Här möts datakvalitet, datamodellering och analys.

## SQL som gemensamt språk

SQL är inte bara användbart för små transaktionstabeller.

Samma grundläggande idéer – urval, filtrering, JOIN, aggregering, CTE:er och fönsterfunktioner – återkommer i många analysplattformar.

Olika system har olika SQL-dialekter och olika sätt att hantera stora datamängder. Men principen att beskriva **vilket resultat vi vill få** i stället för exakt hur det ska beräknas är fortfarande central.

Vi kan alltså använda kunskaper från de tidigare kapitlen i ett större sammanhang, även när lagringen inte längre sker i en enda traditionell SQL-databas.

## Från händelser till kunskap

Bokhandelns operativa databas hjälper oss att veta **vad som hände i en viss beställning**.

En analysmodell kan hjälpa oss att förstå **hur många liknande händelser som skett**, hur de förändras över tid och hur de förhåller sig till andra delar av verksamheten.

Men mer data ger inte automatiskt mer kunskap.

När vi sammanställer information behöver vi hela tiden förstå vad varje rad representerar, var uppgifterna kommer ifrån och vilka antaganden våra beräkningar bygger på.

En bra dataplattform är därför inte bara en plats där vi samlar data. Den behöver också hjälpa oss att bevara och förklara **informationens betydelse när vi använder den på nya sätt**.

## Reflektion

1. Vad är skillnaden mellan att registrera en beställning och att använda samma beställning som underlag för analys?
2. Varför behöver vi bestämma vad en rad i en faktatabell representerar innan vi kan avgöra vilka analyser som är möjliga?
3. Hur kan två rapporter över samma försäljning ge olika men ändå rimliga resultat beroende på hur vi definierar måttet?

# 26. Strukturerade och semistrukturerade data

I bokhandelns databas har vi hittills lagt information i tabeller med bestämda kolumner. En produkt har ett namn, ett pris och ett lagersaldo. En författare har sin egen rad, och relationen mellan produkter och författare representeras i en särskild kopplingstabell.

Men tänk att böckerna också har olika slags beskrivande egenskaper. En roman kanske har ett originalspråk och en serie den tillhör. En fackbok kan ha ett ämnesområde och en upplagebeteckning.

Ska vi lägga till en ny kolumn för varje tänkbar egenskap? Eller finns det information som passar bättre i en mer flexibel struktur?

Det leder oss till skillnaden mellan **strukturerade** och **semistrukturerade data** – och till hur SQL-system kan arbeta med båda.

## Grund

### När strukturen är given

Tabellen `products` kan beskrivas med en tydlig struktur:

| id | title | price | stock | product_type_id |
|---|---|---|---|---|
| 1 | Vägen hem | 200.00 | 12 | 1 |
| 2 | Den blå staden | 150.00 | 0 | 1 |
| 3 | Vinterljus | 249.00 | 4 | 1 |

Varje rad använder samma kolumner. Databasen känner till deras datatyper och kan upprätthålla regler för vilka värden som tillåts.

Det är **strukturerade data**: informationen följer en uttrycklig modell.

Strukturerade data behöver inte vara enkla. Vi kan representera ganska komplicerade företeelser med flera relaterade tabeller.

## När olika poster har olika egenskaper

Anta att vi vill beskriva mer om varje bok:

```
{
  "language": "sv",
  "pages": 284,
  "series": "Vägarna"
}
```

En annan bok kan ha andra uppgifter:

```
{
  "language": "en",
  "edition": 3,
  "subjects": ["databaser", "programmering"]
}
```

Detta är **JSON** (*JavaScript Object Notation*), ett vanligt format för att representera information med namngivna egenskaper och samlingar.

Båda objekten är giltiga JSON, trots att de inte innehåller samma egenskaper.

Det är en del av vad vi menar med **semistrukturerade data**: informationen har en intern struktur, men behöver inte följa ett och samma fasta schema för varje post.

## Vad innehåller ett JSON-värde?

JSON använder några grundläggande typer:

| Form | Exempel |
|---|---|
| Objekt | `{"language": "sv"}` |

| Form | Exempel |
|---|---|
| Array | `["databaser", "SQL"]` |
| Text | `"Vägen hem"` |
| Tal | `249.50` |
| Boolean | `true` eller `false` |
| Null | `null` |

Objekt innehåller egenskaper med namn. Arrayer innehåller värden i en bestämd ordning och kan i sin tur innehålla objekt eller andra arrayer.

Det betyder att ett JSON-värde kan beskriva **en sammansatt struktur inuti ett enda värde**.

## Att lagra JSON i en SQL-tabell

Vi kan tänka oss att bokhandeln kompletterar produkterna med en kolumn för ytterligare metadata.

I MySQL kan vi skriva:

```
ALTER TABLE products
ADD COLUMN metadata JSON;
```

Och fylla i uppgifter för en produkt:

```
UPDATE products
SET metadata = '{
    "language": "sv",
    "pages": 284,
    "series": "Vägarna"
}'
WHERE id = 1;
```

Produkten har fortfarande sitt `id`, `title`, `price` och `stock` som vanliga kolumner.

Men den kan också innehålla ett JSON-värde med fler egenskaper.

MySQL har en särskild JSON-datatyp. Andra databassystem erbjuder andra varianter av stöd för JSON.[47]

### Att läsa en egenskap från JSON

I MySQL kan vi använda `JSON_EXTRACT()`:

```
SELECT
    title,
    JSON_EXTRACT(metadata, '$.language') AS language
FROM products;
```

Uttrycket `'$.language'` är en **JSON-sökväg** (*JSON path*).

Tecknet `$` representerar hela JSON-värdet och `.language` anger egenskapen vi vill läsa.

Vi kan också använda `JSON_UNQUOTE()` för att få ut textvärdet utan JSON-strängens citattecken:

```
SELECT
    title,
    JSON_UNQUOTE(
        JSON_EXTRACT(metadata, '$.language')
    ) AS language
FROM products;
```

På så sätt kan vi använda en del av den semistrukturerade informationen i en vanlig SQL-Query.

### Att filtrera på en JSON-egenskap

Vi kan också välja de produkter vars metadata anger svenska som språk:

```
SELECT id, title
FROM products
WHERE JSON_UNQUOTE(
    JSON_EXTRACT(metadata, '$.language')
) = 'sv';
```

Vi har alltså fortfarande en relationell tabell och en vanlig `SELECT`-sats. Det nya är att ett av värdena i raden i sin tur innehåller en struktur som vi kan söka i.

## Avancerat

### JSON är inte en ersättning för alla relationer

Det vore möjligt att lagra en produkts författare så här:

```
{
  "authors": [
    {"id": 1, "name": "Anna Svensson"},
    {"id": 2, "name": "Erik Lund"}
  ]
}
```

Men vi har redan tabellerna `authors` och `product_authors`.

Varför valde vi dem?

Därför att en författare är en egen företeelse, som kan förekomma i relation till flera produkter. Vi vill kunna identifiera författaren, kontrollera relationerna och ändra information på ett ställe.

Om vi kopierar författarens namn in i varje produkts JSON-värde riskerar vi samma problem med redundans och inkonsekvens som vi diskuterade i normaliseringskapitlet.

**JSON är ett alternativt sätt att representera en struktur, inte en anledning att överge datamodellering.**

### När JSON kan vara lämpligt

JSON kan vara användbart när:

- En uppgift har en tydlig intern struktur, men olika poster kan ha olika egenskaper.

- Informationen kommer från ett externt system och vi behöver bevara dess ursprungliga representation.
- Vi behöver lagra kompletterande metadata som inte används som centrala relationer i vår datamodell.
- En viss sammansatt uppgift naturligt hör ihop och vanligtvis läses som en helhet.

Samtidigt bör vi vara försiktiga om egenskaperna behöver starka databasbegränsningar, frekvent filtrering, uppdatering var för sig eller relationer till andra tabeller.

Ett bra beslut bygger på **hur informationen används och vilka regler den måste följa**.

## Schema finns även när det inte står i tabellens kolumner

Ett JSON-fält kan se flexibelt ut, men applikationen förväntar sig ofta ändå vissa egenskaper och datatyper.

Kanske förutsätter vi att:

```
{
  "language": "sv",
  "pages": 284
}
```

alltid har en `language` som text och en `pages` som ett positivt heltal.

Då har vi fortfarande ett **schema** – men reglerna kanske finns i applikationskod, i en JSON Schema-definition eller i databasens särskilda funktioner för validering.

Frågan blir därför inte om data har struktur eller saknar struktur, utan **var strukturen beskrivs och kontrolleras**.

MySQL erbjuder exempelvis `JSON_SCHEMA_VALID()` för att pröva JSON mot ett schema, men motsvarande funktioner och möjligheter varierar mellan databassystem och versioner.[48]

## Saknade egenskaper, JSON null och SQL NULL

Här behöver vi vara särskilt noggranna.

Det finns en skillnad mellan:

1. Att hela SQL-kolumnen `metadata` innehåller `NULL`.
2. Att JSON-objektet saknar egenskapen `language`.
3. Att JSON-objektet uttryckligen innehåller `"language": null`.

Dessa situationer kan ha olika betydelse för verksamheten.

Databassystemens JSON-funktioner kan dessutom representera och jämföra dem på olika sätt.

Vi bör därför inte automatiskt behandla **saknad egenskap**, **JSON null** och **SQL NULL** som samma sak. Om skillnaden spelar roll måste vi välja rätt funktion och kontrollera resultatet i den databas vi använder.

## Arrayer och upprepade värden

JSON-arrayer är användbara för värden som hör ihop i en samling:

```
{
  "subjects": ["databaser", "programmering", "SQL"]
}
```

Men vi behöver fundera över vad arrayen representerar.

Om `subjects` bara är beskrivande metadata kanske det fungerar bra.

Om ämnen däremot är ett kontrollerat register med egna identifierare, regler och relationer till många produkter kan en separat tabell för ämnen och en kopplingstabell vara en bättre modell.

Det anknyter till vår diskussion om första normalformen: frågan om **vad som räknas som ett enskilt värde** är beroende av hur informationen ska representeras och användas.

JSON i en kolumn är inte automatiskt ett brott mot normalisering. Men JSON gör det möjligt att gömma sådana problem om vi slutar undersöka vad de inre värdena betyder.

## När JSON ska bli relationella rader

Anta att vi tar emot en produktkatalog från ett externt API.

Varje produkt beskrivs med JSON, men vår operativa bokhandel behöver fortfarande kunna söka, prissätta och sälja produkter enligt sin datamodell.

Då kan vi behöva:

1. Läsa in och bevara inkommande JSON.
2. Kontrollera att värden och egenskaper är giltiga.
3. Identifiera produkter och relationer.
4. Transformera relevanta uppgifter till våra relationella tabeller.
5. Dokumentera vilken källa uppgifterna kom från.

Det är en form av **ETL eller ELT**, som vi mötte i föregående kapitel.

Vi behöver inte välja mellan att *endast* lagra rå JSON och att *endast* lagra normaliserade data. I en dataplattform kan olika representationer fylla olika syften.

## Att uppdatera delar av ett JSON-värde

I MySQL kan vi använda `JSON_SET()` för att ändra en egenskap:

```
UPDATE products
SET metadata = JSON_SET(
    metadata,
    '$.pages',
    286
```

```
)
WHERE id = 1;
```

Denna Query förutsätter att `metadata` innehåller ett JSON-värde, exempelvis det objekt vi lade in tidigare.

Det kan vara praktiskt när en enskild egenskap behöver ändras.

Men det betyder inte att ändringen nödvändigtvis är billig eller sker utan att större delar av värdet bearbetas. Databassystemet avgör hur JSON lagras och uppdateras internt.

## Index och prestanda för JSON

Om vi ofta filtrerar på `language` kanske vi vill kunna indexera den egenskapen.

Ett vanligt index på en JSON-kolumn fungerar inte alltid som ett index på ett vanligt textattribut.

Beroende på databassystem kan vi exempelvis använda **genererade kolumner**, uttrycksindex eller särskilda index för JSON-data.

I MySQL kan vi illustrera principen med en genererad kolumn:

```
ALTER TABLE products
ADD COLUMN language_code VARCHAR(10)
GENERATED ALWAYS AS (
    JSON_UNQUOTE(
        JSON_EXTRACT(metadata, '$.language')
    )
) STORED;
```

Därefter kan vi indexera den genererade kolumnen:

```
CREATE INDEX idx_products_language
ON products (language_code);
```

Exemplet är inte en generell rekommendation att kopiera varje JSON-egenskap till en egen kolumn. Tvärtom visar det att **vilka**

**egenskaper vi ofta söker på kan påverka hur vi väljer att modellera och indexera informationen**.

## JSON, XML och andra representationer

JSON är bara ett exempel på semistrukturerade data.

**XML** använder element och attribut och har möjligheter att beskriva strukturer med exempelvis XML Schema.

Andra system använder exempelvis dokumentformat, hierarkiska representationer eller binära format för utbyte och lagring.

Vi behöver därför skilja mellan:

- **Hur informationen representeras**, till exempel som tabellrader eller JSON.
- **Vilka regler informationen följer**, till exempel relationer eller schemaspecifikationer.
- **Hur informationen lagras och görs sökbar**, vilket beror på databassystemets funktioner.

Det är möjligt för en relationsdatabas att lagra dokumentliknande information. Och det är möjligt för en dokumentdatabas att ställa höga krav på struktur.

Gränsen handlar inte bara om en filändelse eller vilken produkt vi valt.

## Flexibilitet kräver fortfarande beslut

En fast tabellstruktur tvingar oss ofta att tidigt bestämma vad en uppgift betyder.

JSON kan låta oss skjuta upp vissa beslut. Det kan vara en styrka när informationen varierar eller förändras snabbt.

Men beslutet försvinner inte.

Förr eller senare behöver vi veta vilka uppgifter som är obligatoriska, vilka värden vi kan lita på, hur vi identifierar samma företeelse över tid och hur olika delar av informationen hänger ihop.

**Semistrukturerade data är inte data utan betydelse. De är data där en del av strukturen uttrycks på ett annat sätt.**

## Reflektion

1. När är det lämpligt att lagra en egenskap i ett JSON-fält, och när bör den få en egen kolumn eller relationell tabell?
2. Vad är skillnaden mellan att ett JSON-värde är syntaktiskt giltigt och att informationen är korrekt för bokhandelns behov?
3. Hur påverkar valet mellan relationella tabeller och semistrukturerade värden möjligheten att kontrollera, söka och förändra informationen?

# 27. Geografiska data

Bokhandeln vet vilka kunder den har och vilka adresser de har registrerat. Men ibland behöver vi kunna besvara frågor som handlar om **var** något finns.

Vilken butik ligger närmast en kund? Hur långt är det mellan två leveransadresser? Vilka kunder bor inom ett visst område?

Hittills har vi främst arbetat med text, tal och datum. Nu ska vi se hur en SQL-databas också kan representera **platser, avstånd och geografiska samband**.

## Grund

### En adress är inte en koordinat

I vår tabell `addresses` har vi exempelvis uppgifter som:

| id | customer_id | street | postal_code | city |
|---|---|---|---|---|
| 1 | 1 | Hemmagatan 1 | 124 56 | Hemmavan |
| 2 | 1 | Storgatan 4 | 154 32 | Storfors |
| 3 | 2 | Parkvägen 2 | 154 32 | Storfors |
| 4 | 3 | NULL | NULL | Malmö |

En adress är i första hand en **beskrivning som kan hjälpa oss att hitta en plats**.

Men stadens namn talar inte om exakt var något ligger. Och samma adress kan ibland avse flera byggnader, entréer eller leveranspunkter.

För att kunna beräkna geografiska avstånd behöver vi ofta en mer uttrycklig representation av platsens position.

## Latitud och longitud

En geografisk position kan anges med två koordinater:

- **Latitud** – läget norr eller söder om ekvatorn.
- **Longitud** – läget öster eller väster om en referensmeridian.

Malmö ligger ungefär vid latitud `55.605` och longitud `13.003`.

Vi skulle kunna lägga till två numeriska kolumner:

```
ALTER TABLE addresses
ADD COLUMN latitude DECIMAL(9, 6),
ADD COLUMN longitude DECIMAL(9, 6);
```

Det fungerar för att **lagra koordinatvärden**, men det innebär inte att databasen automatiskt förstår deras geografiska betydelse.

För geografiska beräkningar erbjuder många databassystem särskilda **spatiala datatyper och funktioner**.

## Punkter som en geografisk datatyp

En **punkt** (*Point*) representerar en position i ett koordinatsystem.

I MySQL 8 kan vi exempelvis konstruera en geografisk punkt med `ST_GeomFromText()`:

```
SELECT ST_GeomFromText(
    'POINT(13.003 55.605)',
    4326,
    'axis-order=long-lat'
);
```

Här anger vi longituden först och latituden därefter.

Talet `4326` är en **SRID** (*Spatial Reference System Identifier*) som identifierar koordinatreferenssystemet WGS 84, ett vanligt system för geografiska positioner.

Vi anger uttryckligen `axis-order=long-lat` för att undvika en vanlig förväxling kring koordinaternas ordning i MySQL.[49]

## Geografisk information i en tabell

Anta att bokhandeln får flera fysiska butiker.

Vi kan skapa en enkel tabell:

```
CREATE TABLE stores (
    id INTEGER PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    location POINT NOT NULL SRID 4326
);
```

Och lägga till en butik i Malmö:

```
INSERT INTO stores (id, name, location)
VALUES (
    1,
    'Bokhandeln Malmö',
    ST_GeomFromText(
        'POINT(13.003 55.605)',
        4326,
        'axis-order=long-lat'
    )
);
```

`POINT` och `SRID` gör platsens representation tydligare än två fristående tal. Samtidigt behöver vi fortfarande veta **vad punkten representerar**: byggnadens mittpunkt, entrén eller exempelvis platsen där varor lämnas av.

## Att beräkna avstånd

MySQL erbjuder `ST_Distance_Sphere()` för att beräkna ett ungefärligt avstånd längs jordytan mellan två geografiska punkter.

Vi kan exempelvis jämföra två illustrativa positioner:

```
SELECT ST_Distance_Sphere(
    ST_GeomFromText(
        'POINT(13.003 55.605)',
```

```
        4326,
        'axis-order=long-lat'
    ),
    ST_GeomFromText(
        'POINT(12.694 55.571)',
        4326,
        'axis-order=long-lat'
    )
) AS distance_meters;
```

Resultatet anges i meter.

Det är viktigt att förstå att detta är ett **geografiskt avstånd**, inte avståndet längs vägar eller den tid det tar att resa mellan platserna.

Beräkningen utgår från en sfärisk approximation av jorden. För vissa användningsområden behöver vi andra avståndsmodeller eller mer detaljerade kartdata.[50]

## Att söka efter närliggande platser

Vi vill kanske hitta butiker inom tio kilometer från en viss position.

I MySQL kan vi skriva:

```
SELECT
    name,
    ST_Distance_Sphere(
        location,
        ST_GeomFromText(
            'POINT(13.003 55.605)',
            4326,
            'axis-order=long-lat'
        )
    ) AS distance_meters
FROM stores
WHERE ST_Distance_Sphere(
    location,
    ST_GeomFromText(
        'POINT(13.003 55.605)',
        4326,
        'axis-order=long-lat'
    )
) <= 10000
ORDER BY distance_meters;
```

Nu kombinerar vi sådant vi redan kan: `SELECT`, `WHERE` och `ORDER BY` – men med en funktion som förstår något om **platser på jordytan**.

Queryn är pedagogiskt enkel. För en stor datamängd kan vi behöva utforma sökningen så att den också kan använda ett spatialt index.

## Avancerat

### Punkter, linjer och områden

Spatiala data är inte begränsade till punkter.

| Geometrityp | Beskriver exempelvis |
|---|---|
| **POINT** | Butikens position |
| **LINESTRING** | En väg eller rutt |
| **POLYGON** | Ett leveransområde |
| **MULTIPOINT** | En samling positioner |
| **MULTIPOLYGON** | Flera geografiskt åtskilda områden |

Vi kan representera en zon som en polygon i stället för att bara lagra namnet på ett område.

Men då behöver vi också välja hur gränsen ska definieras: administrativt område, postnummerområde eller den yta bokhandeln faktiskt levererar till.

Det är återigen ett **modelleringsbeslut**, inte bara ett val av datatyp.

### LINESTRING – en sträcka mellan flera punkter

En punkt anger en position. En **LINESTRING** beskriver i stället en linje som går genom en följd av punkter.

Anta att vi vill representera en sträcka mellan två platser. Den raka förbindelsen kan beskrivas med två koordinatpar:

```
SELECT ST_GeomFromText(
    'LINESTRING(0 0, 4 0)'
);
```

Men om en väg måste gå runt ett hinder kan den beskrivas med flera punkter:

```
SELECT ST_GeomFromText(
    'LINESTRING(0 0, 1 2, 3 2, 4 0)'
);
```

Vi kan föreställa oss skillnaden så här:

```
Rak förbindelse:

A ---------------------- B

Väg med omväg:

        P2 -------- P3
       /              \
      /                \
     A                  B
```

En `LINESTRING` består av raka delsträckor mellan koordinatparen. Den andra linjen blir därför längre än den raka förbindelsen mellan A och B.

Vi kan också låta databasen beräkna längden med `ST_Length()`:

```
SELECT
    ST_Length(
        ST_GeomFromText('LINESTRING(0 0, 4 0)')
    ) AS direct_length,
    ST_Length(
        ST_GeomFromText('LINESTRING(0 0, 1 2, 3 2, 4 0)')
    ) AS route_length;
```

Här använder vi ett enkelt **plant koordinatsystem utan angivet geografiskt SRID**, där koordinatenheterna är godtyckliga. Resultaten blir `4` respektive ungefär `6.47` längdenheter. Det är alltså inte kilometer eller meter.

För verkliga geografiska positioner behöver vi använda ett lämpligt koordinatreferenssystem och en längdberäkning som ger rätt måttenhet.

En viktig skillnad återstår: även om vi ritar en `LINESTRING` som följer en väg vet databasen inte automatiskt vilka vägar som faktiskt går att köra på, vilka som är enkelriktade eller hur korsningar hänger ihop. För att hitta en verklig **kortaste eller snabbaste rutt** behöver vi även ett vägnät och regler för hur det får användas.

## Ligger punkten inom området?

Anta att vi har tabellen `delivery_areas` med polygoner som beskriver våra leveransområden.

Vi kan undersöka om en butik ligger inom ett område:

```
SELECT s.name, d.name AS delivery_area
FROM stores AS s
INNER JOIN delivery_areas AS d
    ON ST_Contains(d.boundary, s.location);
```

Här förutsätter vi att `delivery_areas.boundary` har en kompatibel spatial datatyp och ett kompatibelt koordinatreferenssystem.

`ST_Contains()` undersöker ett **geometriskt förhållande** snarare än att jämföra text eller identifierare.

En viktig detalj är hur gränser behandlas: `ST_Contains()` räknar normalt inte en punkt som ligger exakt på polygonens gräns som en punkt inuti polygonen. Vid andra verksamhetsregler kan exempelvis `ST_Covers()` vara lämplig där den stöds.[51]

## Koordinatreferenssystem och projektioner

Vi har använt WGS 84 med SRID 4326.

Men det finns många koordinatreferenssystem. Vissa beskriver lägen på en ellipsoid med latitud och longitud. Andra använder en **kartprojektion** med plana koordinater, ofta uttryckta i meter.

För geografiska analyser i Sverige används bland annat SWEREF 99-varianter.

Valet av koordinatreferenssystem påverkar vad koordinatvärdena betyder och hur vi kan räkna på dem.

Det går därför inte att anta att två punkter med samma koordinatsiffror representerar samma plats om de använder olika referenssystem.

När vi kombinerar geografiska datamängder behöver vi ofta **transformera koordinaterna** till ett gemensamt system.

Och vi bör skilja på att *ange ett SRID* och att faktiskt *transformera koordinaterna*.

## Spatiala index

När vi i kapitel 21 arbetade med vanliga index såg vi hur databashanteraren kan hitta rader utan att läsa hela tabellen.

Geografiska data har ett liknande behov.

Anta att vi har hundratusentals butiker, utlämningsställen eller leveransadresser.

Om vi beräknar avståndet från en punkt till varje rad kan det bli dyrt.

Ett **spatialt index** hjälper databasen att hitta geografiskt relevanta kandidater.

I MySQL kan vi exempelvis skriva:

```
CREATE SPATIAL INDEX idx_stores_location
ON stores (location);
```

Det här förutsätter att kolumnens definition och lagringsmotor uppfyller kraven för spatial indexering. Vår tidigare definition med `NOT NULL SRID 4326` är avsedd för en sådan användning i MySQL 8.

Spatiala index använder andra principer än ett enkelt B-trädsindex på exempelvis `price`. Ofta arbetar de med geometriernas begränsande rektanglar (*bounding boxes*) för att hitta möjliga träffar.

En avståndsberäkning med `ST_Distance_Sphere()` blir dock **inte automatiskt indexerad** för att ett spatialt index finns. Ofta behöver vi först avgränsa ett sökområde med en indexvänlig spatial relation och sedan beräkna det exakta eller approximativa avståndet för kandidaterna.

Precis som tidigare behöver vi använda `EXPLAIN` och undersöka hur databashanteraren faktiskt kör vår Query.

## När en adress blir en punkt

Det är möjligt att omvandla en textadress till geografiska koordinater genom en process som brukar kallas **geokodning** (*geocoding*).

Men geokodning innebär ett tolkningssteg.

En adress kan saknas, vara felstavad, hänvisa till en hel gata eller bara kunna matchas ungefärligt.

Därför bör vi inte utan vidare ersätta den ursprungliga adressinformationen med en punkt.

I vissa system kan vi behöva spara både adress, koordinater och metadata om hur positionen fastställdes.

Det anknyter till vårt kapitel om datakvalitet: **ett tekniskt giltigt koordinatvärde är inte nödvändigtvis en korrekt eller tillräckligt noggrann position**.

## Plats och tid

Geografisk information förändras också över tid.

En butik kan flytta. En leveransadress kan ändras. Ett distributionsområde kan få en ny gräns.

Om vi vill analysera tidigare beställningar behöver vi fundera över om vi ska använda **den plats som gällde när beställningen gjordes** eller **den plats som gäller idag**.

Det påminner om vår tidigare diskussion om priser och historiska dimensioner.

En plats är alltså inte alltid en statisk egenskap. I vissa sammanhang behöver vi modellera ett geografiskt sakförhållande **tillsammans med dess giltighetstid**.

## Integritet och platsinformation

Detaljerad platsinformation kan vara känslig.

En kundadress kan vara nödvändig för att leverera en bok, men det betyder inte att alla som kan läsa produktinformationen också ska kunna läsa kundernas adresser.

Och om ett system lagrar återkommande positioner kan det ibland avslöja mer om en persons vanor än vad en enskild adress gör.

Därför behöver vi tillämpa de principer vi mötte i säkerhetskapitlet: behörighet, dataminimering, lämpligt skydd och tydliga användningsändamål.

### Vad betyder geografisk närhet?

Två positioner kan ligga nära varandra fågelvägen men vara långt ifrån varandra om vi måste följa vägnätet.

Ett område kan innehålla en adress geometriskt, men ändå inte omfattas av bokhandelns leveransvillkor.

Och två adresser i samma stad behöver inte vara mer lika ur leveranssynpunkt än adresser på var sin sida av en kommungräns.

Vi behöver därför skilja mellan **geometriska förhållanden** och **verksamhetens betydelse av dem**.

SQL kan hjälpa oss att beräkna avstånd och avgöra om en punkt ligger inom en polygon. Men den kan inte på egen hand bestämma vad som är ett rimligt leveransavstånd eller vilken plats som faktiskt är enklast att nå.

## Reflektion

1. Vad är skillnaden mellan en textadress, ett koordinatpar och en spatial punkt med ett angivet referenssystem?
2. Varför räcker det inte att skapa ett spatialt index för att garantera att alla geografiska Queries blir snabba?
3. Hur kan två geografiskt korrekta avstånd ge olika svar på frågan om vilken butik som ligger närmast en kund?

# 28. Att välja och förstå SQL-system

Vi har arbetat med SQL som ett språk för att beskriva, hämta och förändra information. Vi har skapat relationer, skrivit Queries och undersökt hur databasen upprätthåller regler och genomför transaktioner.

Men när vi installerar en databas installerar vi inte bara *SQL*. Vi väljer ett **databashanteringssystem** – en programvara som tolkar våra SQL-satser, lagrar data och ansvarar för hur de kan användas.

Det finns många sådana system. De har mycket gemensamt, men skiljer sig åt på sätt som påverkar både våra Queries och hur vi bygger applikationer.

Hur ska vi förstå skillnaderna utan att förlora det gemensamma?

## Grund

### SQL är ett språk – inte en databas

**SQL** (*Structured Query Language*) beskriver hur vi kan arbeta med relationella data.

**MySQL**, **PostgreSQL**, **SQLite** och **Microsoft SQL Server** är exempel på databashanteringssystem som använder SQL.

Vi kan jämföra skillnaden med ett språk och dess olika användningsmiljöer: två system kan förstå samma grundläggande SQL men ha olika utvidgningar, regler och sätt att arbeta.

En Query som:

```
SELECT title, price
FROM products
```

```
WHERE price > 200
ORDER BY price DESC;
```

fungerar i många SQL-system utan ändringar.

Men all SQL är inte lika portabel. Vi har redan mött exempel på skillnader i datumfunktioner, JSON-hantering och geografiska funktioner.

## SQL-standarden och dialekter

SQL har en internationell standard, som utvecklas över tid.

Standarden beskriver bland annat språkets grundläggande konstruktioner. Men den betyder inte att alla databassystem erbjuder exakt samma funktioner eller tolkar alla situationer på samma sätt.

En **SQL-dialekt** är ett visst systems sätt att stödja och ibland utöka SQL.

Vi har exempelvis använt:

```
SELECT *
FROM products
LIMIT 5 OFFSET 10;
```

Det här fungerar i MySQL, PostgreSQL och SQLite.

SQL-standardens sätt att uttrycka ett motsvarande urval kan i stället se ut så här:

```
SELECT *
FROM products
ORDER BY id
OFFSET 10 ROWS
FETCH NEXT 5 ROWS ONLY;
```

Men stödet för olika varianter skiljer sig. Därför behöver vi lära oss både **den gemensamma idén** och **den syntax som gäller där vi faktiskt arbetar**.

Vi bör dessutom ange `ORDER BY` när urvalet beror på vilka rader som kommer först. Utan uttrycklig sortering finns ingen garanterad ordning.

## Fyra vanliga SQL-system

| System | Typisk egenskap |
|---|---|
| **MySQL** | Vanligt i webbapplikationer och servermiljöer |
| **PostgreSQL** | Brett funktionsstöd och många möjligheter till utbyggnad |
| **SQLite** | Inbyggd databas som ofta används direkt av en applikation |
| **SQL Server** | Integrerat databassystem med omfattande drift- och administrationsfunktioner |

Det är exempel på användningsområden, **inte begränsningar**. Samma system kan användas i många olika sammanhang.

I vår bok har vi ofta använt MySQL som praktisk referens. Det innebär inte att MySQL alltid är det lämpligaste valet.

## Serverdatabas eller inbyggd databas

MySQL, PostgreSQL och SQL Server används ofta som **serverbaserade databashanterare**.

En databasserver kan ta emot anslutningar från flera program och användare. Den ansvarar för bland annat samtidighet, åtkomstkontroll och lagring.

SQLite används vanligtvis som en **inbyggd databas** i en applikation, där informationen lagras i en lokal databasfil.

Det betyder inte att SQLite saknar transaktioner eller relationsstöd. Men arkitekturen är annorlunda: vi har normalt ingen separat SQL-serverprocess som programmen ansluter till på samma sätt.

En lokal mobilapp och en stor webbhandel kan därför ha olika behov, även om de lagrar liknande information.

## Skillnader i syntax

Tänk att vi vill hämta de fem dyraste produkterna.

I MySQL, PostgreSQL och SQLite kan vi skriva:

```
SELECT title, price
FROM products
ORDER BY price DESC, id
LIMIT 5;
```

I SQL Server kan ett motsvarande exempel använda `TOP`:

```
SELECT TOP (5) title, price
FROM products
ORDER BY price DESC, id;
```

Vi försöker beskriva **samma informationsbehov**, men syntaxen skiljer sig.

Det är därför värdefullt att förstå `ORDER BY` och begränsning av rader som begrepp, i stället för att bara memorera en dialekts formuleringar.

## Datatyper är inte identiska överallt

Vi har arbetat med `INTEGER`, `DECIMAL`, `VARCHAR` och `DATE`.

De är vanliga datatyper, men detaljerna varierar.

Exempelvis har SQLite en särskild modell för typaffinitet och dynamisk typning, medan andra system i större utsträckning upprätthåller kolumnens deklarerade datatyp. SQLite kan också använda `STRICT`-tabeller för striktare typregler.

Även `BOOLEAN`, autogenererade identifierare och hur datum och tidszoner lagras skiljer sig mellan systemen.

Sådana skillnader påverkar inte bara **hur vi skriver SQL**, utan också **vilka garantier vi kan förvänta oss av datamodellen**.

## Avancerat

### Samma SQL kan ge olika praktiska förutsättningar

Att två databassystem accepterar samma Query betyder inte att de genomför den på samma sätt.

De kan ha olika strategier för index, optimering, transaktioner, låsning och radversionering.

Exempelvis kan:

- En viss `JOIN` planeras på olika sätt.
- Ett index byggas och användas annorlunda.
- Två samtidiga transaktioner påverka varandra på olika sätt.
- En schemaändring kräva olika mycket låsning eller driftplanering.

Därför är en Querys **logiska betydelse** inte samma sak som dess **faktiska exekvering**.

Vi mötte denna skillnad i kapitlen om transaktioner och Query-prestanda.

### Funktioner som utökar SQL

Databassystem kan erbjuda funktioner som går utöver den gemensamma SQL-kärnan.

Vi har redan sett exempel:

| Område | Exempel på skillnader |
|---|---|
| Datum och tid | `DATE_FORMAT()` i MySQL, `DATE_TRUNC()` i PostgreSQL |

| Område | Exempel på skillnader |
|---|---|
| JSON | Olika typer, operatorer och JSON-funktioner |
| Geografiska data | Olika spatiala funktioner och tillägg |
| Fulltextsökning | Olika index, språkstöd och söksyntax |
| Programlogik i databasen | Skillnader i procedurer, funktioner och triggers |

Ett exempel är **PostGIS**, en utvidgning till PostgreSQL med omfattande funktioner för geografiska data.

Vi ska alltså inte anta att varje SQL-system erbjuder alla tekniker vi sett i boken på exakt samma sätt.

## Vad kan vi flytta mellan system?

En grundläggande Query som använder `SELECT`, `WHERE` och `JOIN` kan ofta flyttas ganska enkelt.

Men en faktisk applikation är beroende av mer än enskilda Queries:

- Datatyper, standardvärden och begränsningar.
- Identifierare och regler för autogenererade nycklar.
- Index, Views och eventuella databasfunktioner.
- Transaktionernas beteende och isoleringsnivåer.
- Behörigheter, användarkonton och roller.
- Drivrutiner, anslutningssträngar och migrationsskript.

Att byta databashanterare är därför inte alltid bara att ersätta anslutningssträngen.

Ju mer vi använder ett visst systems specialfunktioner, desto viktigare blir det att förstå hur beroendet påverkar möjligheten att byta system senare.

Men att undvika *alla* systemspecifika funktioner kan också innebära att vi avstår från användbara möjligheter. Portabilitet är en avvägning, inte ett självändamål.

## Hur väljer vi ett SQL-system?

I stället för att fråga vilket system som är *bäst* kan vi fråga vilka krav vår tillämpning faktiskt har.

| Fråga | Varför den spelar roll |
|---|---|
| Var ska programmet köras? | En lokal applikation har andra driftbehov än ett publikt system |
| Hur många ska använda det samtidigt? | Samtidighet, anslutningar och skrivbelastning påverkar arkitekturvalet |
| Hur viktig är tillgängligheten? | Drift, säkerhetskopiering och återställning behöver planeras |
| Vilka datatyper och funktioner krävs? | JSON, geografiska data och andra specialfunktioner kan skilja sig |
| Vilken kompetens finns? | Ett system behöver kunna utvecklas, felsökas och förvaltas |
| Vilka tekniska beroenden har vi? | Drivrutiner, molntjänster och integrationer påverkar valet |

Krav som prestanda, kostnad och skalbarhet behöver bedömas i den **faktiska miljön**, inte enbart utifrån allmänna påståenden om olika produkter.

## Ett exempel: bokhandeln växer

Anta att bokhandeln börjar som en liten lokal applikation för att hålla reda på produkter och lager.

En inbyggd databas som SQLite skulle kunna vara lämplig om allt hanteras i en och samma applikation och miljö.

När bokhandeln senare får en webbplats, flera butiker och många samtidiga användare kan en serverbaserad databas bli mer praktisk.

Men det betyder inte att SQLite automatiskt är otillräckligt så snart flera personer använder systemet, eller att en större serverdatabas

alltid är bättre. Vi behöver undersöka arkitekturen och hur belastningen faktiskt ser ut.

Ett sådant byte påverkar också vårt arbetssätt. Vi behöver tänka på migrering av data, testning av Queries, behörigheter och hur programmen ansluter.

## Drift är en del av teknikvalet

Två system kan vara likvärdiga för de Queries vi skriver men ställa olika krav på drift.

Vi behöver ta ställning till exempelvis:

- Säkerhetskopiering och återställning.
- Övervakning, loggar och larm.
- Uppgraderingar och säkerhetsuppdateringar.
- Replikering och hantering av driftavbrott.
- Ansvar för servermiljö, åtkomst och tillgänglighet.

Det finns också hanterade databastjänster där en leverantör tar ansvar för delar av driften.

Det avlägsnar inte vårt ansvar för datamodell, behörigheter, kvalitetsregler och hur applikationen använder informationen.

## Standardisering och självständighet

En fördel med ett standardiserat språk är att vi kan ta med oss mycket av vår kunskap mellan olika system.

Men **standardisering av språk** är inte samma sak som **oberoende av teknikleverantörer**.

Vi kan använda SQL och ändå vara beroende av ett visst systems funktioner, verktyg eller driftsmiljö.

För vissa verksamheter är det viktigt att kunna flytta data och applikationer mellan miljöer. För andra väger integration, prestanda eller tillgång till viss funktionalitet tyngre.

Vi behöver därför kunna skilja mellan:

**Öppna principer** – idéer och standarder som kan användas i flera system.

**Systemspecifika möjligheter** – funktioner som gör ett visst system särskilt användbart.

**Praktisk flyttbarhet** – vad vi faktiskt kan överföra utan stora förändringar.

Det är tre relaterade men inte identiska saker.

### Att välja ett system är att välja förutsättningar

En databashanterare bestämmer inte vad en kund, produkt eller beställning betyder. Det är vårt arbete att modellera.

Men vårt val av databashanterare påverkar **hur modellen kan uttryckas, vilka regler som kan upprätthållas och vilka verktyg vi har för att arbeta med informationen**.

Det betyder att teknikvalet varken bör göras enbart utifrån favoritprodukter eller enbart utifrån en lista över SQL-funktioner.

Vi behöver förstå informationsbehovet, verksamhetens utveckling och vilka förutsättningar som måste finnas för att informationen ska förbli användbar över tid.

## Reflektion

1. Varför kan två databashanterare som båda använder SQL ändå kräva olika lösningar för samma applikation?

2. När är det rimligt att använda en systemspecifik funktion även om den gör det svårare att byta databashanterare?
3. Varför räcker det inte att jämföra SQL-syntax när vi ska välja vilket databassystem en verksamhet behöver?

# 29. Vad är en databas inte?

Vi började den här boken med en enkel uppgift: att hålla reda på information.

En kund har ett namn. En bok har en titel. En beställning hör till en kund och innehåller produkter. Genom att bestämma vad dessa uppgifter betyder och hur de hör ihop har vi byggt en databas som kan besvara allt mer avancerade frågor.

Vi har också sett hur en databas kan hjälpa oss att upprätthålla regler, bevara historik och hantera förändringar.

Men ju mer vi lär oss att göra med SQL, desto viktigare blir en annan fråga:

**Vad kan en databas inte göra åt oss?**

## Grund

### Databasen är inte verkligheten

Vi kan börja med en rad ur bokhandelns produkttabell:

| id | title | price | stock |
|---|---|---|---|
| 1 | Vägen hem | 200.00 | 12 |

Raden säger att produkten med id `1` heter `Vägen hem`, kostar 200 kronor och har ett lagersaldo på 12 exemplar.

Men den **är inte boken**. Den är en representation av något i bokhandelns verksamhet.

Om en anställd hittar ytterligare fem exemplar på lagret förändras inte databasen automatiskt. Någon eller något behöver registrera förändringen.

Och om databasen säger att det finns 12 exemplar trots att det bara finns 11 på hyllan är det verkligheten och databasen som inte längre stämmer överens.

Det här är ingen liten teknisk detalj. Det är en grundläggande skillnad mellan **ett sakförhållande** och **vår beskrivning av det**.

## En datatyp bestämmer inte betydelsen

Vi kan lagra priset som `DECIMAL(10, 2)` och lagersaldot som `INTEGER`.

Det hjälper oss att bestämma vilka slags värden kolumnerna kan innehålla.

Men det talar inte om för oss vad **pris** egentligen betyder.

Är det priset inklusive moms? Gäller det i butik eller på webbplatsen? Är det ordinarie pris eller kampanjpris?

På samma sätt kan `stock = 12` betyda tolv fysiska exemplar på lagret – men kanske inte tolv exemplar som är tillgängliga för nya beställningar. Några kan redan vara reserverade.

Datatyper beskriver hur värden kan representeras och behandlas. **Betydelsen behöver vi definiera utifrån sammanhanget.**

## En Query kan ge rätt svar på fel fråga

Vi har lärt oss att skriva Queries som räknar, filtrerar och sammanfattar.

Anta att bokhandeln vill veta sin omsättning.

Vi kan skriva:

```
SELECT
    SUM(quantity * unit_price) AS total_amount
FROM order_items;
```

SQL kan beräkna summan utan problem.

Men vad har vi egentligen räknat?

Vi har summerat orderradernas registrerade värden. Vi har inte undersökt om beställningarna betalats, avbrutits eller returnerats. Vi har inte heller bestämt hur rabatter och moms ska hanteras.

Beräkningen kan vara **tekniskt korrekt** och ändå besvara en annan fråga än den verksamheten avsåg.

Databasen kan utföra den beräkning vi beskriver. Den kan inte själv avgöra vilken definition av omsättning vi borde använda.

## Databasen är inte applikationen

Vi har använt databasen för att registrera beställningar och minska lagersaldon.

Men ett köp är mer än dessa förändringar.

Kunden ska kunna välja varor, förstå priset, genomföra betalning och få besked om vad som händer. Bokhandeln kan behöva kontakta leverantörer, organisera leverans och hantera returer.

Delar av detta innebär att läsa eller förändra data. Andra delar handlar om **arbetsflöden, beslut och kommunikation med människor eller andra system**.

En databas kan stödja sådana processer. Det betyder inte att alla processer bör byggas in i databasen.

Det är ofta bättre att låta databasen ansvara för informationens struktur och integritet, medan applikationen organiserar hur verksamhetens handlingar genomförs.

Gränsen är inte absolut. Vi har redan sett att en verksamhetsregel ibland behöver uttryckas i både applikationen och databasen.

### Databasen är inte en garanti för sanningen

Vi kan skydda en kolumn med `NOT NULL`, `UNIQUE` och `CHECK`.

Det ger viktiga garantier, men bara för de regler vi faktiskt har formulerat.

Vi kan exempelvis förhindra ett negativt lagersaldo:

```
CHECK (stock >= 0)
```

Ändå kan databasen innehålla ett felaktigt positivt värde.

En databas kan kontrollera att data följer en modell. Den kan inte på egen hand kontrollera att **modellen och uppgifterna motsvarar verkligheten**.

Därför behöver vi också människor, källor, rutiner och kvalitetsarbete.

## Avancerat

### Modellen väljer vad vi ser

I en databasmodell behöver vi bestämma vilka företeelser som ska ha egna tabeller, vilka egenskaper de har och vilka relationer som är viktiga.

Det innebär också att vi väljer bort något.

Bokhandelns modell beskriver kunder, produkter, författare och beställningar. Den säger däremot mycket lite om varför en kund väljer en viss bok, vad läsaren upplever eller hur en anställd bedömer en bok som litteratur.

Det är inte nödvändigtvis en brist. Modellen är byggd för vissa ändamål.

Men vi bör komma ihåg att **det som inte finns i modellen inte automatiskt är oviktigt**.

En databas är alltid en avgränsad beskrivning av verkligheten.

## Verksamhetsregler är inte alltid databasregler

Vi kan uttrycka många regler med nycklar och begränsningar:

```
CHECK (quantity > 0)
```

Det säger att en orderrad ska innehålla ett positivt antal.

Men tänk att bokhandeln har regeln att en kund får reservera högst tre exemplar av en efterfrågad bok under en kampanj.

Gäller begränsningen per beställning, per kund, per dygn eller för hela kampanjperioden? Räknas tidigare annullerade beställningar?

Innan vi kan implementera regeln måste vi förstå vad den betyder.

Vissa regler kan uttryckas direkt i databasens struktur. Andra behöver kontroller över flera händelser eller sammanhang och måste hanteras i transaktioner, applikationslogik eller särskilda processer.

Databasen kan hjälpa oss att upprätthålla ett beslut. Den kan inte fatta alla **begreppsliga beslut** åt oss.

## Historik är inte samma sak som en ändrad rad

Vi har sett att en produkt kan få ett nytt pris:

```
UPDATE products
SET price = 225.00
WHERE id = 1;
```

Nu beskriver `products.price` det aktuella priset.

Men instruktionen säger inte varför priset förändrades, vem som fattade beslutet eller vilket pris som gällde vid en tidigare tidpunkt.

För att besvara sådana frågor behöver vi **modellera historiken**: exempelvis orderradens pris vid köptillfället, tidsperioder för en prislista eller en logg över förändringar.

Historik uppstår alltså inte automatiskt bara för att vi använder ett databassystem som kan spara och förändra data.

Vi behöver bestämma **vilka förändringar som är meningsfulla att bevara**.

## En View är ett perspektiv, inte en ny verklighet

Vi lärde oss att Views kan ge namn åt återanvändbara Queries och fungera som logiska modeller.

En View kan visa produkter tillsammans med produkttyper, eller sammanfatta lagervärden.

Men den kan inte göra sin information sannare bara genom att presentera den tydligare.

Två Views kan utgå från samma tabeller och ändå ge olika resultat därför att de använder olika urval, grupperingar eller definitioner.

Det betyder inte att en av dem måste vara fel. De kanske beskriver olika perspektiv.

Vi behöver kunna fråga oss **vilka antaganden som ligger bakom varje representation**.

## Det går inte att fråga fram information som saknas

En Query kan vara hur välskriven som helst, men den kan inte återställa ett sakförhållande som aldrig registrerades och inte kan härledas från annan information.

Om vi bara sparar produktens nuvarande pris kan vi inte i efterhand säkert avgöra vad det kostade förra året.

Om vi bara sparar ett kundnamn utan en tillförlitlig identifierare kan vi inte alltid avgöra om två poster beskriver samma person.

Och om vi bara sparar försäljning per månad kan vi inte återskapa varje enskild orderrad.

Det är därför datamodellering påverkar **vilka frågor som över huvud taget går att besvara senare**.

## Lagring är inte förståelse

En dataplattform kan samla information från bokhandelns webbplats, kassasystem, lager och leverantörer.

Men när data förs samman behöver vi fortfarande förstå vad de olika systemen menar.

Två system kan använda samma ord för olika saker. Eller olika ord för samma företeelse.

Det är möjligt att skriva SQL som tekniskt förenar deras uppgifter, utan att de därför har blivit **begreppsligt jämförbara**.

En större mängd data ger inte automatiskt bättre förståelse.

Vi behöver gemensamma definitioner, kända källor och en medvetenhet om vilka skillnader vi förenklar när information sammanställs.

## Är en databas en källa till kunskap?

Vi kan använda databasen för att ta reda på vilka produkter som har sålts, vilka kunder som finns registrerade och hur lagersaldot förändrats.

Sådana svar kan vara viktiga underlag för kunskap.

Men skillnaden mellan **data**, **information** och **kunskap** är inte bara en fråga om tekniska nivåer.

För att förstå ett resultat behöver vi ofta veta hur uppgifterna samlades in, vilka antaganden som gjordes och vilket sammanhang vi tolkar dem i.

Bokhandelns databas kan visa att försäljningen av en produkt ökade. Den säger inte automatiskt *varför*.

Vi kanske behöver undersöka en kampanj, en recension, en leveransförsening eller något annat som inte finns i databasen.

**SQL hjälper oss att ställa precisa frågor till lagrad information. Att förstå svarens betydelse är ett större arbete.**

## Vad återstår när vi kan SQL?

Vi har lärt oss att skapa tabeller, formulera Queries, modellera relationer, förändra data, skydda information och analysera den.

Det ger oss många tekniska möjligheter.

Men innan vi börjar skriva SQL behöver vi fortfarande kunna ställa några enklare frågor:

**Vad är det vi vill beskriva?**

**Vad betyder uppgifterna?**

**Vilka samband behöver vi bevara?**

**Vad behöver vara sant för att informationen ska vara användbar?**

**Vilka frågor vill vi kunna besvara – nu och i framtiden?**

Ju bättre vi förstår detta, desto mer meningsfull blir också vår SQL.

Och när ett informationsbehov förändras behöver vi vara beredda att förändra vår modell, våra regler och ibland också vår uppfattning om vad som är viktigt att registrera.

Vi började med att konstatera att människor alltid har behövt hålla reda på saker.

Databaser är ett av våra mest användbara verktyg för det. Men de är fortfarande **verktyg för att beskriva och arbeta med information**.

De ersätter inte verkligheten, våra beslut eller vårt ansvar att förstå den.

## Reflektion

1. Kan en Query ge ett korrekt resultat som ändå leder till ett felaktigt beslut? Vad skulle det bero på?
2. Hur påverkar datamodellen vilka saker vi senare kan förstå – och vilka frågor vi aldrig kommer att kunna besvara?

3. Var går gränsen mellan det ansvar vi kan lägga på databasen och det ansvar människor och verksamheter alltid behöver behålla?

---

1. För bästa kompatibilitet mellan olika SQL-miljöer skriver vi ett blanksteg efter `--`. MySQL kräver att `--` följs av ett blanktecken eller kontrolltecken, medan vissa andra databassystem är mindre restriktiva.↩
2. PostgreSQL har en boolesk datatyp. I MySQL är `BOOLEAN` ett alias för `TINYINT(1)`, och SQLite representerar vanligtvis booleska värden med heltalen 0 och 1. SQL Server använder `BIT`. Vi använder `BOOLEAN` som ett begripligt, standardnära begrepp här.↩
3. PostgreSQL stöder exempelvis sammansatta typer (*composite types*). Detta är något annat än att bara skapa en tabell som representerar en entitet.↩
4. Skiftlägeskänslighet hos `LIKE` varierar mellan databassystem och inställningar, exempelvis kollationer. Räkna därför inte med att `'B%'` alltid ger exakt samma träffar i alla miljöer.↩
5. MySQL, PostgreSQL och SQLite använder `LIMIT`. Standard-SQL har bland annat `FETCH FIRST ... ROWS ONLY` och `OFFSET ... FETCH`. SQL Server har också `TOP`.↩
6. `CHAR_LENGTH` används i MySQL och PostgreSQL; SQL Server använder exempelvis `LEN`, och SQLite använder `length`. Syntaxen för `SUBSTRING` varierar också. Exempelvis används ofta `SUBSTRING(text, start, antal)` i MySQL. Textsammanfogning med `CONCAT` eller `||` varierar mellan dialekterna.↩
7. Datatypernas namn och reglerna för automatisk konvertering varierar mellan databassystem. Även stöd för och beteende hos `CAST` kan skilja sig mellan olika typer.↩
8. `DATETIME` finns i bland annat MySQL och SQL Server men är inte en gemensam typ med identiskt beteende i alla SQL-system. `TIMESTAMP` finns i SQL-standarden, men bland annat MySQL

och PostgreSQL behandlar tidszoner olika. Vi går igenom den skillnaden under *Avancerat*.↩

9. MySQL och PostgreSQL har `NOW()`. `CURRENT_TIMESTAMP` är standardnära och stöds även av SQLite och SQL Server, men precision och exakt tidssemantik kan skilja sig åt.↩
10. Exemplet använder MySQL-syntax. PostgreSQL kan exempelvis subtrahera två `DATE`-värden direkt, SQL Server använder `DATEDIFF` med ett argument för tidsenheten och SQLite har andra funktioner för datumberäkningar. Se respektive dialektreferens.↩
11. Vissa SQL-system accepterar särskilda undantag eller mer tillåtande grupperingsregler. MySQL använder exempelvis normalt `ONLY_FULL_GROUP_BY`, vilket begränsar tvetydiga frågor. SQLite tillåter vissa icke-grupperade kolumner men resultatet kan då bli missvisande. Vi håller oss till den tydligare, standardnära principen.↩
12. SQLite lade till `RIGHT JOIN` och `FULL OUTER JOIN` i version 3.39.0. MySQL har `LEFT JOIN` och `RIGHT JOIN` men inte ett direkt `FULL OUTER JOIN`-uttryck. Mer om detta i dialekthäftena.↩
13. Äldre MySQL-versioner saknar `INTERSECT` och `EXCEPT`. Exemplen i detta kapitel förutsätter en SQL-miljö som stöder dem. För äldre versioner behöver motsvarande informationsbehov uttryckas på andra sätt, till exempel med `EXISTS` och `NOT EXISTS`.↩
14. En del databasimplementationer har dessutom särskilda regler kring primärnycklar, `NULL` och begränsningar. Exempelvis kräver vissa SQLite-tabellformer en uttrycklig `NOT NULL` för att en icke-heltals-primärnyckel alltid ska avvisa `NULL`. Det här kapitlet beskriver den avsedda, standardnära modellen.↩
15. I äldre MySQL-versioner kunde `CHECK`-syntax accepteras utan att villkoren upprätthölls. Kontrollera version och databassystem när reglerna är kritiska.↩

16. Många SQL-system tillåter flera `NULL`-värden i en `UNIQUE`-kolumn eftersom okända värden inte behandlas som identiska. SQL Server har andra standardregler för enkla unika begränsningar. Kollationer kan dessutom påverka om olika textvärden räknas som samma.↩
17. Stödet för referensåtgärder och detaljerna kring `RESTRICT`, `NO ACTION`, `SET NULL` och `CASCADE` skiljer sig något mellan SQL-system. I MySQL behöver tabellerna bland annat använda en lagringsmotor som stöder främmande nycklar, exempelvis InnoDB.↩
18. William Kent, *A Simple Guide to Five Normal Forms in Relational Database Theory*, Communications of the ACM 26(2), 1983, s. 120–125. Tillgänglig på https://knowledge.kitchen/content/courses/database-design/normal-forms-simple-guide/.↩
19. PostgreSQL och SQLite stöder `RETURNING` vid flera datamanipulerande satser, medan SQL Server har `OUTPUT`. Stöd och syntax varierar mellan databassystem och versioner.↩
20. `START TRANSACTION` används i MySQL. Andra databassystem kan exempelvis använda `BEGIN`. Transaktionsstöd och exakt beteende beror också på databasens lagringsmotor och inställningar.↩
21. Syntaxen `ADD COLUMN` stöds brett, men detaljer och möjligheterna att lägga till `NOT NULL`, standardvärden eller vissa begränsningar skiljer sig mellan MySQL, PostgreSQL, SQLite och SQL Server.↩
22. PostgreSQL använder exempelvis `ALTER COLUMN ... TYPE`. SQL Server använder `ALTER COLUMN` med en datatypsdefinition, medan SQLite har mer begränsade möjligheter att ändra befintliga kolumner direkt. MySQL-exemplet ovan använder `MODIFY COLUMN`.↩
23. Bland annat PostgreSQL, moderna MySQL-versioner och SQLite stöder `RENAME COLUMN`. SQL Server använder i regel `sp_rename` för att byta namn på en kolumn.↩

24. Exakt stöd för transaktionell DDL varierar mellan databassystem och mellan olika operationer. En rollback-strategi för en migration bör därför testas i den faktiska databasmiljön.↩↩
25. MySQL har exempelvis vyer med `SQL SECURITY DEFINER` eller `INVOKER`, och PostgreSQL har andra regler och möjligheter för säkerhetskontext. Vi återkommer till åtkomstkontroll i säkerhetskapitlet.↩
26. Vissa databassystem erbjuder särskilda mekanismer, exempelvis `INSTEAD OF`-triggers, för att definiera hur skrivningar genom mer komplexa vyer ska hanteras. Vilka vyer som automatiskt kan uppdateras skiljer sig åt.↩
27. PostgreSQL har inbyggt stöd för materialiserade vyer med `CREATE MATERIALIZED VIEW` och `REFRESH MATERIALIZED VIEW`. SQL Server har indexerade vyer med särskilda krav. MySQL har inte motsvarande allmänna inbyggda materialiserade vyfunktion, utan liknande lösningar byggs vanligtvis på annat sätt.↩
28. PostgreSQL kan exempelvis slå samman vissa CTE:er med omgivande Query och stöder `MATERIALIZED` och `NOT MATERIALIZED` i relevanta sammanhang. MySQL och SQL Server har andra optimeringsstrategier. Läsbarhet och faktisk exekveringsplan bör hållas isär.↩
29. MySQL och PostgreSQL stöder `WITH RECURSIVE`. SQLite stöder också rekursiva CTE:er. SQL Server använder rekursiva CTE:er men skriver normalt `WITH` utan nyckelordet `RECURSIVE` och har bland annat `MAXRECURSION` för att begränsa antal nivåer. Detaljer kring typning, djupgränser och cykelhantering varierar.↩
30. SQL-dialekterna har delvis olika stöd för ramtyper och uttryck. En vanlig standardram vid `ORDER BY` är `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. Den omfattar också likvärdiga sorteringsvärden (*peers*), vilket kan skilja sig från `ROWS`.↩

31. MySQL har stöd för fönsterfunktioner från version 8.0. SQLite införde dem i version 3.25. PostgreSQL och SQL Server har haft stöd under lång tid. Funktioner och syntax för vissa fönsterramar kan skilja sig mellan systemen.↩
32. MySQL med InnoDB använder normalt `REPEATABLE READ` som standard. PostgreSQL använder normalt `READ COMMITTED`. SQL Server har vanligtvis `READ COMMITTED`, men hur läsningar hanteras påverkas bland annat av inställningar för radversionering. SQLite har en annan samtidighetsmodell. Kontrollera alltid den faktiska miljön.↩
33. `SELECT ... FOR UPDATE` stöds bland annat av MySQL/InnoDB och PostgreSQL. SQL Server har andra låsningsmekanismer och syntax, och SQLite har inte samma `FOR UPDATE`-syntax. Exemplet ska köras mot en databas som faktiskt stöder denna låsning.↩
34. MySQL/InnoDB lagrar normalt tabellens data organiserade kring ett klustrat primärnyckelindex. PostgreSQL skapar ett unikt B-trädsindex för en primärnyckel, men tabellens rader lagras normalt separat. SQL Server kan använda klustrade eller icke-klustrade index.↩
35. MySQL, PostgreSQL, SQLite och SQL Server har verktyg för att se exekveringsplaner, men kommandon och presentationsformat varierar. MySQL och PostgreSQL använder exempelvis `EXPLAIN`, SQLite erbjuder `EXPLAIN QUERY PLAN` och SQL Server har egna sätt att visa uppskattade och faktiska exekveringsplaner.↩
36. MySQL/InnoDB kräver index som stöd för främmande nyckelrelationer och kan skapa saknade stödindex automatiskt. PostgreSQL skapar däremot inte automatiskt index på den refererande kolumnen enbart för att en främmande nyckel definieras.↩
37. PostgreSQL erbjuder `EXPLAIN ANALYZE` och nyare MySQL-versioner har `EXPLAIN ANALYZE`. De kör normalt Queryn och

mäter exekveringen. För dataförändrande satser behöver detta särskild försiktighet eftersom själva förändringen kan utföras.↩

38. `MySqlConnector` är en fristående ADO.NET-drivrutin för MySQL, tillgänglig via NuGet. Andra databassystem har andra drivrutiner, exempelvis `Npgsql` för PostgreSQL och `Microsoft.Data.SqlClient` för SQL Server. Metodnamn och beteende kan skilja sig något mellan drivrutiner.↩
39. `CREATE USER` och hantering av identiteter varierar mellan databassystem. Exemplet använder MySQL:s kontomodell. I andra miljöer kan roller, externa identitetsleverantörer eller serverövergripande inloggningar användas.↩
40. MySQL 8 stöder roller, men tilldelade roller är inte alltid automatiskt aktiva. PostgreSQL och SQL Server har andra rollmodeller och syntaxer för tilldelning och aktivering. Kontrollera alltid de faktiska effektiva rättigheterna.↩
41. I MySQL kan vyer exempelvis använda `SQL SECURITY DEFINER` eller `INVOKER`. Det påverkar vilken identitets behörigheter som används för åtkomst till underliggande objekt. Vyer är inte automatiskt en fullständig säkerhetsgräns och kan behöva kompletteras med andra mekanismer.↩
42. Funktioner, begränsningar och säkerhetsmodell för radbaserade behörigheter är databasspecifika. Rätt utformning kräver bland annat förståelse för hur identitet och sessionskontext förs vidare från applikationen.↩
43. Se OWASP Password Storage Cheat Sheet. Där beskrivs rekommenderade algoritmer, salt, parametrar och migrering av äldre lösenordshashar. Vanlig SHA-256, även med ett enkelt salt, är inte en ersättning för lösenordshashning.↩
44. Se OWASP Cryptographic Storage Cheat Sheet för vägledning om kryptering i applikations-, databas- och lagringslager samt säker hantering av nycklar och krypteringslägen.↩
45. Se OWASP Key Management Cheat Sheet. Att byta nyckel kräver även hänsyn till äldre säkerhetskopior,

återställningsförmåga och hur gamla nycklar slutligen ska avvecklas.↩

46. PostgreSQL har bland annat `DATE_TRUNC('month', ...)`; SQL Server har andra möjligheter att gruppera på år och månad. Hur datum och tidszoner tolkas kan också påverka resultatet.↩
47. PostgreSQL har bland annat `json` och `jsonb`. SQLite har JSON-funktioner och stöd för JSON-representationer men inte samma särskilda JSON-kolumntyp som MySQL. SQL Server har länge haft JSON-funktioner över textvärden och stöd för en särskild `json`-typ varierar med version och produktvariant.↩
48. Funktioner för JSON-validering är databasspecifika. Även om ett JSON-värde är syntaktiskt giltigt säger det inte automatiskt att egenskaperna följer verksamhetens regler.↩
49. MySQL 8 använder spatiala referenssystem och har särskilda regler för axelordningen när geometrier skapas från WKT. Alternativet `axis-order=long-lat` gäller bland annat `ST_GeomFromText()`. Syntax och axelhantering skiljer sig mellan databassystem. I PostGIS används ofta `ST_SetSRID(ST_MakePoint(longitude, latitude), 4326)`.↩
50. `ST_Distance_Sphere()` är en MySQL-funktion för sfäriskt avstånd. PostgreSQL med PostGIS har exempelvis `ST_Distance` för olika geometri- och geografityper samt `ST_DistanceSphere`. Funktionernas måttenheter och beräkningsmodeller kan skilja sig åt.↩
51. Spatiala predikat och detaljerna i deras topologiska betydelse följer särskilda geometriska regler. `ST_Covers()` finns exempelvis i PostGIS, men inte som en allmänt portabel ersättning i alla SQL-system. Kontrollera också koordinatreferenssystem och geometriernas giltighet.↩